

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

**ỨNG DỤNG THUẬT TOÁN A* TỐI ƯU QUỶ
ĐẠO ĐƯỜNG ĐI CHO ROBOT TỰ HÀNH**

NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

Sinh viên: **ĐINH QUANG DŨNG**

MSSV: 22161230

NGUYỄN QUỐC BẢO

MSSV: 22161221

Hướng dẫn: **TS. TRƯƠNG QUANG PHÚC**

TP. HỒ CHÍ MINH – 6/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT THÀNH PHỐ HỒ CHÍ MINH

KHOA ĐIỆN - ĐIỆN TỬ

BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

ỨNG DỤNG THUẬT TOÁN A* TỐI ƯU QUỲ ĐẠO ĐƯỜNG ĐI CHO ROBOT TỰ HÀNH

NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

Sinh viên: **ĐINH QUANG DŨNG**

MSSV: 22161230

NGUYỄN QUỐC BẢO

MSSV: 22161221

Hướng dẫn: **TS. TRƯƠNG QUANG PHÚC**

TP. HỒ CHÍ MINH – 6/2026

THÔNG TIN KHÓA LUẬN TỐT NGHIỆP

1. Thông tin sinh viên

Họ và tên sinh viên: Đinh Quang Dũng	MSSV: 22161230
Email: 22161230@student.hcmute.edu.vn	Điện thoại: 0948803703
Họ và tên sinh viên: Nguyễn Quốc Bảo	MSSV: 22161221
Email: 22161221@student.hcmute.edu.vn	Điện thoại: 0352300254

2. Thông tin đề tài

- Tên đề tài: Ứng dụng thuật toán A* tối ưu quỹ đạo đường đi cho robot tự hành
- Đơn vị quản lý: Bộ môn Kỹ Thuật Điện tử - Thông tin, Khoa Điện Điện Tử, Trường Đại Học Sư Phạm Kỹ Thuật Tp. Hồ Chí Minh.
- Thời gian thực hiện: Từ ngày 2 / 3 / 2026 đến ngày 13 / 6 / 2026
- Thời gian bảo vệ trước hội đồng: Ngày 3 / 7 / 2026

3. Lời cam đoan của sinh viên

Tôi Đinh Quang Dũng, Nguyễn Quốc Bảo cam đoan KLTN là công trình nghiên cứu của tôi, dưới sự hướng dẫn của Tiến sĩ Trương Quang Phúc. Kết quả công bố trong KLTN là trung thực và không sao chép từ bất kì công trình nào khác.

Tp. HCM, ngày ... tháng ... năm 20...

SV thực hiện đồ án
(Ký và ghi rõ họ tên)

.....

Giảng viên hướng dẫn xác nhận quyền báo cáo đã được chỉnh sửa theo đề nghị được ghi trong biên bản của Hội đồng đánh giá Khóa luận tốt nghiệp.

.....

Tp. HCM, ngày ... tháng ... năm 20..

Xác nhận của Bộ Môn

Giảng viên hướng dẫn
(Ký, ghi rõ họ tên và học hàm – học vị)

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: ỨNG DỤNG THUẬT TOÁN A* TỐI ƯU QUỶ ĐẠO ĐƯỜNG ĐI CHO ROBOT TỰ HÀNH

Sinh viên: + Đinh Quang Dũng

MSSV: 22161230

+ Nguyễn Quốc Bảo

MSSV: 22161221

Hướng dẫn: TS. Trương Quang Phúc

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

Đặt vấn đề rõ ràng, có tính mới, và phù hợp nhu cầu thực tiễn.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

Phương pháp thực hiện hợp lý dựa trên cơ sở lý thuyết, có phân tích, so sánh, và đánh giá.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá / kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

Kết quả có phân tích, đánh giá, kiểm định chặt chẽ, đảm bảo độ tin cậy.

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

Kết luận phù hợp, thuyết phục, có đóng góp mới.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

Bố cục hợp lý, cấu trúc rõ ràng, trình bày đúng định dạng mẫu, thể hiện năng lực tốt.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

Kỹ năng giao tiếp và làm việc nhóm chuyên nghiệp.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

Trích dẫn tài liệu tham khảo trung thực, đúng quy định.

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

Tỉ lệ tương đồng theo đúng quy định của nhà Trường.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

Các kết quả thực hiện trên quy mô nhỏ, cần kiểm chứng trên phạm vi rộng hơn để đánh giá khả năng đáp ứng của thiết kế.

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Tinh thần làm việc nghiêm túc, cầu thị.

Đề nghị của giảng viên hướng dẫn

Ghi rõ: “Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp”

Báo cáo đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, được phép bảo vệ.

Tp. HCM, ngày 23 tháng 06 năm 2026

Người nhận xét

(Ký và ghi rõ họ tên)

Trương Quang Phúc

LỜI CẢM ƠN

Trước hết, nhóm em xin gửi lời cảm ơn chân thành đến quý thầy cô Trường Đại học Công Nghệ Kỹ Thuật thành phố Hồ Chí Minh, đặc biệt là quý thầy cô Khoa Điện - Điện tử và Bộ môn Kỹ thuật Điện tử - Thông tin đã tận tình giảng dạy, truyền đạt kiến thức chuyên môn và tạo điều kiện thuận lợi trong suốt quá trình học tập tại trường. Những kiến thức nền tảng về điện tử, viễn thông, vi mạch, và hệ thống nhúng là cơ sở quan trọng giúp nhóm em thực hiện đề tài đồ án tốt nghiệp này.

Nhóm em xin bày tỏ lòng biết ơn sâu sắc đến Thầy Trương Quang Phúc, người đã trực tiếp hướng dẫn, định hướng và góp ý trong quá trình thực hiện đề tài. Sự hướng dẫn tận tình của thầy đã giúp chúng em có thêm cơ sở để tiếp cận bài toán một cách khoa học, từ việc nghiên cứu lý thuyết, cho đến quá trình thiết kế, thi công và thử nghiệm mô hình robot tự hành trong thực tế.

Mặc dù đã cố gắng trong quá trình thực hiện đồ án tốt nghiệp, vẫn không thể tránh khỏi những thiếu sót do giới hạn về thời gian, kinh nghiệm nghiên cứu và điều kiện thực nghiệm. Chúng em kính mong nhận được những ý kiến đóng góp quý báu từ quý thầy cô để đề tài được hoàn thiện hơn trong tương lai.

TÓM TẮT

Trong những năm gần đây, robot tự hành được ứng dụng rộng rãi trong các lĩnh vực vận chuyển, giám sát và tự động hóa nhờ khả năng hoạt động độc lập trong môi trường thực tế. Để thực hiện nhiệm vụ này, robot cần có khả năng nhận thức môi trường, xây dựng bản đồ, xác định vị trí và lập kế hoạch di chuyển đến mục tiêu. Trong số các phương pháp tìm đường hiện nay, thuật toán A* được sử dụng phổ biến nhờ khả năng tìm kiếm hiệu quả trên bản đồ dạng lưới. Tuy nhiên, khi triển khai trên các hệ thống robot thực tế, yêu cầu giảm thời gian xử lý và chi phí tính toán vẫn là vấn đề cần được quan tâm.

Đề tài tập trung nghiên cứu, thiết kế và thi công mô hình robot tự hành bốn bánh hoạt động trong môi trường trong nhà. Hệ thống sử dụng máy tính nhúng làm bộ xử lý trung tâm, kết hợp với các cảm biến và các phần tử điều khiển chuyển động để thu thập dữ liệu và điều hướng robot. Nền tảng phần mềm được xây dựng trên ROS 2, tích hợp các chức năng tạo bản đồ bằng SLAM, định vị và điều hướng thông qua Nav2. Trên cơ sở đó, phương pháp tìm đường được cải tiến bằng cách áp dụng thêm trọng số W nhằm giảm số lượng nút cần mở rộng trong quá trình tìm kiếm.

Kết quả thực nghiệm cho thấy mô hình robot có khả năng xây dựng bản đồ 2D, định vị vị trí hiện tại và di chuyển đến đích trên bản đồ môi trường. Việc áp dụng trọng số W giúp giảm thời gian lập kế hoạch và số lượng nút mở rộng so với phương pháp truyền thống, trong khi quỹ đạo tạo ra vẫn đáp ứng yêu cầu điều hướng của robot. Kết quả đạt được cho thấy tính khả thi của giải pháp trong việc nâng cao hiệu quả lập kế hoạch đường đi cho robot tự hành hoạt động trong môi trường trong nhà.

MỤC LỤC

DANH MỤC HÌNH ẢNH	VIII
DANH MỤC BẢNG	X
CÁC TỪ VIẾT TẮT	XII
CHƯƠNG 1 TỔNG QUAN.....	1
1.1 GIỚI THIỆU	1
1.2 TÌNH HÌNH NGHIÊN CỨU TRONG VÀ NGOÀI NƯỚC	1
1.2.1 Trong nước	1
1.2.2 Ngoài nước	2
1.3 MỤC TIÊU ĐỀ TÀI	2
1.4 PHƯƠNG PHÁP NGHIÊN CỨU	3
1.5 GIỚI HẠN ĐỀ TÀI.....	3
1.6 BỐ CỤC ĐỒ ÁN	4
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT	5
2.1 ROBOT DI ĐỘNG TỰ HÀNH	5
2.1.1 Tổng quan về robot di động tự hành	5
2.1.2 Các loại robot di động tự hành phổ biến	5
2.1.3 Ứng dụng của xe tự hành.....	6
2.1.4 Các thách thức và công nghệ hỗ trợ xe tự hành	7
2.1.5 Mô hình động học của Robot di động tự hành	7
2.2 HỆ ĐIỀU HÀNH ROS 2.....	9
2.2.1 Khái niệm	9
2.2.2 Kiến trúc của ROS 2.....	10
2.2.3 Các thành phần chính trong ROS 2	11
2.2.4 ROS 2 Humble Hawksbill	12
2.2.5 Ứng dụng ROS 2 trong thực tiễn.....	12
2.3 GIỚI THIỆU VỀ HỆ THỐNG SLAM.....	13
2.3.1 Khái quát về SLAM.....	13
2.3.2 Quy trình xử lý SLAM trong ROS 2	13

2.3.3	Một số gói SLAM phổ biến trong ROS 2	14
2.3.4	Ứng dụng của SLAM trong ROS 2	14
2.4	TỔNG QUAN VỀ NAV2	15
2.4.1	Khái quát về Nav2	15
2.4.2	Kiến trúc và các thành phần chính trong Nav2	15
2.4.3	Yêu cầu hệ thống của Nav2.....	16
2.5	TỔNG QUAN VỀ THUẬT TOÁN A* VÀ WEIGHTED A*	17
2.5.1	Bài toán lập kế hoạch đường đi cho robot tự hành.....	17
2.5.2	Nguyên lý hoạt động của thuật toán A*	17
2.5.3	Hàm đánh giá chi phí trong thuật toán A*	20
2.5.4	Hàm heuristic trong thuật toán A*	22
2.5.5	Ưu điểm và hạn chế của thuật toán A*	23
2.5.6	Thuật toán Weighted A*	23
2.5.7	So sánh giữa thuật toán A* và Weighted A*	24
2.6	HỆ THỐNG ĐIỀU HƯỚNG CHO ROBOT DI ĐỘNG TỰ HÀNH	27
2.6.1	Tổng quan về hệ thống điều hướng.....	27
2.6.2	Định vị, xây dựng bản đồ và lập kế hoạch đường đi.....	27
2.7	TỔNG QUAN VỀ PID	28
2.7.1	Khái niệm về PID	28
2.7.2	Cấu trúc và công thức của PID.....	29
2.7.3	Vai trò từng khâu của PID.....	29
2.8	GIỚI THIỆU LINH KIỆN	30
2.8.1	Động cơ JGB37-520.....	30
2.8.2	IC điều khiển động cơ TB6612FNG	30
2.8.3	Cảm biến Lidar A1M8.....	31
2.8.4	Cảm biến gia tốc góc BNO080.....	33
2.8.5	Mạch thu sóng RF và tay cầm RF	34
2.8.6	Bộ mã hóa.....	35
2.8.7	IC chuyển đổi USB UART.....	36
2.8.8	IC mở rộng chân PCF8575	37

2.8.9	IC hạ áp XL4016	38
2.8.10	IC ổn áp AMS1117-3.3	39
2.8.11	Relay	40
2.8.12	Raspberry pi 5.....	41
2.8.13	Esp32-s3	42
CHƯƠNG 3	THIẾT KẾ HỆ THỐNG.....	44
3.1	YÊU CẦU CỦA HỆ THỐNG	44
3.2	MÔ HÌNH HỆ THỐNG	44
3.2.1	Sơ đồ khối tổng quát hệ thống.....	45
3.2.2	Nguyên lý hoạt động tổng quát của hệ thống.....	46
3.3	THIẾT KẾ PHẦN CỨNG	46
3.3.1	Chức năng của phần cứng	46
3.3.2	Sơ đồ khối phần cứng.....	47
3.3.3	Khối động cơ	48
3.3.4	Khối thiết bị ngoại vi.....	50
3.3.5	Khối xử lý trung tâm	54
3.3.6	Khối nguồn	56
3.3.7	Thiết kế mạch điều khiển	59
3.4	THIẾT KẾ PHẦN MỀM	62
3.4.1	Cấu trúc hệ thống phần mềm.....	62
3.4.2	Hệ tọa độ và cây TF của hệ thống	63
3.4.3	Tính toán odometry và hợp nhất dữ liệu cảm biến.....	64
3.4.4	Hệ thống xây dựng bản đồ bằng SLAM Toolbox	65
3.4.5	Hệ thống định vị và điều hướng bằng Nav2.....	66
3.5	THUẬT TOÁN A* VÀ WA* TRONG LẬP KẾ HOẠCH ĐƯỜNG ĐI.....	68
3.5.1	Mô hình bài toán tìm đường đi trên bản đồ chi phí.....	68
3.5.2	Thuật toán A* truyền thống.....	69
3.5.3	Thuật toán WA*	71
3.5.4	Quy trình thực hiện thuật toán WA* trong hệ thống robot	73
3.6	TÍCH HỢP VÀ VẬN HÀNH HỆ THỐNG	73

3.6.1	Quy trình khởi động hệ thống	74
3.6.2	Quy trình vận hành robot trong các chế độ làm việc	75
CHƯƠNG 4	KẾT QUẢ VÀ ĐÁNH GIÁ MÔ HÌNH	76
4.1	KẾT QUẢ MÔ HÌNH THI CÔNG	76
4.2	KẾT QUẢ MÔ PHỎNG	77
4.3	KẾT QUẢ VẼ VÀ LƯU BẢN ĐỒ TRÊN RVIZ 2.....	82
4.4	KẾT QUẢ THỰC NGHIỆM KHẢ NĂNG DI CHUYỂN ĐẾN ĐIỂM ĐÍCH CỦA ROBOT	84
4.5	KẾT QUẢ THỰC NGHIỆM KHI THAY ĐỔI TRỌNG SỐ W.....	87
4.5.1	Thực nghiệm tại vị trí Goal 1	87
4.5.2	Thực nghiệm tại vị trí Goal 2	90
4.5.3	Thực nghiệm tại vị trí Goal 3	92
4.6	KẾT QUẢ THỰC NGHIỆM KHẢ NĂNG TRÁNH VẬT CẢN	95
CHƯƠNG 5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	98
5.1	KẾT LUẬN	98
5.2	HƯỚNG PHÁT TRIỂN.....	99
PHỤ LỤC		100
TÀI LIỆU THAM KHẢO		101

DANH MỤC HÌNH ẢNH

Hình 2.1 Xe tự động dẫn đường.....	5
Hình 2.2 Robot di động tự hành.....	6
Hình 2.3 Mô hình động học của robot dẫn động vi sai.....	8
Hình 2.4 Kiến trúc ROS 2	10
Hình 2.5 Mô hình truyền thông giữa các node trong ROS 2	11
Hình 2.6 Kiến trúc của Nav2.....	16
Hình 2.7 Minh họa quá trình tìm kiếm của thuật toán A* trên đồ thị đơn giản.....	20
Hình 2.8 Biểu đồ độ phức tạp thuật toán	26
Hình 2.9 Động cơ JGB37-520.....	30
Hình 2.10 IC driver TB6612FNG	31
Hình 2.11 Cảm biến lidar A1M8.....	32
Hình 2.12 Cảm biến gia tốc góc BNO080	33
Hình 2.13 Mạch thu và tay cầm RF	34
Hình 2.14 Sơ đồ chân bộ mã hóa	35
Hình 2.15 IC chuyển đổi USB UART CP2102	36
Hình 2.16 IC mở rộng chân PCF8575	37
Hình 2.17 IC hạ áp XL4016.....	38
Hình 2.18 IC ổn áp AMS 1117-3.3	39
Hình 2.19 Relay	40
Hình 2.20 Board mạch raspberry pi 5	41
Hình 2.21 Sơ đồ chân module Esp32-S3	42
Hình 3.1 Sơ đồ khối tổng quát hệ thống	45
Hình 3.2 Sơ đồ khối hệ thống phần cứng.....	47
Hình 3.3 Sơ đồ nguyên lý kết nối TB6612FNG với ESP32-S3 và PCF8575.....	48
Hình 3.4 Sơ đồ nguyên lý mạch thu RF với ESP32-S3	50
Hình 3.5 Sơ đồ kết nguyên lý nối giữa bộ mã hóa với ESP32-S3 và TB6612FNG	51
Hình 3.6 Sơ đồ chân của raspberry pi 5 và module BNO080.....	53
Hình 3.7 Sơ đồ chân lidar A1M8 và mạch chuyển đổi USB UART	54

Hình 3.8 Sơ đồ nguyên lý mạch ổn áp dùng AMS1117	58
Hình 3.9 Sơ đồ nguyên lý mạch hạ áp 5V dùng XL4016	58
Hình 3.10 Một số cổng kết nối và linh kiện hỗ trợ mạch.....	60
Hình 3.11 Sơ đồ nguyên lý mạch PCB	61
Hình 3.12 Cấu trúc hệ thống phần mềm	62
Hình 3.13 Cây TF của hệ thống	64
Hình 3.14 Odometry và hợp nhất dữ liệu cảm biến	65
Hình 3.15 Trình bày luồng xử lý dữ liệu trong chế độ xây dựng bản đồ.....	66
Hình 3.16 Quá trình lập kế hoạch đường đi.....	66
Hình 3.17 Quá trình bám quỹ đạo và điều khiển chuyển động.....	67
Hình 3.18 Lưu đồ thuật toán A*	70
Hình 3.19 Sơ đồ khối quy trình khởi động robot tự hành	74
Hình 4.1 Kết quả mô hình thi công	76
Hình 4.2 Vị trí bố trí linh kiện bên trong robot.....	77
Hình 4.3 Các quỹ đạo tới Goal 1 khi thay đổi W	78
Hình 4.4 Các quỹ đạo tới Goal 2 khi W thay đổi.....	79
Hình 4.5 Biểu đồ số lượng node được duyệt theo trọng số W	81
Hình 4.6 Biểu đồ thời gian lập kế hoạch theo trọng số W	82
Hình 4.7 Hình ảnh bản đồ thực tế và sau khi được vẽ hoàn chỉnh	83
Hình 4.8 Vị trí bắt đầu và 3 Goal được sử dụng để thực nghiệm thực trong thực tế	84
Hình 4.9 Biểu đồ sai số trung bình của robot tại các vị trí đích.....	86
Hình 4.10 Biểu đồ thời gian di chuyển trung bình của robot tại các vị trí đích.....	86
Hình 4.11 Các quỹ đạo di chuyển tới Goal 1 khi W thay đổi.....	88
Hình 4.12 Các quỹ đạo tới Goal 2 khi W thay đổi.....	90
Hình 4.13 Các quỹ đạo tới Goal 3 khi W thay đổi.....	92
Hình 4.14 Biểu đồ ảnh hưởng của trọng số W đến số lượng node được duyệt	94
Hình 4.15 Biểu đồ ảnh hưởng của trọng số W đến thời gian lập kế hoạch	94
Hình 4.16 Vật cản mới được thêm vào bản đồ	95
Hình 4.17 Quỹ đạo robot khi phát hiện vật cản mới.....	96

DANH MỤC BẢNG

Bảng 2.1 Thông số kỹ thuật động cơ JGB37-520	30
Bảng 2.2 Thông số kỹ thuật TB6612FNG	31
Bảng 2.3 Thông số kỹ thuật cảm biến lidar A1M8	32
Bảng 2.4 Thông số kỹ thuật cảm biến gia tốc góc BNO080.....	33
Bảng 2.5 Mô tả nhóm chân của mạch thu RF	34
Bảng 2.6 Thông số kỹ thuật bộ mã hóa.....	35
Bảng 2.7 Thông số kỹ thuật IC CP2102	36
Bảng 2.8 Thông số kỹ thuật IC PCF8575	37
Bảng 2.9 Thông số kỹ thuật XL4016	38
Bảng 2.10 Thông số kỹ thuật AMS1117-3.3	39
Bảng 2.11 Bảng thông số kỹ thuật Relay	40
Bảng 2.12 Thông số kỹ thuật raspberry pi 5	41
Bảng 2.13 Thông số kỹ thuật ESP32-S3	43
Bảng 3.1 Sơ đồ kết nối chân TB6612FNG trái với ESP32-S3 và PCF8575	49
Bảng 3.2 Sơ đồ kết nối chân TB6612FNG phải với ESP32-S3 và PCF8575.....	49
Bảng 3.3 Sơ đồ kết nối chân mạch thu RF và ESP32-S3	50
Bảng 3.4 Sơ đồ kết nối 4 bộ mã hóa với ESP32-S3 và TB6612fNG	52
Bảng 3.5 Sơ đồ kết nối chân BNO080 với Raspberry pi 5	53
Bảng 3.6 Kết nối giữa ESP32-S3 với các thiết bị ngoại vi	55
Bảng 3.7 Sơ đồ kết nối của raspberry pi 5 với các thiết bị khác.....	56
Bảng 3.8 Công suất tiêu thụ các thiết bị	57
Bảng 3.9 Công suất tiêu thụ ở từng nhánh.....	57
Bảng 4.1 Số node và thời gian lập kế hoạch ở Goal 1	78
Bảng 4.2 Số node và thời gian lập kế hoạch ở Goal 2	80
Bảng 4.3 Kết quả đo đạt khả năng di chuyển tới 3 goal	85
Bảng 4.4 Số node, thời gian vẽ, thời gian di chuyển và chiều và chiều dài quãng đường khi thay đổi W tới Goal 1	88
Bảng 4.5 Số node, thời gian vẽ, thời gian di chuyển và chiều và chiều dài quãng đường khi thay đổi W tới Goal 2	91

Bảng 4.6 Số node, thời gian vẽ, thời gian di chuyển và chiều và chiều dài quãng đường khi thay đổi W tới Goal 3	93
---	----

CÁC TỪ VIẾT TẮT

Viết tắt	Ý nghĩa
AGV	Automated Guided Vehicle
AMR	Autonomous Mobile Robot
AMCL	Adaptive Monte Carlo Localization
BLE	Bluetooth Low Energy
BT	Behavior Tree
CPU	Central Processing Unit
DDS	Data Distribution Service
DOF	Degrees of Freedom
DWA	Dynamic Window Approach
EKF	Extended Kalman Filter
FoV	Field of View
GPIO	General Purpose Input/Output
GPU	Graphics Processing Unit
GPS	Global Positioning System
I2C	Inter-Integrated Circuit
IC	Integrated Circuit
IMU	Inertial Measurement Unit
LiDAR	Light Detection and Ranging
LTS	Long-Term Support
MISO	Master In Slave Out
MOSI	Master Out Slave In
Nav2	Navigation2
PCB	Printed Circuit Board
PID	Proportional–Integral–Derivative
PPR	Pulses Per Revolution
PWM	Pulse Width Modulation

RAM	Random Access Memory
RF	Radio Frequency
RGB-D	Red Green Blue – Depth
ROS	Robot Operating System
ROS 2	Robot Operating System 2
RPM	Revolutions Per Minute
RTAB-Map	Real-Time Appearance-Based Mapping
SLAM	Simultaneous Localization and Mapping
SPI	Serial Peripheral Interface
SRAM	Static Random Access Memory
TEB	Timed Elastic Band
TF	Transform
ToF	Time-of-Flight
UART	Universal Asynchronous Receiver-Transmitter
USB	Universal Serial Bus
VFH	Vector Field Histogram
WA*	Weighted A*
ADC	Analog-to-Digital Converter
CLI	Command Line Interface
DC	Direct Current
ORB-SLAM	Oriented FAST and Rotated BRIEF - Simultaneous Localization and Mapping
RViz2	Robot Visualization 2
SSH	Secure Shell
TF2	Transform Library 2
UI	User Interface

CHƯƠNG 1

TỔNG QUAN

1.1 GIỚI THIỆU

Trong những năm gần đây, robot tự hành ngày càng được nghiên cứu và ứng dụng rộng rãi trong các lĩnh vực như vận chuyển, giám sát và hỗ trợ con người trong môi trường trong nhà. Để hoạt động tự động, robot cần có khả năng nhận biết môi trường, xây dựng bản đồ, xác định vị trí và lập kế hoạch đường đi đến mục tiêu. Trong đó, bài toán lập kế hoạch quỹ đạo đóng vai trò quan trọng vì ảnh hưởng trực tiếp đến hiệu quả di chuyển và khả năng hoàn thành nhiệm vụ của robot.

Hiện nay, các hệ thống robot tự hành thường sử dụng các cảm biến như LiDAR và encoder kết hợp với nền tảng ROS 2, các thuật toán SLAM và hệ thống điều hướng Nav2 để xây dựng bản đồ, định vị và điều hướng robot [1]. Trong các phương pháp tìm đường, thuật toán A* được sử dụng phổ biến nhờ khả năng tìm kiếm đường đi dựa trên chi phí di chuyển và chi phí ước lượng đến đích. Tuy nhiên, khi không gian tìm kiếm lớn hoặc môi trường có nhiều vật cản, thuật toán có thể phải mở rộng nhiều nút, làm tăng thời gian xử lý và chi phí tính toán.

Đề tài tập trung thiết kế và thi công mô hình robot tự hành bốn bánh trên nền tảng ROS 2, tích hợp chức năng xây dựng bản đồ, định vị và điều hướng trong môi trường trong nhà. Bên cạnh việc sử dụng thuật toán A* để lập kế hoạch đường đi, đề tài áp dụng thêm trọng số W nhằm giảm số lượng nút mở rộng và rút ngắn thời gian tìm kiếm quỹ đạo. Hiệu quả của phương pháp được đánh giá thông qua các tiêu chí như số lượng nút mở rộng, thời gian lập kế hoạch và khả năng di chuyển thực tế của robot. Kết quả đạt được là cơ sở để đánh giá khả năng ứng dụng của phương pháp trong các hệ thống robot tự hành thực tế.

1.2 TÌNH HÌNH NGHIÊN CỨU TRONG VÀ NGOÀI NƯỚC

1.2.1 Trong nước

Trong [5], các tác giả đã nghiên cứu và phát triển robot có khả năng xây dựng bản đồ và định vị đồng thời trên nền tảng ROS. Nghiên cứu góp phần khẳng định

khả năng ứng dụng của các thuật toán SLAM trong việc hỗ trợ robot nhận biết môi trường và xác định vị trí hoạt động. Tuy nhiên, công trình chủ yếu tập trung vào khối xây dựng bản đồ và định vị, chưa đề cập đầy đủ đến bài toán điều hướng robot trong môi trường thực tế.

Bên cạnh đó, nghiên cứu [6] đã đề xuất hệ thống định vị cho robot di động trên nền tảng ROS bằng cách kết hợp dữ liệu từ nhiều cảm biến nhằm nâng cao độ chính xác xác định vị trí. Kết quả nghiên cứu cho thấy việc hợp nhất dữ liệu cảm biến giúp cải thiện chất lượng định vị của robot. Tuy nhiên, nghiên cứu tập trung vào khía cạnh định vị và chưa xây dựng một hệ thống hoàn chỉnh bao gồm xây dựng bản đồ, lập kế hoạch và điều hướng tự động.

Ngoài ra, trong nghiên cứu [7], các tác giả đã xây dựng hệ thống điều hướng cho robot tự hành thông qua việc tạo quỹ đạo cục bộ và tránh vật cản trong môi trường hoạt động. Mặc dù hệ thống đáp ứng yêu cầu điều hướng trong môi trường ứng dụng cụ thể, nghiên cứu chưa thực hiện đánh giá và so sánh hiệu quả của các phương pháp lập kế hoạch đường đi nhằm tối ưu thời gian tìm kiếm và chất lượng quỹ đạo.

1.2.2 Ngoài nước

Trong [8], các tác giả đã phát triển nền tảng robot tự hành dựa trên ROS nhằm thực hiện các chức năng xây dựng bản đồ, định vị và điều hướng trong môi trường trong nhà. Nghiên cứu tập trung vào thiết kế nền tảng phần cứng và tích hợp hệ thống điều hướng tự động, qua đó cho thấy khả năng ứng dụng hiệu quả của ROS trong các hệ thống robot di động. Tuy nhiên, nghiên cứu chủ yếu tập trung vào kiến trúc hệ thống và khả năng vận hành của robot, chưa đi sâu vào việc tối ưu thuật toán lập kế hoạch đường đi.

Bên cạnh đó, trong [9], các tác giả đã xây dựng hệ thống điều hướng tự động cho robot di động trên nền tảng ROS, kết hợp SLAM để xây dựng bản đồ, định vị, thuật toán A* để lập kế hoạch đường đi toàn cục và DWA để tránh vật cản cục bộ. Kết quả nghiên cứu cho thấy hệ thống có khả năng thực hiện nhiệm vụ điều hướng chính xác trong môi trường trong nhà. Tuy nhiên, nghiên cứu chưa thực hiện đánh giá và so sánh các biến thể của thuật toán A* nhằm cải thiện hiệu quả lập kế hoạch đường đi.

1.3 MỤC TIÊU ĐỀ TÀI

Đề tài được thực hiện nhằm thiết kế, thi công và đánh giá mô hình robot tự hành có khả năng xây dựng bản đồ, định vị và di chuyển đến vị trí mục tiêu trong môi trường trong nhà, diện tích dưới 50m². Hệ thống được xây dựng trên nền tảng ROS 2, tích hợp các chức năng thu nhận dữ liệu cảm biến, xây dựng bản đồ môi trường, xác định vị trí robot và điều hướng tự động đến đích.

Bên cạnh đó, đề tài nghiên cứu ứng dụng thuật toán A* trong bài toán lập kế hoạch đường đi cho robot tự hành. Trên cơ sở thuật toán A* truyền thống, đề tài thực hiện cải tiến bằng cách bổ sung trọng số W để hình thành thuật toán WA*, từ đó giảm số lượng nút mở rộng, rút ngắn thời gian tìm kiếm và nâng cao hiệu quả lập kế hoạch quỹ đạo. Hệ thống được kiểm chứng thông qua mô phỏng và thực nghiệm, đồng thời đánh giá hiệu quả của thuật toán A* và WA* dựa trên các tiêu chí như số lượng nút đã xét, thời gian lập kế hoạch, chiều dài quỹ đạo và khả năng di chuyển của robot trong môi trường thực tế.

1.4 PHƯƠNG PHÁP NGHIÊN CỨU

Đề tài được thực hiện bằng cách kết hợp nhiều phương pháp nghiên cứu khác nhau như nghiên cứu lý thuyết, thiết kế và xây dựng hệ thống, mô phỏng và thực nghiệm, so sánh và đánh giá. Đầu tiên là nghiên cứu tài liệu liên quan để tìm hiểu các kiến thức liên quan đến robot tự hành, hệ điều hành ROS 2, SLAM, Nav2 và thuật toán A*. Trên cơ sở đó sẽ thực hiện thiết kế và xây dựng hệ thống nhằm phát triển mô hình robot tự hành có khả năng thu nhận dữ liệu cảm biến, xây dựng bản đồ, định vị và điều hướng trong môi trường trong nhà.

Sau khi hoàn thiện hệ thống, mô phỏng và thực nghiệm được sử dụng để kiểm tra khả năng hoạt động của robot trong các nhiệm vụ xây dựng bản đồ, định vị và di chuyển đến vị trí mục tiêu. Cuối cùng là so sánh và đánh giá được áp dụng để phân tích hiệu quả của thuật toán A* và thuật toán WA* thông qua các tiêu chí như số lượng nút mở rộng, thời gian lập kế hoạch, chiều dài quỹ đạo và khả năng di chuyển thực tế của robot.

1.5 GIỚI HẠN ĐỀ TÀI

Đề tài được thực hiện trong phạm vi môi trường trong nhà với mặt sàn tương đối bằng phẳng và sử dụng bản đồ 2D được xây dựng từ dữ liệu LiDAR, do đó chưa đánh giá khả năng hoạt động của hệ thống trong môi trường ngoài trời, địa hình phức tạp hoặc các điều kiện thay đổi liên tục. Việc nhận biết môi trường chủ yếu dựa trên dữ liệu từ các cảm biến được trang bị trên robot, chưa tích hợp các chức năng xử lý ảnh và nhận dạng đối tượng bằng camera. Về mặt thuật toán, đề tài tập trung nghiên cứu thuật toán A* và phương pháp cải tiến WA* cho bài toán lập kế hoạch đường đi, chưa thực hiện so sánh với nhiều thuật toán tìm đường khác hoặc tối ưu chuyên sâu các bộ điều khiển chuyển động. Ngoài ra, các kết quả đánh giá được thực hiện trên một số kịch bản mô phỏng và thực nghiệm cụ thể nên chưa thể phản ánh đầy đủ mọi điều kiện vận hành có thể gặp trong thực tế.

1.6 BỐ CỤC ĐỒ ÁN

Chương 1. TỔNG QUAN

Chương này trình bày tổng quan về đề tài, lý do chọn đề tài, mục tiêu thực hiện, phạm vi giới hạn, phương pháp nghiên cứu, đối tượng và phạm vi nghiên cứu. Bên cạnh đó, chương này cũng giới thiệu khái quát về hướng cải tiến robot tự hành.

Chương 2. CƠ SỞ LÝ THUYẾT

Chương này trình bày các cơ sở lý thuyết liên quan đến đề tài, bao gồm tổng quan về robot tự hành, cơ bản động học robot, ROS 2, SLAM, Nav2, hệ thống điều hướng và thuật toán A*, WA*.

Chương 3. THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

Chương này trình bày quá trình thiết kế và xây dựng hệ thống robot tự hành, bao gồm thiết kế phần cứng và phần mềm. Nội dung chương tập trung mô tả cấu trúc hệ thống, chức năng các khối phần cứng, cơ sở lựa chọn linh kiện, kiến trúc kết nối giữa các thành phần, đồng thời trình bày việc xây dựng hệ thống phần mềm trên nền tảng ROS 2, tích hợp các chức năng xây dựng bản đồ, định vị, điều hướng và lập kế hoạch đường đi bằng thuật toán A* và WA*.

Chương 4. KẾT QUẢ VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này trình bày các kết quả đạt được sau khi thiết kế và thi công hệ thống, bao gồm kết quả mô hình robot, kết quả xây dựng bản đồ, kết quả lập kế hoạch đường đi và kết quả robot di chuyển đến vị trí mục tiêu. Đồng thời, chương này đánh giá hiệu quả của thuật toán A* có trọng số W, so sánh với thuật toán A* thông thường thông qua các tiêu chí như số lượng nút mở rộng, thời gian lập kế hoạch và khả năng vận hành của robot trong môi trường thực nghiệm.

Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết các nội dung đã thực hiện, đánh giá mức độ hoàn thành mục tiêu đề tài và nêu ra những kết quả đạt được của hệ thống robot tự hành. Bên cạnh đó, chương này đề xuất các hướng phát triển nhằm nâng cao độ ổn định, khả năng điều hướng và hiệu quả tối ưu quỹ đạo của robot trong các nghiên cứu tiếp theo.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1 ROBOT DI ĐỘNG TỰ HÀNH

2.1.1 Tổng quan về robot di động tự hành

Robot di động tự hành là hệ thống robot có khả năng tự di chuyển trong môi trường làm việc mà không cần con người điều khiển trực tiếp liên tục. Để thực hiện được nhiệm vụ này, robot thường sử dụng các cảm biến như LiDAR, camera, encoder để nhận biết môi trường, xác định vị trí và phát hiện vật cản. Dựa trên dữ liệu thu nhận được, hệ thống sẽ xây dựng bản đồ, lập kế hoạch đường đi và điều khiển robot di chuyển đến vị trí mục tiêu. Theo tài liệu về robot tự hành AMR, robot có thể tự định vị và di chuyển dựa trên bản đồ cùng các cảm biến gắn trên xe, trong đó LiDAR và camera thường được sử dụng để hỗ trợ quá trình vận hành an toàn [8].

2.1.2 Các loại robot di động tự hành phổ biến

Xe tự hành dẫn đường tự động AGV, là loại xe di động thường được sử dụng trong nhà máy, kho hàng và dây chuyền sản xuất để vận chuyển vật tư hoặc hàng hóa. AGV thường di chuyển theo các tuyến đường cố định đã được thiết lập trước bằng dây dẫn, băng từ, cảm biến hoặc các dấu hiệu định hướng trong môi trường [16], mẫu robot AGV xem trong hình 2.1.



Hình 2.1 Xe tự động dẫn đường

Robot di động tự hành AMR, là loại robot có khả năng di chuyển linh hoạt hơn so với AGV. AMR không cần phụ thuộc hoàn toàn vào tuyến đường cố định mà có thể sử dụng cảm biến, camera, laser scanner và phần mềm điều hướng để xây dựng bản đồ, nhận biết vật cản và lựa chọn đường đi phù hợp. Khi gặp vật cản, AMR có thể tự tính toán đường đi thay thế để tiếp tục nhiệm vụ. Nhờ khả năng thích nghi tốt với môi trường động, AMR phù hợp với các hệ thống sản xuất và kho hàng hiện đại, nơi bố trí mặt bằng có thể thay đổi thường xuyên [16], mẫu robot AMR xem trong hình 2.2.



Hình 2.2 Robot di động tự hành

2.1.3 Ứng dụng của xe tự hành

Xe tự hành là một trong những hướng phát triển quan trọng của lĩnh vực robot và tự động hóa nhờ khả năng tự nhận biết môi trường, xác định vị trí và di chuyển đến mục tiêu mà không cần sự điều khiển trực tiếp liên tục của con người. Với sự phát triển của các công nghệ cảm biến, xử lý dữ liệu và trí tuệ nhân tạo, xe tự hành ngày càng được ứng dụng rộng rãi trong nhiều lĩnh vực khác nhau. Trong công nghiệp và kho vận, các robot tự hành được sử dụng để vận chuyển nguyên vật liệu, linh kiện và hàng hóa giữa các khu vực làm việc, góp phần giảm sức lao động thủ công, nâng cao năng suất và tăng mức độ tự động hóa trong quá trình sản xuất [17].

Bên cạnh đó, xe tự hành còn được ứng dụng trong các lĩnh vực như y tế, dịch vụ, giao hàng và nông nghiệp. Các hệ thống này có thể hỗ trợ vận chuyển thuốc men, vật tư y tế, hàng hóa hoặc thực hiện các công việc giám sát và hỗ trợ sản xuất. Để đáp ứng các yêu cầu đó, robot cần được trang bị khả năng xây dựng bản đồ môi trường, định vị vị trí hiện tại, lập kế hoạch đường đi và tránh vật cản trong quá trình di chuyển. Nhờ những ưu điểm về tính linh hoạt, khả năng làm

việc tự động và hiệu quả vận hành, xe tự hành đang trở thành nền tảng quan trọng trong các hệ thống robot thông minh và tự động hóa hiện đại [17].

2.1.4 Các thách thức và công nghệ hỗ trợ xe tự hành

Xe tự hành phải đối mặt với nhiều thách thức như nhận biết môi trường, xác định vị trí chính xác, xây dựng bản đồ và lập kế hoạch đường đi trong điều kiện có vật cản. Để giải quyết các vấn đề này, hệ thống thường sử dụng các cảm biến như LiDAR, IMU và encoder để thu thập dữ liệu về môi trường và trạng thái chuyển động của robot, đồng thời kết hợp các thuật toán SLAM nhằm xây dựng bản đồ và định vị trong môi trường hoạt động. Bên cạnh đó, các nền tảng phần mềm như ROS 2 và Nav2 đóng vai trò kết nối và quản lý các thành phần trong hệ thống, hỗ trợ quá trình điều hướng tự động. Trên cơ sở thông tin về bản đồ và vị trí hiện tại, các thuật toán lập kế hoạch đường đi như A* được sử dụng để tìm quỹ đạo từ vị trí xuất phát đến vị trí đích, giúp robot di chuyển an toàn và hiệu quả trong môi trường làm việc.

2.1.5 Mô hình động học của Robot di động tự hành

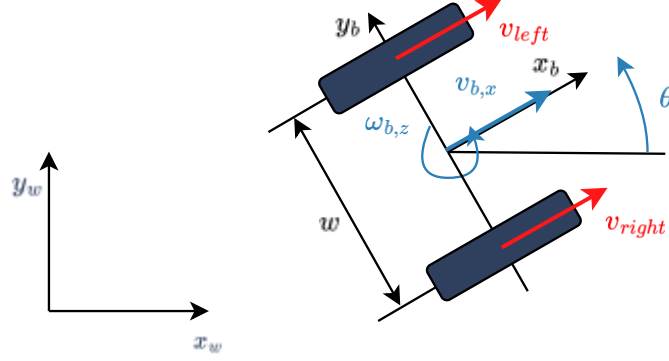
Đối với robot hoạt động trên mặt phẳng, trạng thái của robot được mô tả thông qua hệ tọa độ toàn cục và hệ tọa độ gắn với thân xe. Hệ tọa độ toàn cục được sử dụng để xác định vị trí của robot trong môi trường hoặc trên bản đồ, trong khi hệ tọa độ thân xe được dùng để biểu diễn hướng chuyển động và các đại lượng vận tốc tương đối của robot. Theo mô hình động học của robot di động, trạng thái của robot trên mặt phẳng được xác định bởi ba đại lượng gồm tọa độ x, tọa độ y và góc định hướng θ , từ đó có thể mô tả đầy đủ vị trí và hướng của robot trong không gian làm việc. Trạng thái của robot được biểu diễn qua công thức:

$$q = (x, y, \theta), \quad (2.1)$$

trong đó x là tọa độ vị trí của robot theo trục x trong hệ tọa độ toàn cục, y là tọa độ vị trí của robot theo trục y trong hệ tọa độ toàn cục, θ là góc định hướng của robot so với hệ tọa độ tham chiếu, q là vector trạng thái biểu diễn vị trí và hướng của robot trên mặt phẳng. Khi robot di chuyển, các giá trị x, y và θ thay đổi theo thời gian. Sự thay đổi này phụ thuộc vào vận tốc tuyến tính và vận tốc góc của robot. Vì vậy, việc xác định hệ tọa độ và trạng thái của robot là cơ sở để xây dựng phương trình động học, tính toán chuyển động và phục vụ các bài toán định vị, lập bản đồ và điều hướng trong hệ thống robot tự hành [18].

Robot sử dụng cơ cấu truyền động 4 bánh, mỗi bánh được dẫn động bởi một động cơ DC có encoder riêng. Bốn bánh được chia thành hai nhóm truyền động gồm nhóm bánh trái và nhóm bánh phải. Khi hai nhóm bánh quay cùng chiều và cùng vận tốc, robot di chuyển thẳng, khi vận tốc hai nhóm bánh khác nhau, robot sẽ rẽ trái hoặc rẽ phải, khi hai nhóm bánh quay ngược chiều nhau, robot có thể

quay tại chỗ. Robot được điều khiển bằng cách thay đổi chênh lệch vận tốc giữa hai nhóm bánh trái và phải. Với đặc điểm này, robot 4 bánh có thể được mô hình hóa gần tương đương với robot dẫn động vi sai [19], minh họa động học robot dẫn động vi sai xem ở hình 2.3 .



Hình 2.3 Mô hình động học của robot dẫn động vi sai

Vận tốc chuyển động của thân robot được xác định bởi vận tốc của bánh hoặc nhóm bánh bên trái và bên phải. Quan hệ động học thuận của robot vi sai được biểu diễn qua công thức:

$$V_{b,x} = \frac{V_{Right} + V_{Left}}{2}, \quad (2.2)$$

$$\omega_{b,z} = \frac{V_{Right} - V_{Left}}{L}, \quad (2.3)$$

trong đó $V_{b,x}$ là vận tốc tuyến tính của robot theo trục x của thân robot, $\omega_{b,z}$ là vận tốc góc của robot quanh trục z, V_{Right} là vận tốc của bánh hoặc nhóm bánh bên phải, V_{Left} là vận tốc của bánh hoặc nhóm bánh bên trái, L là khoảng cách giữa hai bánh hoặc hai nhóm bánh trái và phải [20].

Từ vận tốc tuyến tính và vận tốc góc của thân robot, trạng thái của robot trong hệ tọa độ toàn cục được mô tả bằng các phương trình động học thuận:

$$\dot{x} = V_{b,x} \cos(\theta), \quad (2.4)$$

$$\dot{y} = V_{b,x} \sin(\theta), \quad (2.5)$$

$$\dot{\theta} = \omega_{b,z}, \quad (2.6)$$

trong đó x và y là tọa độ vị trí của robot trong hệ tọa độ toàn cục, θ là góc định hướng của robot so với hệ tọa độ toàn cục, $\dot{x}$ là tốc độ thay đổi tọa độ x theo thời gian, $\dot{y}$ là tốc độ thay đổi tọa độ y theo thời gian, $\dot{\theta}$ là tốc độ thay đổi góc định

hướng của robot theo thời gian. Từ các công thức (2.4), (2.5), (2.6) trên cho thấy vị trí của robot thay đổi phụ thuộc vào vận tốc tuyến tính $V_{b,x}$ và góc định hướng θ . Khi robot có vận tốc góc $\omega_{b,z} \neq 0$, hướng của robot sẽ thay đổi theo thời gian, làm cho quỹ đạo di chuyển có dạng cong hoặc quay quanh một tâm tức thời. Đối với robot 4 bánh, hai bánh bên trái được xem như một nhóm truyền động trái, hai bánh bên phải được xem như một nhóm truyền động phải. Vì vậy, robot có thể được mô hình hóa gần tương đương với robot dẫn động vi sai để phục vụ tính toán động học và điều khiển chuyển động.

Các trường hợp chuyển động đặc biệt. Khi vận tốc hai bánh bằng nhau ($V_{\text{Right}} = V_{\text{Left}}$), robot di chuyển thẳng, khi vận tốc hai bánh khác nhau ($V_{\text{Right}} \neq V_{\text{Left}}$), robot chuyển động theo quỹ đạo cong, khi hai bánh quay ngược chiều nhau với cùng độ lớn vận tốc ($V_{\text{Right}} = -V_{\text{Left}}$), robot có thể quay tại chỗ. Các đặc điểm này phù hợp với cơ cấu truyền động của robot 4 bánh, trong đó hướng chuyển động được thay đổi bằng sự chênh lệch vận tốc giữa hai nhóm bánh trái và phải. Robot dẫn động vi sai thuộc nhóm robot non-holonomic, nên không thể di chuyển tức thời theo mọi hướng trong mặt phẳng [20]. Đối với loại robot này, chuyển động chủ yếu gồm tiến, lùi và quay nhờ sự chênh lệch vận tốc giữa hai bên bánh. Vì vậy, robot không thể dịch chuyển ngang trực tiếp như các robot sử dụng bánh omni hoặc mecanum.

2.2 HỆ ĐIỀU HÀNH ROS 2

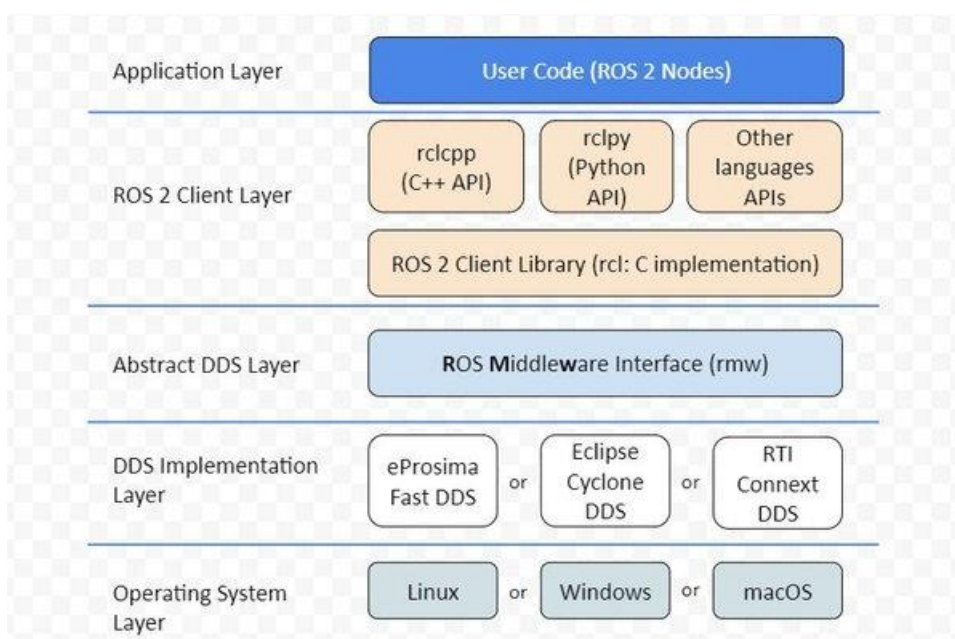
2.2.1 Khái niệm

ROS 2, viết đầy đủ là Robot Operating System 2, là phiên bản cải tiến của ROS, là một nền tảng phần mềm mã nguồn mở hỗ trợ phát triển các ứng dụng robot. ROS 2 là tập hợp các thư viện phần mềm và công cụ giúp xây dựng các ứng dụng robot, từ trình điều khiển phần cứng, thuật toán xử lý đến các công cụ hỗ trợ phát triển hệ thống [21]. Mặc dù có tên là “Operating System”, ROS 2 không phải là hệ điều hành theo nghĩa truyền thống như Windows hoặc Linux, mà đóng vai trò như một lớp trung gian phần mềm giúp các thành phần trong hệ thống robot giao tiếp và phối hợp với nhau.

ROS 2 được xây dựng theo mô hình trung gian, hỗ trợ truyền nhận dữ liệu giữa các tiến trình khác nhau thông qua cơ chế publish và subscribe có kiểu dữ liệu rõ ràng, cho phép các tiến trình trong robot trao đổi thông điệp với nhau. Trong một hệ thống ROS 2, các chương trình thường được tổ chức thành các node. Các node này có thể xử lý cảm biến, điều khiển động cơ, xây dựng bản đồ, định vị hoặc lập kế hoạch đường đi. Mối liên kết giữa các node trong hệ thống được gọi là ROS graph [21].

2.2.2 Kiến trúc của ROS 2

Kiến trúc ROS 2 được xây dựng theo dạng phân tán, trong đó hệ thống robot được chia thành nhiều thành phần phần mềm độc lập gọi là node. Mỗi node thường đảm nhiệm một chức năng riêng như đọc dữ liệu cảm biến, xử lý bản đồ, tính toán odometry hoặc điều khiển động cơ. Các node trong ROS 2 không hoạt động riêng lẻ mà liên kết với nhau thông qua một mạng giao tiếp gọi là ROS graph, nó biểu diễn mối quan hệ giữa các node và các kênh giao tiếp trong hệ thống. Thông qua ROS graph, các node có thể gửi và nhận dữ liệu bằng nhiều cơ chế khác nhau như topic, service, action và parameter. Cách tổ chức này giúp hệ thống robot có tính mô-đun, dễ mở rộng và dễ kiểm tra từng chức năng riêng biệt [21], tổng quan kiến trúc ROS 2 xem ở hình 2.4.

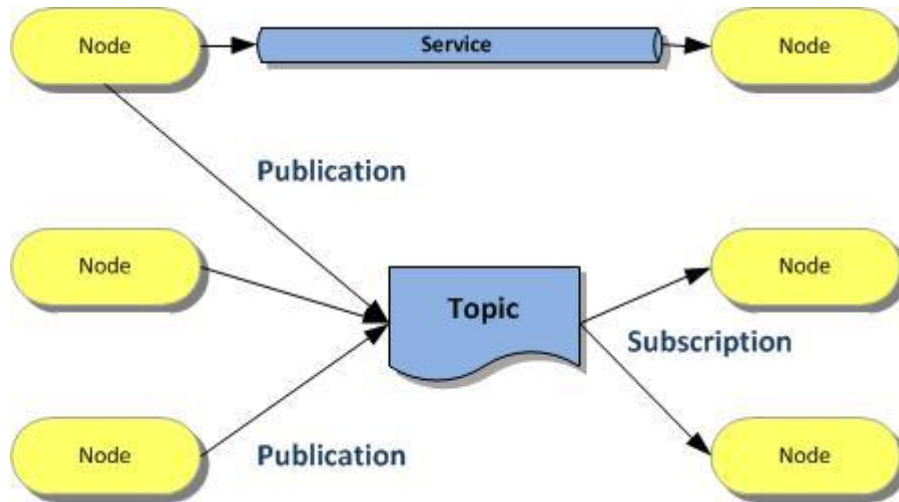


Hình 2.4 Kiến trúc ROS 2

Bên dưới lớp khách hàng là lớp giao tiếp trung gian ROS, có nhiệm vụ tạo cầu nối giữa ROS 2 và các hệ thống DDS. Thông qua lớp này, ROS 2 có thể sử dụng nhiều DDS khác nhau như Fast DDS, Cyclone DDS hoặc RTI Connexant DDS mà không cần thay đổi mã nguồn của ứng dụng. DDS đảm nhiệm việc khám phá node, trao đổi dữ liệu theo cơ chế publish-subscribe và truyền thông giữa các tiến trình trong hệ thống. Nhờ kiến trúc phân tán này, ROS 2 có khả năng hỗ trợ hiệu quả các hệ thống robot gồm nhiều node, nhiều cảm biến và có thể hoạt động trên nhiều nền tảng hệ điều hành khác nhau như Linux, Windows và macOS.

2.2.3 Các thành phần chính trong ROS 2

Các thành phần chính trong ROS 2 bao gồm node, topic, service, action, parameter, message và package. Đây là các thành phần cơ bản giúp tổ chức, giao tiếp và triển khai phần mềm trong hệ thống robot [21], minh họa hệ thống truyền thông trong ROS 2 xem ở hình 2.5.



Hình 2.5 Mô hình truyền thông giữa các node trong ROS 2

Trong hệ thống ROS 2, node là đơn vị xử lý cơ bản trong ROS 2. Mỗi node thường đảm nhiệm một chức năng cụ thể. Việc chia hệ thống thành nhiều node giúp phần mềm dễ quản lý và thuận tiện hơn khi phát triển hệ thống lớn. Topic là kênh truyền dữ liệu theo mô hình publish và subscribe. Một node có thể xuất bản dữ liệu lên topic, trong khi các node khác có thể đăng ký topic đó để nhận dữ liệu. Topic thường được dùng cho các dữ liệu thay đổi liên tục như dữ liệu quét laser, vận tốc robot hoặc trạng thái cảm biến. Service là cơ chế giao tiếp theo dạng yêu cầu và phản hồi. Một node gửi yêu cầu đến service server và chờ nhận kết quả trả về. Cơ chế này phù hợp với các tác vụ ngắn, có kết quả rõ ràng, ví dụ gọi một chức năng cấu hình hoặc truy vấn trạng thái hệ thống. Action được sử dụng cho các tác vụ kéo dài. Action cho phép gửi mục tiêu, nhận phản hồi trong quá trình thực hiện và nhận kết quả sau khi hoàn thành. Trong robot tự hành, action thường được dùng cho các nhiệm vụ như điều hướng robot đến một vị trí mục tiêu. Parameter là các giá trị cấu hình gắn với từng node. Thông qua parameter, người dùng có thể điều chỉnh hoạt động của node mà không cần thay đổi mã nguồn. Ví dụ, các tham số về tốc độ robot, khung tọa độ, tần số cập nhật hoặc thông số cảm biến có thể được cấu hình thông qua parameter. Package là đơn vị tổ chức mã nguồn trong ROS 2. Một package có thể chứa node, file cấu hình, file launch, message, service hoặc các thư viện cần thiết. Trong thực tế, một

hệ thống robot thường gồm nhiều package khác nhau để triển khai các chức năng riêng biệt.

Có thể thấy trong ROS 2, hệ thống phần mềm được tổ chức thành nhiều node, trong đó mỗi node đảm nhiệm một chức năng riêng biệt như thu nhận dữ liệu cảm biến, xử lý thông tin hoặc điều khiển cơ cấu chấp hành. Thông qua topic, các node có thể trao đổi dữ liệu theo mô hình publish/subscribe. Node phát dữ liệu được gọi là publisher, còn node nhận dữ liệu được gọi là subscriber. Một topic có thể được nhiều node cùng xuất bản hoặc cùng đăng ký nhận dữ liệu. Cơ chế này phù hợp với các dữ liệu thay đổi liên tục như dữ liệu LiDAR, IMU, odometry hoặc trạng thái cảm biến. ROS 2 còn hỗ trợ cơ chế service theo mô hình yêu cầu và phản hồi. Trong cơ chế này, một node gửi yêu cầu đến node khác để thực hiện một chức năng cụ thể và nhận lại kết quả phản hồi. Service thường được sử dụng cho các tác vụ ngắn, cần phản hồi ngay sau khi thực hiện.

2.2.4 ROS 2 Humble Hawksbill

ROS 2 Humble Hawksbill là một bản phân phối của ROS 2 được phát hành vào năm 2022. Đây là phiên bản hỗ trợ dài hạn, thường được gọi là LTS, với thời gian hỗ trợ đến tháng 5 năm 2027. ROS 2 Humble được phát triển hướng đến nền tảng Ubuntu 22.04 Jammy Jellyfish, đồng thời hỗ trợ kiến trúc amd64 và arm64. Vì vậy, phiên bản này phù hợp cho các hệ thống robot chạy trên máy tính cá nhân, máy tính nhúng. ROS 2 Humble cung cấp đầy đủ các thành phần cần thiết để phát triển hệ thống robot, bao gồm node, topic, service, action, parameter, launch file, công cụ dòng lệnh ROS 2 CLI và các công cụ trực quan như RViz2. Bên cạnh đó, nhiều gói phần mềm quan trọng trong robot di động tự hành như SLAM Toolbox, Nav2, TF2 và các gói điều khiển đều có thể được triển khai trên nền tảng ROS 2 Humble [21].

2.2.5 Ứng dụng ROS 2 trong thực tiễn

ROS 2 được ứng dụng rộng rãi trong nhiều lĩnh vực robot nhờ khả năng xây dựng kiến trúc phần mềm theo hướng mô đun, hỗ trợ giao tiếp linh hoạt giữa các thành phần và tích hợp nhiều công cụ phục vụ quá trình phát triển hệ thống robot. Một trong những hướng ứng dụng phổ biến nhất của ROS 2 là trong các hệ thống robot di động tự hành, khi được kết hợp với các nền tảng như Nav2, SLAM và RViz2 để thực hiện các chức năng quan trọng như xây dựng bản đồ môi trường, định vị vị trí robot, lập kế hoạch đường đi và điều hướng trong môi trường thực tế. Bên cạnh đó, ROS 2 còn được sử dụng trong các hệ thống cánh tay robot và robot thao tác thông qua sự hỗ trợ của MoveIt 2, cho phép thực hiện các bài toán lập kế hoạch chuyển động, điều khiển cơ cấu chấp hành, thao tác với vật thể, xử lý dữ liệu không gian ba chiều và giải quyết các vấn đề liên quan đến động học robot. Trong lĩnh vực sản xuất và tự động hóa công nghiệp, nền tảng

ROS công nghiệp mở rộng khả năng của ROS 2 để đáp ứng các yêu cầu như robot lắp đặt, kiểm tra sản phẩm, vận chuyển vật liệu và tích hợp robot vào dây chuyền sản xuất. Ngoài các ứng dụng trên robot thực tế, ROS 2 còn được sử dụng phổ biến trong nghiên cứu và mô phỏng, khi kết hợp với các công cụ như Gazebo nhằm kiểm thử thuật toán, mô phỏng hoạt động của cảm biến, đánh giá khả năng điều hướng và phát triển phần mềm trước khi triển khai trên hệ thống robot thật. Nhờ tính linh hoạt và khả năng mở rộng cao, ROS 2 trở thành một nền tảng quan trọng trong quá trình nghiên cứu, phát triển và triển khai các hệ thống robot hiện đại.

2.3 GIỚI THIỆU VỀ HỆ THỐNG SLAM

2.3.1 Khái quát về SLAM

SLAM là thuật toán định vị và xây dựng bản đồ đồng thời. Đây là bài toán cho phép robot vừa xây dựng bản đồ của môi trường chưa biết trước, vừa xác định vị trí của chính nó trong bản đồ đó. Trong hệ thống robot tự hành, SLAM đóng vai trò quan trọng. Vì robot cần biết môi trường xung quanh có cấu trúc như thế nào và bản thân robot đang ở vị trí nào để có thể lập kế hoạch di chuyển. Robot sử dụng dữ liệu từ các cảm biến như LiDAR, camera, IMU và encoder để quan sát môi trường và ước lượng chuyển động. Dữ liệu cảm biến được xử lý để tạo ra bản đồ, đồng thời cập nhật vị trí của robot theo thời gian. Đối với robot di động trong nhà, SLAM thường sử dụng LiDAR 2D để tạo bản đồ lưới, trong đó vùng trống, vùng có vật cản và vùng chưa biết được biểu diễn trên bản đồ 2D. Bản đồ này là cơ sở để robot thực hiện định vị, lập kế hoạch đường đi và điều hướng đến vị trí mục tiêu [10].

2.3.2 Quy trình xử lý SLAM trong ROS 2

Trong ROS 2, quá trình SLAM thường được triển khai thông qua sự phối hợp giữa các node xử lý dữ liệu cảm biến, dữ liệu chuyển động và thuật toán xây dựng bản đồ. Quy trình SLAM bắt đầu bằng việc thu thập dữ liệu từ các cảm biến trên robot như LiDAR, camera và các cảm biến khác nhằm nhận biết môi trường xung quanh và trạng thái chuyển động của robot. Các dữ liệu được thu thập, sau đó được tiền xử lý và đồng bộ nhằm đảm bảo sự liên kết giữa dữ liệu quét môi trường và thông tin chuyển động. Đồng thời đưa các dữ liệu này về cùng một hệ tọa độ tham chiếu để phục vụ quá trình tính toán. Dựa trên các thông tin cảm biến, hệ thống tiến hành ước lượng vị trí và hướng di chuyển của robot, từ đó cập nhật trạng thái pose của robot trong môi trường. Song song với quá trình định vị, các lần quét cảm biến liên tiếp được ghép nối để xây dựng và cập nhật bản đồ môi trường theo thời gian thực. Trong quá trình robot di chuyển, sai số tích lũy có thể xuất hiện do nhiễu cảm biến và sai số ước lượng, vì vậy các thuật toán SLAM thực hiện bước hiệu chỉnh thông qua việc phát hiện vòng lặp,

khi robot nhận biết lại các khu vực đã từng đi qua để tối ưu hóa quỹ đạo và giảm sai số của bản đồ. Kết quả cuối cùng của quá trình SLAM là bản đồ môi trường được xây dựng, thường được biểu diễn dưới dạng bản đồ 2D phục vụ cho các nhiệm vụ định vị và điều hướng của robot tự hành [10].

2.3.3 Một số gói SLAM phổ biến trong ROS 2

Trong ROS 2, có nhiều gói phần mềm hỗ trợ bài toán SLAM tùy thuộc vào loại cảm biến và mục đích sử dụng. Một trong những gói phổ biến là SLAM Toolbox. Đây là gói SLAM 2D được phát triển cho ROS 2, hỗ trợ các chế độ như mapping đồng bộ, mapping bất đồng bộ và localization. SLAM Toolbox phù hợp với robot di động sử dụng LiDAR 2D trong môi trường trong nhà và thường được dùng kết hợp với Nav2, ngoài SLAM Toolbox ra còn 1 số gói khác như GMapping, Hector SLAM và Cartographer. Trong đó, GMapping là một phương pháp SLAM dựa trên bộ lọc hạt, thường được sử dụng để xây dựng bản đồ dạng lưới từ dữ liệu quét laser kết hợp với thông tin odometry của robot. Hector SLAM là một phương pháp SLAM sử dụng trực tiếp dữ liệu LiDAR để ước lượng vị trí và xây dựng bản đồ, có ưu điểm trong các hệ thống có cảm biến laser tốc độ quét cao. Cartographer do Google phát triển là một hệ thống SLAM thời gian thực, hỗ trợ xây dựng bản đồ 2D và 3D, đồng thời có khả năng tối ưu hóa quỹ đạo thông qua kỹ thuật đóng vòng lặp. Bên cạnh các phương pháp dựa trên LiDAR, các hệ thống SLAM dựa trên hình ảnh cũng được sử dụng phổ biến như ORB-SLAM và RTAB-Map, trong đó ORB-SLAM khai thác đặc trưng hình ảnh để ước lượng chuyển động camera, còn RTAB-Map hỗ trợ SLAM dựa trên dữ liệu hình ảnh và cảm biến chiều sâu, phù hợp với các ứng dụng yêu cầu bản đồ không gian ba chiều [15].

2.3.4 Ứng dụng của SLAM trong ROS 2

SLAM được ứng dụng rộng rãi trong các hệ thống robot cần hoạt động trong môi trường chưa biết trước hoặc môi trường có sự thay đổi. Trong robot di động tự hành, SLAM giúp robot tạo bản đồ khu vực làm việc, xác định vị trí hiện tại và cung cấp dữ liệu cho hệ thống điều hướng. Các robot AMR trong kho vận, robot dịch vụ trong bệnh viện, robot giao hàng trong khuôn viên và robot nghiên cứu trong phòng thí nghiệm đều có thể sử dụng SLAM để hỗ trợ quá trình di chuyển tự động.

Trong ROS 2, SLAM thường được kết hợp với Nav2 để tạo thành quy trình điều hướng hoàn chỉnh. Ở giai đoạn đầu, robot sử dụng SLAM để xây dựng bản đồ môi trường. Sau đó, bản đồ được lưu lại và sử dụng cho các lần điều hướng tiếp theo. Khi có bản đồ, hệ thống định vị và Nav2 sẽ sử dụng bản đồ này để xác định vị trí robot, lập kế hoạch đường đi và điều khiển robot di chuyển đến vị trí mục tiêu [10].

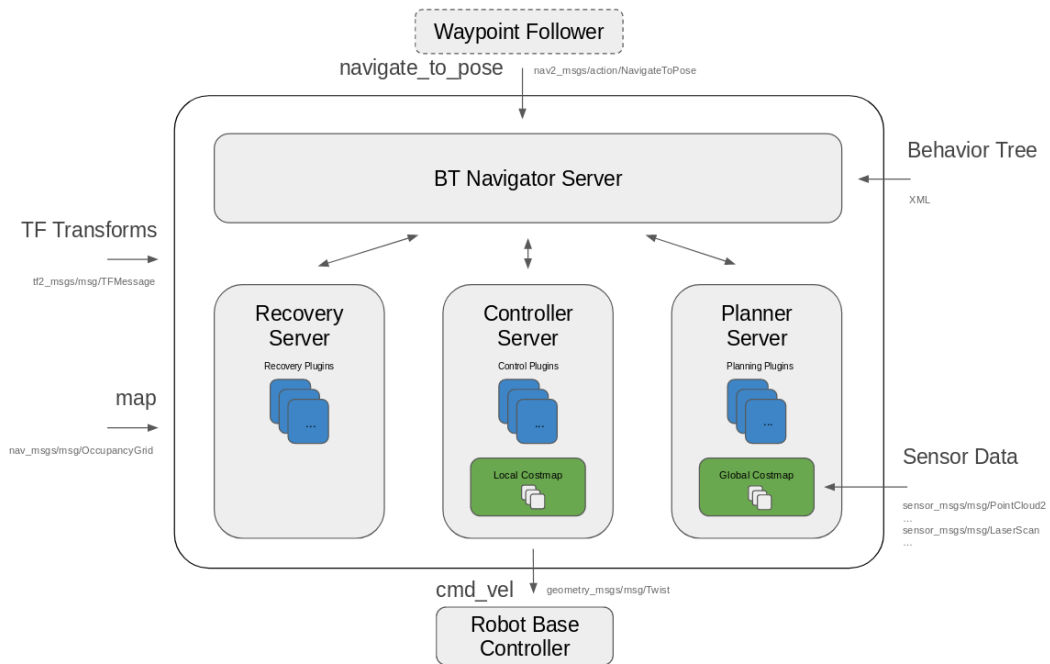
2.4 TỔNG QUAN VỀ NAV2

2.4.1 Khái quát về Nav2

Nav2, là hệ thống điều hướng mã nguồn mở được phát triển cho ROS 2. Nav2 cung cấp các công cụ cần thiết để robot di động có thể thực hiện các nhiệm vụ điều hướng như nạp và lưu bản đồ, định vị robot trên bản đồ, lập kế hoạch đường đi, điều khiển robot bám theo quỹ đạo và tránh va chạm trong quá trình di chuyển. Nav2 được xem là thế hệ kế tiếp của Nav trên ROS 1. So với hệ thống điều hướng cũ, Nav2 được xây dựng trên nền tảng ROS 2, hỗ trợ kiến trúc phân tán, quản lý vòng đời node, cơ chế giao tiếp hiện đại và khả năng tùy chỉnh cao. Một điểm nổi bật của Nav2 là sử dụng cây hành vi để điều phối các hành vi điều hướng, giúp hệ thống có thể tổ chức nhiệm vụ phức tạp theo cấu trúc linh hoạt hơn. Trong hệ thống robot di động, Nav2 thường đóng vai trò là lớp phần mềm điều hướng cấp cao. Khi nhận được vị trí mục tiêu, hệ thống sẽ sử dụng thông tin bản đồ, vị trí hiện tại của robot, dữ liệu cảm biến và costmap để tính toán đường đi phù hợp. Sau đó, bộ điều khiển cục bộ sẽ tạo lệnh vận tốc để robot di chuyển theo đường đi đã lập kế hoạch [22].

2.4.2 Kiến trúc và các thành phần chính trong Nav2

Kiến trúc Nav2 được xây dựng theo mô hình gồm nhiều server chức năng, trong đó mỗi thành phần đảm nhiệm một nhiệm vụ riêng trong quá trình điều hướng robot. Cấu trúc tổng thể của Nav2 và sự kết nối giữa các thành phần trong hệ thống được minh họa trong hình 2.6. Các thành phần này được điều phối thông qua BT Navigator, sử dụng cây hành vi để xác định trình tự thực hiện nhiệm vụ, quản lý các trạng thái thành công, thất bại và kích hoạt các cơ chế phục hồi khi robot gặp sự cố trong quá trình di chuyển. Trong quá trình điều hướng, BT Navigator tiếp nhận mục tiêu từ người dùng và gọi các server phù hợp để thực hiện các nhiệm vụ như lập kế hoạch đường đi, điều khiển chuyển động và xử lý lỗi. Planner Server chịu trách nhiệm xây dựng đường đi toàn cục từ vị trí hiện tại của robot đến vị trí mục tiêu dựa trên bản đồ hoặc global costmap, trong khi Controller Server sử dụng đường đi đã tạo kết hợp với dữ liệu từ local costmap để tính toán lệnh vận tốc, giúp robot bám theo quỹ đạo và tránh vật cản trong quá trình di chuyển [8].



Hình 2.6 Kiến trúc của Nav2

Bên cạnh các thành phần lập kế hoạch và điều khiển, Nav2 còn bao gồm nhiều module hỗ trợ khác nhằm hoàn thiện quá trình điều hướng. Costmap 2D được sử dụng để biểu diễn môi trường dưới dạng bản đồ chi phí, trong đó global costmap phục vụ lập kế hoạch toàn cục và local costmap hỗ trợ tránh vật cản trong phạm vi gần robot. Smoother Server thực hiện làm mượt đường đi nhằm tạo ra quỹ đạo liên tục và phù hợp hơn với đặc tính chuyển động của robot. Recovery hoặc Behavior Server cung cấp các hành vi phục hồi như quay tại chỗ, lùi robot hoặc xóa costmap khi hệ thống gặp lỗi. Ngoài ra, Map Server có nhiệm vụ cung cấp bản đồ tĩnh cho hệ thống, còn AMCL sử dụng bản đồ có sẵn để ước lượng vị trí của robot trong môi trường. Nhờ kiến trúc dạng server và khả năng mở rộng thông qua plugin, Nav2 cho phép người phát triển dễ dàng thay đổi thuật toán lập kế hoạch, bộ điều khiển, phương pháp làm mượt quỹ đạo hoặc cơ chế phục hồi nhằm phù hợp với từng loại robot và điều kiện hoạt động khác nhau [22].

2.4.3 Yêu cầu hệ thống của Nav2

Để Nav2 có thể hoạt động ổn định trong hệ thống robot tự hành, nền tảng robot cần đáp ứng một số yêu cầu cơ bản về phần mềm, dữ liệu đầu vào và cấu hình hệ thống. Trước hết, robot cần được triển khai trên nền tảng ROS 2 với các node cung cấp đầy đủ thông tin phục vụ quá trình điều hướng, bao gồm bản đồ môi trường, dữ liệu odometry, dữ liệu cảm biến và thông tin hệ tọa độ TF. Trong đó, hệ tọa độ TF đóng vai trò quan trọng trong việc liên kết vị trí giữa các thành phần của robot, với cấu trúc phổ biến gồm các frame như map, odom, base_link

và các frame của cảm biến. Frame map thường được cung cấp bởi hệ thống SLAM hoặc bộ định vị, odom được tạo từ hệ thống odometry nhằm mô tả chuyển động tương đối của robot, còn các cảm biến như LiDAR, camera hoặc IMU được liên kết với base_link để đảm bảo dữ liệu không gian được biểu diễn chính xác trong cùng một hệ tọa độ [22].

Bên cạnh dữ liệu vị trí và tọa độ, Nav2 cần các thông tin cảm biến để nhận biết môi trường xung quanh và xây dựng costmap phục vụ quá trình lập kế hoạch và tránh vật cản. Các cảm biến phổ biến như LiDAR 2D, camera chiều sâu hoặc radar có thể cung cấp dữ liệu về vật cản, từ đó được chuyển đổi thành mô hình môi trường để các thuật toán điều hướng xử lý. Ngoài ra, hệ thống cần được cấu hình phù hợp các thông số như kích thước robot, thông số costmap, thuật toán planner, controller và các plugin điều hướng. Việc thiết lập chính xác các tham số này giúp Nav2 tạo ra quỹ đạo phù hợp, đảm bảo robot di chuyển an toàn, ổn định và đáp ứng tốt với điều kiện môi trường hoạt động [22].

2.5 TỔNG QUAN VỀ THUẬT TOÁN A* VÀ WEIGHTED A*

2.5.1 Bài toán lập kế hoạch đường đi cho robot tự hành

Lập kế hoạch đường đi là một bài toán quan trọng trong hệ thống robot tự hành, với mục tiêu xác định quỹ đạo di chuyển phù hợp, để robot có thể đi từ vị trí ban đầu đến vị trí mục tiêu trong môi trường hoạt động. Đường đi được tạo ra cần đảm bảo các yêu cầu như tránh va chạm với vật cản, có chi phí di chuyển thấp và phù hợp với khả năng vận động của robot. Trong robot di động, bài toán lập kế hoạch đường đi thường được thực hiện dựa trên thông tin bản đồ môi trường, trong đó không gian hoạt động có thể được biểu diễn dưới dạng bản đồ lưới hoặc đồ thị để phục vụ quá trình tìm kiếm đường đi [11].

Hiện nay, nhiều thuật toán đã được nghiên cứu và ứng dụng cho bài toán lập kế hoạch đường đi, có thể kể đến các thuật toán tìm kiếm trên đồ thị như Dijkstra, A*, D* và Theta*, cùng với các phương pháp tối ưu hóa như Genetic Algorithm, Particle Swarm Optimization và Ant Colony Optimization [11]. Trong đó, các thuật toán tìm kiếm dựa trên hàm đánh giá heuristic, đặc biệt là A*, được sử dụng phổ biến trong robot tự hành nhờ khả năng kết hợp giữa chi phí di chuyển thực tế và ước lượng khoảng cách đến mục tiêu để tìm ra đường đi hiệu quả. Vì vậy, thuật toán A* trở thành một trong những phương pháp nền tảng trong các hệ thống lập kế hoạch đường đi và được sử dụng rộng rãi trong các ứng dụng điều hướng robot hiện nay.

2.5.2 Nguyên lý hoạt động của thuật toán A*

Thuật toán A* được giới thiệu lần đầu vào năm 1968 bởi Hart, Nilsson và Raphael, là một trong những thuật toán tìm kiếm đường đi heuristic nổi tiếng và

được nghiên cứu sâu rộng nhất trong lĩnh vực trí tuệ nhân tạo. Đây là sự kết hợp giữa phương pháp toán học và phương pháp heuristic, tích hợp thông tin từ miền bài toán vào một lý thuyết toán học hình thức về tìm kiếm đồ thị, đồng thời chứng minh được tính tối ưu của một lớp các chiến lược tìm kiếm [12]. Để thực hiện điều này, thuật toán duy trì hai danh sách chính trong suốt quá trình tìm kiếm. Đầu tiên là danh sách OPEN chứa các nút biên đã được sinh ra nhưng chưa được mở rộng, được tổ chức theo thứ tự ưu tiên dựa trên giá trị hàm chi phí f , và thứ hai là danh sách CLOSED chứa các nút bên trong đã được mở rộng. Thuật toán bắt đầu với trạng thái khởi đầu s trong danh sách OPEN và dừng lại khi nút được chọn để mở rộng là một nút đích [13].

Quá trình tìm kiếm của A^* diễn ra theo một chu trình lặp gồm bốn bước. Trước tiên, nút xuất phát s được đánh dấu là "mở" và giá trị $f(s)$ được tính toán. Tiếp theo, thuật toán chọn ra nút mở n có giá trị f nhỏ nhất để xét. Nếu có nhiều nút bằng giá trị nhau, việc lựa chọn có thể tùy ý nhưng luôn ưu tiên bất kỳ nút n thuộc tập đích T nào. Nếu $n \in T$, nút này được đánh dấu là "đóng" (closed) và thuật toán kết thúc. Ngược lại, n được đánh dấu là đóng và toán tử kế tiếp Γ được áp dụng lên n . Giá trị f được tính cho mỗi nút kế tiếp của n chưa được đánh dấu là đóng, sau đó các nút này được đánh dấu là mở. Đồng thời, bất kỳ nút đóng n_i nào là nút kế tiếp của n mà giá trị $f(n_i)$ hiện tại nhỏ hơn giá trị khi n_i được đánh dấu là đóng sẽ được mở lại, rồi quá trình quay lại bước chọn nút mở có f nhỏ nhất. Trong suốt quá trình này, mỗi khi một nút được mở rộng, thuật toán lưu trữ cùng với mỗi nút kế tiếp n cả chi phí đến n theo đường đi có chi phí thấp nhất tìm được cho đến nay, cùng một con trỏ đến nút tiền nhiệm của n theo đường đi đó. Thuật toán kết thúc tại một nút đích t khi không còn nút nào được mở rộng thêm, và đường đi có chi phí tối thiểu từ s đến t được tái tạo lại bằng cách truy vết ngược từ t về s qua các con trỏ đã lưu [12].

Để quyết định nút nào cần mở rộng tiếp theo trong chu trình trên, A^* sử dụng hàm đánh giá $f(n)$, được định nghĩa là tổng của hai thành phần:

$$f(n) = g(n) + h(n), \quad (2.7)$$

trong đó $f(n)$ là giá trị ước lượng tổng chi phí của đường đi tối ưu đi qua nút n , $g(n)$ là chi phí của đường đi từ nút xuất phát s đến nút n với chi phí nhỏ nhất tìm được cho đến thời điểm hiện tại bởi A^* , còn $h(n)$ là ước lượng chi phí của đường đi tối ưu từ nút n đến nút đích được ưu tiên. Cần lưu ý rằng $g(n) \geq g(n)$, trong đó $g(n)$ là chi phí thực tế của đường đi tối ưu từ s đến n .

Chính vì $g(n)$ chỉ là chi phí tốt nhất tìm được tại một thời điểm nhất định, giá trị này có thể được cập nhật trong quá trình tìm kiếm. Một nút kế tiếp n' của nút đang được mở rộng n có thể được sinh ra nhiều lần nếu n' có nhiều hơn một nút tiền nhiệm. Khi tìm được đường đi rẻ hơn từ s đến n' , giá trị $g(n')$ sẽ được cập

nhật, và nếu n' đang ở trong danh sách CLOSED, nó sẽ được mở lại, tức đưa trở lại danh sách OPEN. Đường đi rẻ nhất đã biết đến n' được ký hiệu là $p(n')$ và cũng được cập nhật tương ứng [14]. Tuy vậy, khi hàm heuristic thỏa mãn tính nhất quán, A^* có một tính chất quan trọng là hàm f không giảm theo trình tự các nút được đóng: nếu $(n_1, n_2, \dots, n_t)$ là chuỗi các nút được đóng bởi A^* và $p \leq q$, thì:

$$\hat{f}(n_p) \leq \hat{f}(n_q), \quad (2.8)$$

trong đó n_p, n_q là các nút thứ p và thứ q trong chuỗi các nút được đóng bởi A^* , còn p, q là chỉ số thứ tự của các nút trong chuỗi, với $p \leq q$ [12]. Hệ quả của tính chất này là khi một nút được đóng, giá trị f của nút đó không vượt quá $f(s)$, tức chi phí tối ưu từ nút xuất phát. Điều này đảm bảo A^* không bao giờ cần mở lại một nút đã đóng, bởi nếu A^* mở rộng một nút thì đường đi tối ưu đến nút đó đã được tìm thấy. Như vậy, quy định mở lại nút đóng nêu ở bước bốn của thuật toán nói trên trên thực chất trở nên không cần thiết và có thể loại bỏ [10].

Tính chất đơn điệu không giảm của f có liên hệ trực tiếp với tính đơn điệu, hay tính nhất quán, của hàm heuristic. Một hàm heuristic h được gọi là thỏa mãn tính nhất quán nếu với bất kỳ nút kế tiếp q' nào của q :

$$h(q) \leq k(q, q') + h(q'), \quad (2.9)$$

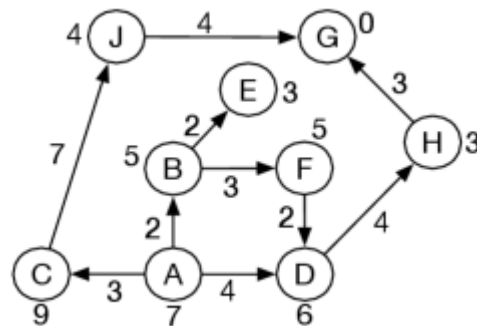
trong đó $h(q)$ là giá trị heuristic tại nút q ước lượng chi phí từ q đến nút đích, $h(q')$ là giá trị heuristic tại nút kế tiếp q' ước lượng chi phí từ q' đến nút đích, còn $k(q, q')$ là chi phí của đường đi tối ưu từ nút q đến nút q' . Khi điều kiện (2.9) được thỏa mãn, A^* tìm ra đường đi tối ưu đến tất cả các nút được mở rộng. Cụ thể hơn, nếu một nút q được mở rộng thì đường đi rẻ nhất đến q đã được tìm thấy, tức là:

$$g(q) = g^*(q), \quad (2.10)$$

trong đó $g(q)$ là chi phí đường đi từ nút xuất phát s đến nút q theo đường đi hiện tại, còn $g^*(q)$ là chi phí của đường đi tối ưu từ nút xuất phát s đến nút q . Trong trường hợp này, việc cập nhật chi phí không bao giờ cần thiết sau lần mở rộng đầu tiên, mỗi nút chỉ được mở rộng đúng một lần và không có sự suy giảm hiệu năng do việc mở lại các nút [14].

Để minh họa cơ chế hoạt động của thuật toán A^* , có thể xem xét ví dụ tìm đường đi từ nút A đến nút đích G trên đồ thị hình 2.7, trong đó mỗi cạnh có chi phí xác định và mỗi nút được gán một giá trị heuristic $h(n)$ ước lượng chi phí còn lại đến đích (cụ thể: $h(A)=7, h(B)=5, h(C)=9, h(D)=6, h(E)=3, h(F)=5, h(G)=0, h(H)=3, h(J)=4$). Ban đầu, frontier chỉ chứa đường đi xuất phát $\langle A \rangle$ với $f = 0 + 7 = 7$. A^* mở rộng nút này, sinh ra ba đường đi con ứng với các nút kề của A , mỗi

đường đi được gán giá trị f riêng (tổng chi phí thực tế đã đi cộng với heuristic của nút cuối). Tại mỗi bước, A* luôn chọn đường đi có f nhỏ nhất trong frontier để mở rộng tiếp đây chính là điểm khác biệt so với lowest-cost-first search (chỉ dựa vào cost) và greedy best-first search (chỉ dựa vào h). Trong quá trình tìm kiếm, có lúc hai đường đi cùng có giá trị f bằng nhau, khi đó thuật toán không quy định rõ thứ tự chọn (có thể dùng tiêu chí phụ như chọn đường có h nhỏ hơn). Đáng chú ý, nhánh đi qua nút C, dù có vẻ hứa hẹn ban đầu, không bao giờ được mở rộng vì giá trị f của nó luôn lớn hơn các đường đi khác đang được xét. Điều này cho thấy A* có khả năng loại bỏ sớm các nhánh không thể dẫn đến lời giải tối ưu mà không cần khám phá toàn bộ không gian tìm kiếm phía sau nó. Cuối cùng, đường đi đến G được tìm thấy với tổng chi phí nhỏ nhất, và do hàm h ở đây là admissible (không bao giờ ước lượng cao hơn chi phí thực), lời giải này được đảm bảo là tối ưu [4].



Hình 2.7 Minh họa quá trình tìm kiếm của thuật toán A* trên đồ thị đơn giản

2.5.3 Hàm đánh giá chi phí trong thuật toán A*

Để xác định nút nào cần được mở rộng tiếp theo trong quá trình tìm kiếm, thuật toán A* sử dụng một hàm đánh giá chi phí $f(n)$ cho mỗi nút n . Hàm này đóng vai trò trung tâm trong toàn bộ cơ chế hoạt động của A*, quyết định thứ tự duyệt các nút và ảnh hưởng trực tiếp đến chất lượng cũng như hiệu quả của quá trình tìm kiếm. Hàm đánh giá chi phí $f(n)$ được định nghĩa là tổng chi phí của đường đi tối ưu bị ràng buộc phải đi qua nút n , từ nút xuất phát s đến nút đích được ưu tiên. Hàm này có thể được phân tách thành tổng của hai thành phần [12]

$$f(n) = g(n) + h(n), \quad (2.11)$$

trong đó $f(n)$ là tổng chi phí thực tế của đường đi tối ưu bị ràng buộc phải đi qua nút n , từ nút xuất phát s đến nút đích được ưu tiên, $g(n)$ là chi phí thực tế của đường đi tối ưu từ nút xuất phát s đến nút n , còn $h(n)$ là chi phí thực tế của đường đi tối ưu từ nút n đến nút đích được ưu tiên của n .

Tuy nhiên, trong thực tế, các giá trị $g(n)$ và $h(n)$ chính xác thường không thể tính được trực tiếp. Do đó, A^* sử dụng các ước lượng $\hat{g}(n)$ và $\hat{h}(n)$ để xấp xỉ hai thành phần này, tạo thành hàm ước lượng :

$$\hat{f}(n) = \hat{g}(n) + \hat{h}(n), \quad (2.12)$$

trong đó $\hat{f}(n)$ là giá trị ước lượng tổng chi phí đường đi tối ưu đi qua nút n , $\hat{g}(n)$ là chi phí của đường đi từ nút xuất phát s đến nút n với chi phí nhỏ nhất tìm được cho đến thời điểm hiện tại bởi A^* , còn $\hat{h}(n)$ là ước lượng chi phí của đường đi tối ưu từ nút n đến nút đích được ưu tiên.

Thành phần $\hat{g}(n)$ tích lũy chi phí chuyển tiếp $c(r, r')$ của tất cả các chuyển tiếp $r \rightarrow r'$ xảy ra trên đường đi hiện tại từ nút xuất phát s đến nút n :

$$\hat{g}(n) = \sum_{r \rightarrow r' \in p(n)} c(r, r'), \quad (2.13)$$

trong đó $\hat{g}(n)$ là tổng chi phí tích lũy trên đường đi hiện tại từ s đến n , $p(n)$ là đường đi hiện tại tốt nhất đã biết từ nút xuất phát s đến nút n , còn $c(r, r')$ là chi phí của cung chuyển tiếp từ nút r đến nút r' , với $c(r, r') \geq \delta > 0$, trong đó δ là một hằng số dương. Lưu ý rằng $\hat{g}(n)$ luôn thỏa mãn bất đẳng thức:

$$\hat{g}(n) \geq g(n), \quad (2.14)$$

trong đó $g(n)$ là chi phí thực tế của đường đi tối ưu từ s đến n . Điều này có nghĩa là chi phí đường đi tốt nhất tìm được cho đến thời điểm hiện tại luôn không nhỏ hơn chi phí đường đi tối ưu thực sự. Bất đẳng thức xảy ra khi A^* đã tìm được đường đi tối ưu đến n [14].

Trong khi đó, thành phần $\hat{h}(n)$ là ước lượng chi phí từ nút n đến nút đích, được tính dựa trên thông tin từ miền bài toán. Đây là thành phần mang tính heuristic, phân biệt A^* với các thuật toán tìm kiếm không có thông tin như Dijkstra. Để đảm bảo tính chấp nhận được của A^* , $\hat{h}(n)$ phải là một cận dưới của chi phí thực tế $h(n)$, tức là phải thỏa mãn điều kiện:

$$\hat{h}(n) \leq h(n), \quad (2.15)$$

trong đó $\hat{h}(n)$ là giá trị ước lượng chi phí từ nút n đến nút đích, còn $h(n)$ là chi phí thực tế của đường đi tối ưu từ nút n đến nút đích. Khi điều kiện (2.15) được thỏa mãn, hàm heuristic $\hat{h}(n)$ được gọi là chấp nhận được, đây là điều kiện cần thiết để đảm bảo A^* luôn tìm ra đường đi tối ưu [14].

Hàm $f(n)$ còn tính chất quan trọng liên quan đến chi phí tối ưu. Tại nút xuất phát s , $f(s) = h(s)$, tức bằng chi phí của đường đi tối ưu không bị ràng buộc từ s đến nút đích được ưu tiên. Với mỗi nút n nằm trên đường đi tối ưu, $f(n) = f(s)$, trong khi với mỗi nút n không nằm trên đường đi tối ưu, $f(n) > f(s)$. Từ đó, A^* sử

dùng $\hat{f}(n)$ như một ước lượng của $f(n)$ để lựa chọn nút mở rộng tiếp theo. Nút có giá trị $\hat{f}(n)$ nhỏ nhất trong danh sách OPEN sẽ được ưu tiên mở rộng, vì đây là nút có tiềm năng cao nhất để nằm trên đường đi tối ưu [12]. Như vậy, hàm đánh giá $\hat{f}(n)$ cân bằng giữa hai yếu tố trái ngược nhau. Thông qua $\hat{g}(n)$, A^* ưu tiên các nút đã được tiếp cận với chi phí thực tế thấp, đảm bảo không bỏ qua các đường đi ngắn đã đi qua. Thông qua $\hat{h}(n)$, thuật toán định hướng tìm kiếm về phía nút đích, tránh việc duyệt toàn bộ không gian trạng thái một cách mù quáng. Sự kết hợp này cho phép A^* tìm kiếm có định hướng nhưng vẫn đảm bảo tính tối ưu, phân biệt nó với tìm kiếm tham lam chỉ sử dụng $\hat{h}(n)$ hoặc thuật toán Dijkstra chỉ sử dụng $\hat{g}(n)$. Cụ thể, khi $\hat{h}(n) = 0$ với mọi n , A^* tương đương với thuật toán Dijkstra và mở rộng các nút theo thứ tự chi phí thực tế tăng dần, còn khi $\hat{g}(n)$ bị bỏ qua, A^* trở thành tìm kiếm tham lam chỉ dựa trên heuristic [14].

2.5.4 Hàm heuristic trong thuật toán A^*

Hàm heuristic $\hat{h}(n)$ là thành phần cốt lõi tạo nên sức mạnh của thuật toán A^* , cho phép thuật toán sử dụng thông tin đặc thù từ miền bài toán để định hướng quá trình tìm kiếm về phía nút đích một cách hiệu quả. Việc lựa chọn và thiết kế hàm heuristic phù hợp có ảnh hưởng trực tiếp đến cả tính đúng đắn lẫn hiệu quả tính toán của thuật toán A^* [12]. Điều kiện cơ bản để đảm bảo tính chấp nhận được của A^* là hàm heuristic $\hat{h}(n)$ phải là một cận dưới của chi phí thực tế, tức là phải thỏa mãn với mọi nút n :

$$\hat{h}(n) \leq h(n), \quad (2.16)$$

trong đó $\hat{h}(n)$ là giá trị ước lượng heuristic tại nút n , còn $h(n)$ là chi phí thực tế của đường đi tối ưu từ nút n đến nút đích được ưu tiên. Khi điều kiện (2.16) được thỏa mãn, hàm heuristic được gọi là chấp nhận được. Định lý cơ bản của thuật toán A^* phát biểu rằng nếu $\hat{h}(n) \leq h(n)$ với mọi n , thì A^* là thuật toán chấp nhận được. Trường hợp đặc biệt $\hat{h}(n) = 0$ với mọi n luôn thỏa mãn điều kiện này, khi đó A^* tương đương với thuật toán Dijkstra [12].

Ngoài tính chấp nhận được, một tính chất quan trọng khác là tính nhất quán. Hàm heuristic $h(n)$ được gọi là nhất quán nếu với bất kỳ cặp nút n' và n nào thì bất đẳng thức sau đây được thỏa mãn:

$$h(n') \leq k(n', n) + h(n), \quad (2.17)$$

trong đó $h(n')$ là giá trị heuristic tại nút n' , $h(n)$ là giá trị heuristic tại nút n , còn $k(n', n)$ là chi phí của đường đi tối ưu từ nút n' đến nút n . Tính nhất quán kéo theo tính chấp nhận được nhưng không có chiều ngược lại, do đó $I_CON \subseteq I_AD$, tức tập các bài toán có heuristic nhất quán là tập con của tập các bài toán có heuristic chấp nhận được [19], [20]. Khi tính nhất quán được thỏa mãn, A^* không bao giờ cần mở lại một nút đã đóng, vì đường đi tối ưu đến nút đó đã được tìm thấy ngay

lần mở rộng đầu tiên. Chất lượng của hàm heuristic ảnh hưởng trực tiếp đến số lượng nút mà A^* phải mở rộng. Nếu $\hat{h}_1(n)$ và $\hat{h}_2(n)$ là hai hàm heuristic chấp nhận được với $\hat{h}_1(n) \leq \hat{h}_2(n)$ với mọi n , thì A^* sử dụng $\hat{h}_2$ sẽ không bao giờ mở rộng nhiều nút hơn A^* sử dụng $\hat{h}_1$. Điều này có nghĩa là hàm heuristic càng chính xác, tức là càng gần với $h(n)$ thực tế, thì A^* càng hiệu quả về mặt tính toán. Tuy nhiên, chi phí tính toán $\hat{h}(n)$ cũng cần được cân nhắc, nếu quá lớn, lợi ích từ việc giảm số nút mở rộng có thể bị triệt tiêu [12].

2.5.5 Ưu điểm và hạn chế của thuật toán A^*

Thuật toán A^* có một số ưu điểm nổi bật so với các thuật toán tìm kiếm khác. Thứ nhất, A^* đảm bảo tìm ra đường đi tối ưu khi hàm heuristic thỏa mãn điều kiện chấp nhận được. Thứ hai, A^* được chứng minh là tối ưu về mặt hiệu quả tính toán trong lớp các thuật toán tìm kiếm tốt nhất sử dụng cùng thông tin heuristic, theo nghĩa không bao giờ mở rộng nhiều nút hơn mức cần thiết để đảm bảo tìm ra đường đi tối ưu. Thứ ba, bằng cách điều chỉnh hàm heuristic $\hat{h}(n)$, A^* cho phép cân bằng linh hoạt giữa tính tối ưu và hiệu quả tính toán, tạo ra một họ các thuật toán phù hợp với nhiều bài toán khác nhau [12], [13].

Hạn chế lớn nhất là yêu cầu bộ nhớ cao, vì thuật toán phải lưu trữ toàn bộ các nút trong danh sách OPEN và CLOSED trong suốt quá trình tìm kiếm, dẫn đến mức tiêu thụ bộ nhớ có thể lên đến mức hàm mũ theo độ sâu tìm kiếm. Ngoài ra, trong trường hợp xấu nhất, thời gian tính toán của A^* cũng có thể là hàm mũ theo độ sâu tìm kiếm khi hàm heuristic kém chính xác. Hiệu quả của A^* phụ thuộc rất lớn vào chất lượng hàm heuristic, một hàm heuristic không tốt có thể khiến A^* phải duyệt gần như toàn bộ không gian trạng thái. Hơn nữa, A^* cũng gặp khó khăn trong các môi trường động, nơi chi phí cạnh hoặc cấu trúc đồ thị thay đổi trong quá trình tìm kiếm, đòi hỏi phải lập kế hoạch lại từ đầu [12], [14].

2.5.6 Thuật toán Weighted A^*

Thuật toán Weighted A^* (WA^*) là một biến thể gần đúng của thuật toán A^* , được Pohl đề xuất lần đầu vào năm 1970 nhằm tăng tốc độ tìm kiếm bằng cách chấp nhận một mức độ sai lệch có kiểm soát so với nghiệm tối ưu [13]. Thay vì tìm kiếm chính xác đường đi ngắn nhất, WA^* ưu tiên tốc độ xử lý bằng cách điều chỉnh hàm đánh giá để thiên về phía đích nhiều hơn, từ đó giảm đáng kể số lượng nút cần mở rộng trong quá trình tìm kiếm. Ý tưởng cốt lõi của WA^* là thay đổi hàm đánh giá $f(q)$ bằng cách thêm một trọng số $\varepsilon > 0$ vào thành phần heuristic $h(q)$. Cụ thể, thay vì sử dụng hàm đánh giá $f(q) = g(q) + h(q)$ như trong A^* truyền thống, WA^* sử dụng hàm đánh giá:

$$f^\uparrow(q) = g(q) + (1 + \varepsilon) \cdot h(q), \quad (2.18)$$

trong đó $f^\uparrow(q)$ là giá trị đánh giá chi phí có trọng số tại nút q , $g(q)$ là chi phí đường đi thực tế từ nút xuất phát s đến nút q theo đường đi hiện tại, $h(q)$ là ước lượng heuristic chi phí từ nút q đến nút đích, còn ε là tham số trọng số, với $\varepsilon > 0$. Khi $\varepsilon = 0$, hàm đánh giá $f^\uparrow(q)$ trở về dạng $f(q) = g(q) + h(q)$ của thuật toán A^* truyền thống. Khi $\varepsilon > 0$, thành phần heuristic được khuếch đại, khiến thuật toán ưu tiên mở rộng các nút có ước lượng chi phí đến đích nhỏ hơn. Nhờ đó, quá trình tìm kiếm tập trung theo hướng đích và bỏ qua nhiều nhánh không triển vọng, giúp giảm số nút phải xét và rút ngắn thời gian lập kế hoạch [14].

Mặc dù WA^* không đảm bảo tìm ra đường đi tối ưu tuyệt đối, thuật toán vẫn cung cấp một cận trên có thể kiểm chứng được cho độ lệch của nghiệm so với nghiệm tối ưu. Tính chất này được gọi là tính ε -chấp nhận được: nếu $h(q)$ là heuristic chấp nhận được, WA^* luôn tìm ra đường đi có chi phí không vượt quá $(1+\varepsilon)$ lần chi phí tối ưu. Cụ thể, với mọi nút q tại thời điểm mở rộng, điều kiện sau đây được thỏa mãn:

$$f(q) \leq (1 + \varepsilon) \cdot C^*, \quad (2.19)$$

trong đó $f(q)$ là giá trị đánh giá chi phí tại nút q tại thời điểm mở rộng, còn C^* là chi phí của đường đi tối ưu từ nút xuất phát đến nút đích. Điều kiện (2.19) cho phép người dùng kiểm soát trực tiếp sự cân bằng giữa chất lượng nghiệm và tốc độ tìm kiếm thông qua việc điều chỉnh giá trị ε . Khi ε càng nhỏ, nghiệm tìm được càng gần tối ưu nhưng tốc độ tìm kiếm chậm hơn do phải xét nhiều nút hơn. Ngược lại, khi ε lớn, thuật toán bỏ qua nhiều nút hơn và đạt tốc độ tìm kiếm cao hơn nhưng chấp nhận mức sai lệch lớn hơn so với nghiệm tối ưu [14].

Trọng số ε đóng vai trò then chốt trong việc điều chỉnh hành vi của WA^* . Khi ε tăng dần, hàm đánh giá $f^\uparrow(q)$ ngày càng bị chi phối bởi thành phần $h(q)$, khiến thuật toán có xu hướng tiệm cận về hành vi của tìm kiếm tham lam theo hướng đích. Điều này có nghĩa là thuật toán ưu tiên mở rộng các nút gần đích nhất theo ước lượng heuristic mà ít quan tâm đến chi phí đường đi thực tế $g(q)$. Ngược lại, khi ε tiệm cận về 0, WA^* dần trở về hành vi của A^* truyền thống với đầy đủ tính tối ưu. Trong thực tế, việc lựa chọn giá trị ε phù hợp phụ thuộc vào yêu cầu cụ thể của bài toán. Với các ứng dụng yêu cầu thời gian thực như điều hướng robot tự hành, một giá trị ε vừa phải thường được ưu tiên để cân bằng giữa chất lượng đường đi và tốc độ lập kế hoạch [14].

2.5.7 So sánh giữa thuật toán A^* và Weighted A^*

Để làm rõ sự khác biệt giữa A^* và WA^* , phần này trình bày so sánh theo sáu tiêu chí chính: hàm đánh giá, tính tối ưu, số lượng nút mở rộng, thời gian lập kế hoạch, chiều dài quỹ đạo và độ phức tạp thuật toán. Trước tiên, về hàm đánh giá, A^* sử dụng hàm đánh giá tiêu chuẩn tại công thức (2.20), trong đó chi phí đường

đi thực tế $g(q)$ và ước lượng heuristic $h(q)$ có vai trò ngang nhau, trong công thức (2.21) WA^* thêm trọng số W vào thành phần heuristic, khiến thuật toán thiên về hướng đích nhiều hơn:

$$f(q) = g(q) + h(q), \quad (2.20)$$

$$f^\uparrow(q) = g(q) + (1 + W) \cdot h(q), \quad (2.21)$$

sự khác biệt này có ý nghĩa quan trọng: trong A^* , quyết định mở rộng nút dựa đồng thời trên cả chi phí đã đi và ước lượng chi phí còn lại. Trong WA^* , ước lượng chi phí còn lại được nhân lên $(1+W)$ lần, khiến thuật toán ưu tiên các nút gần đích hơn và bỏ qua nhiều nhánh tìm kiếm không triển vọng [14].

Về tính tối ưu A^* đảm bảo tìm ra đường đi tối ưu tuyệt đối khi heuristic $h(q)$ là chấp nhận được, tức là $h(q) \leq h^*(q)$ với mọi nút q , trong đó $h^*(q)$ là chi phí thực tế từ q đến đích. Điều này có nghĩa là đường đi tìm được bởi A^* luôn có chi phí bằng C^* . WA^* không đảm bảo tính tối ưu tuyệt đối nhưng đảm bảo tính chấp nhận được có giới hạn: chi phí đường đi tìm được C_{WA^*} thỏa mãn:

$$C^* \leq C_{WA^*} \leq (1 + W) \cdot C^*, \quad (2.22)$$

nhờ đó, người dùng có thể chủ động kiểm soát mức độ sai lệch so với tối ưu thông qua việc điều chỉnh W . Khi $W = 0$, WA^* trở về A^* với tính tối ưu tuyệt đối [14].

Khác biệt rõ nhất giữa hai thuật toán nằm ở số lượng nút mở rộng. Trong A^* , mọi nút q thỏa mãn $f(q) < C^*$ đều bắt buộc phải được mở rộng trước khi thuật toán kết thúc [17], khiến A^* phải khám phá toàn bộ vùng không gian tìm kiếm có chi phí nhỏ hơn C^* , đặc biệt tốn kém trong các môi trường có không gian tìm kiếm lớn. Trong WA^* , điều kiện mở rộng được nới lỏng thành $f(q) \leq (1+W) \cdot C^*$, cho phép thuật toán bỏ qua nhiều nút mà A^* bắt buộc phải xét, nhờ đó số lượng nút mở rộng của WA^* thường ít hơn đáng kể so với A^* , đặc biệt khi W lớn. Hệ quả trực tiếp của điều này thể hiện ở thời gian lập kế hoạch. Do mở rộng ít nút hơn, WA^* thường có thời gian lập kế hoạch ngắn hơn A^* trên cùng một bản đồ và cùng cặp điểm xuất phát, đích đến, và sự chênh lệch này càng rõ rệt khi không gian tìm kiếm lớn hoặc có nhiều vật cản. Trong thực tế, với các giá trị W vừa phải, WA^* có thể giảm thời gian lập kế hoạch đáng kể trong khi độ lệch so với nghiệm tối ưu vẫn nằm trong giới hạn chấp nhận được [14].

Về chiều dài quỹ đạo, vì A^* luôn tìm ra đường đi tối ưu, quỹ đạo tạo ra bởi A^* có chiều dài ngắn nhất có thể trên bản đồ. Quỹ đạo của WA^* thì có thể dài hơn một lượng tối đa là $W \times 100\%$ so với quỹ đạo tối ưu theo bất đẳng thức (2.22), tuy nhiên trong thực tế độ lệch thường nhỏ hơn nhiều so với cận lý thuyết này, đặc biệt khi heuristic có chất lượng tốt [14].

Cuối cùng, về độ phức tạp thuật toán, đối với A^* , hàm đánh giá kết hợp chi phí thực tế và ước lượng heuristic theo công thức (2.23), trong đó $g(n)$ là chi phí thực tế từ nút xuất phát đến nút n , còn $h(n)$ là hàm heuristic ước lượng chi phí từ n đến đích. Độ phức tạp thời gian trong trường hợp xấu nhất của A^* được xác định theo (2.24), với b là hệ số phân nhánh trung bình và d là độ sâu lời giải. Độ phức tạp không gian cũng tương tự, do thuật toán phải lưu toàn bộ danh sách OPEN và CLOSED. Khi hàm heuristic thỏa mãn đồng thời điều kiện chấp nhận được (2.25) và điều kiện nhất quán (2.26), A^* đảm bảo tìm được đường đi tối ưu.

$$f(n) = g(n) + h(n) \quad (2.23)$$

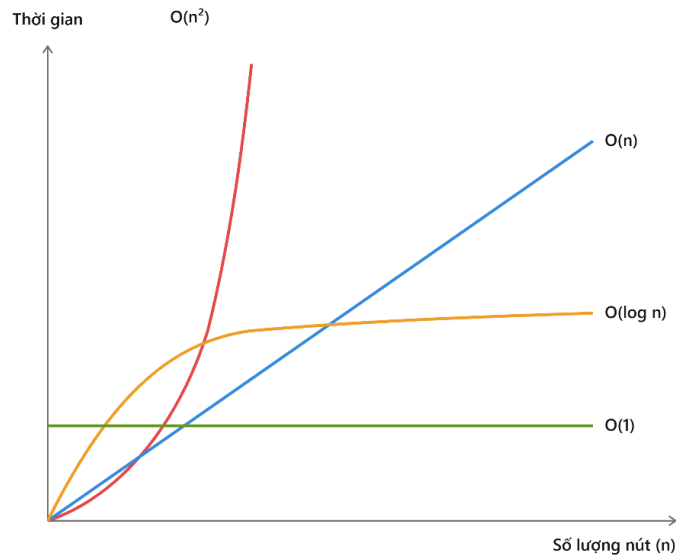
$$O(b^d) \quad (2.24)$$

$$h(n) \leq h^*(n), \forall n \quad (2.25)$$

$$h(n) \leq c(n, n') + h(n') \quad (2.26)$$

$$f(n) = g(n) + W \cdot h(n), W > 1 \quad (2.27)$$

$$C_{WA^*} \leq W \cdot C^* \quad (2.28)$$



Hình 2.8 Biểu đồ độ phức tạp thuật toán

Đối với WA^* , hàm đánh giá được điều chỉnh bằng hệ số trọng số $W > 1$ theo công thức (2.27), khiến tìm kiếm thiên lệch về phía đích. Đường tìm được đảm bảo trong giới hạn nêu tại (2.28), trong đó C^* là chi phí đường tối ưu. Khi $W = 1$, WA^* tương đương A^* tiêu chuẩn. Khi W tăng, số nút cần mở rộng giảm đáng kể. Về lý thuyết, cả A^* và WA^* đều có độ phức tạp thời gian và không gian trong trường hợp xấu nhất là $O(b^d)$ [12]. Tuy nhiên trong thực tế, WA^* thường hoạt động hiệu quả hơn đáng kể nhờ giảm số lượng nút cần mở rộng. Nhờ trọng số W

làm thiên lệch tìm kiếm về phía đích, WA^* có xu hướng tìm ra nghiệm ở độ sâu thấp hơn, dẫn đến độ phức tạp thực tế thấp hơn nhiều so với A^* trong phần lớn các trường hợp. Hình 2.8 minh họa sự khác biệt này: A^* với heuristic tốt kéo độ phức tạp thực tế xuống gần mức $O(\log n)$, trong khi WA^* với W phù hợp cho phép hệ thống tiếp cận mức $O(1)$ về số nút mở rộng trong nhiều tình huống thực tế, đây là lý do thuật toán này được ưu tiên trong các hệ thống điều hướng robot có yêu cầu phản hồi nhanh [14].

2.6 HỆ THỐNG ĐIỀU HƯỚNG CHO ROBOT DI ĐỘNG TỰ HÀNH

2.6.1 Tổng quan về hệ thống điều hướng

Hệ thống điều hướng là thành phần cốt lõi của robot di động tự hành, cho phép robot di chuyển từ vị trí hiện tại đến đích mong muốn một cách an toàn và hiệu quả trong môi trường thực tế. Không giống với các hệ thống điều khiển truyền thống hoạt động trên quỹ đạo định sẵn, robot tự hành phải liên tục xử lý thông tin từ môi trường xung quanh, đưa ra quyết định theo thời gian thực và thích nghi với các tình huống bất ngờ như vật cản xuất hiện đột ngột hay thay đổi về địa hình [2]. Một hệ thống điều hướng hoàn chỉnh thường bao gồm bốn chức năng chính liên kết chặt chẽ với nhau [2]. Thứ nhất là định vị, tức xác định vị trí và hướng hiện tại của robot trong môi trường. Thứ hai là xây dựng bản đồ, nhằm tạo lập và duy trì mô hình không gian của môi trường xung quanh. Thứ ba là lập kế hoạch đường đi, trong đó hệ thống tính toán quỹ đạo di chuyển tối ưu từ vị trí hiện tại đến đích. Thứ tư là điều khiển chuyển động, thực thi đường đi đã lập kế hoạch thông qua các tín hiệu điều khiển cơ cấu chấp hành.

2.6.2 Định vị, xây dựng bản đồ và lập kế hoạch đường đi

Định vị là bài toán xác định trạng thái pose của robot, bao gồm tọa độ vị trí (x,y) và góc định hướng θ tại mọi thời điểm trong quá trình hoạt động, đây là điều kiện tiên quyết để robot có thể thực hiện lập kế hoạch đường đi chính xác. Có hai phương pháp định vị chính đang được sử dụng. Phương pháp thứ nhất là định vị tương đối, trong đó hệ thống tích lũy các số đo từ cảm biến chuyển động như encoder và IMU để ước lượng vị trí robot so với điểm xuất phát, với ưu điểm là đơn giản, tần suất cập nhật cao và không phụ thuộc vào bản đồ môi trường, tuy nhiên sai số tích lũy theo thời gian khiến phương pháp này không đủ tin cậy khi dùng độc lập trong thời gian dài. Phương pháp thứ hai là định vị tuyệt đối, trong đó robot xác định vị trí dựa trên các đặc trưng nhận biết được trong môi trường hoặc dữ liệu bản đồ đã biết trước [2].

Xây dựng bản đồ là quá trình robot thu thập và tổ chức thông tin về cấu trúc vật lý của môi trường, đóng vai trò là nền tảng không gian cho tất cả các quyết định điều hướng sau đó. Các dạng biểu diễn bản đồ phổ biến hiện nay gồm bốn

loại chính. Bản đồ lưới chia môi trường thành các ô vuông đều nhau, mỗi ô lưu trữ xác suất bị chiếm dụng bởi vật cản, đây là dạng phổ biến nhất trong robot di động trong nhà do dễ xây dựng, dễ cập nhật và tương thích tốt với các thuật toán lập kế hoạch đường đi. Bản đồ đặc trưng lưu trữ vị trí của các đặc trưng nổi bật trong môi trường như góc tường, cột hay biển báo, nhẹ hơn về bộ nhớ nhưng đòi hỏi quá trình trích xuất và đối sánh đặc trưng phức tạp. Bản đồ topo biểu diễn môi trường dưới dạng đồ thị các nút và cạnh, trong đó mỗi nút là một vị trí đặc trưng và cạnh thể hiện khả năng di chuyển giữa các vị trí, thích hợp cho lập kế hoạch cấp cao trong môi trường quy mô lớn. Bản đồ 3D dạng đám mây điểm được tạo từ LiDAR 3D hoặc camera độ sâu, cung cấp thông tin chi tiết về hình dạng không gian ba chiều và thường được dùng trong các ứng dụng ngoài trời hoặc yêu cầu độ chính xác cao [2].

Lập kế hoạch đường đi là quá trình xác định một lộ trình khả thi và tối ưu để robot di chuyển từ vị trí hiện tại đến đích mong muốn trong khi tránh va chạm với các vật cản trong môi trường, hoạt động dựa trên bản đồ môi trường đã được xây dựng và thông tin định vị từ hệ thống SLAM. Về mặt kiến trúc, lập kế hoạch đường đi thường được tổ chức theo hai cấp độ phối hợp chặt chẽ với nhau. Lập kế hoạch toàn cục tính toán lộ trình tổng thể từ điểm xuất phát đến đích dựa trên bản đồ môi trường đã biết trước, cung cấp định hướng chung cho robot nhưng không phản ứng được với các vật cản động xuất hiện ngoài bản đồ. Lập kế hoạch cục bộ điều chỉnh lộ trình theo thời gian thực dựa trên dữ liệu cảm biến tức thời, xử lý các vật cản chưa được ghi nhận trên bản đồ hoặc các đối tượng di chuyển thông qua các phương pháp tiêu biểu như DWA, VFH và TEB, trong đó bộ lập kế hoạch liên tục tính toán lại quỹ đạo ngắn hạn trong một cửa sổ không gian hữu hạn xung quanh robot. Chất lượng của lập kế hoạch đường đi phụ thuộc trực tiếp vào độ chính xác của bản đồ và kết quả định vị từ hệ thống SLAM, vì sai lệch trong bản đồ lưới do tích lũy lỗi odometry hoặc nhiễu cảm biến có thể dẫn đến lộ trình không khả thi hoặc va chạm với vật cản thực tế, do đó trong các hệ thống điều hướng hiện đại, vòng lặp SLAM và bộ lập kế hoạch đường đi thường được tích hợp chặt chẽ để cho phép cập nhật lộ trình ngay khi bản đồ được hiệu chỉnh [2].

2.7 TỔNG QUAN VỀ PID

2.7.1 Khái niệm về PID

Bộ điều khiển PID là một cơ cấu điều khiển vòng kín sử dụng tín hiệu sai lệch giữa giá trị đặt và giá trị thực tế của hệ thống để tính toán tín hiệu điều khiển đầu ra. Đây là bộ điều khiển phản hồi phổ biến nhất trong công nghiệp và tự động hóa nhờ cấu trúc đơn giản, dễ chỉnh định và khả năng đáp ứng tốt với nhiều đối tượng điều khiển khác nhau. Nguyên lý hoạt động dựa trên việc liên tục đo sai

lệch $e(t)$ theo công thức (2.29), trong đó $r(t)$ là giá trị đặt và $y(t)$ là giá trị đầu ra thực tế của hệ thống [3].

$$e(t) = r(t) - y(t) \quad (2.29)$$

2.7.2 Cấu trúc và công thức của PID

Bao gồm ba thành phần là tỉ lệ (P), tích phân (I) và vi phân (D), được kết hợp song song để tạo ra tín hiệu điều khiển $u(t)$. Công thức tổng quát trong miền thời gian được cho bởi (2.30), còn hàm truyền tương ứng trong miền Laplace được biểu diễn theo (2.31). Trong đó K_p là hệ số khuếch đại tỉ lệ, K_i là hệ số khuếch đại tích phân, K_d là hệ số khuếch đại vi phân và $e(t)$ là sai lệch điều khiển tại thời điểm t [3].

$$u(t) = K_p \cdot e(t) + K_i \int_0^t e(\tau) d\tau + K_d \cdot \frac{de(t)}{dt} \quad (2.30)$$

$$G_{PID}(s) = K_p + \frac{K_i}{s} + K_d \cdot s \quad (2.31)$$

2.7.3 Vai trò từng khâu của PID

Khâu tỉ lệ P tạo ra tín hiệu điều khiển tỉ lệ trực tiếp với sai lệch hiện tại theo công thức (2.32). Khâu P phản ứng tức thời với sai lệch, giúp đẩy nhanh tốc độ đáp ứng, tuy nhiên nếu chỉ dùng riêng khâu P, hệ thống thường tồn tại sai lệch tĩnh ở trạng thái xác lập. Khâu tích phân I tạo ra tín hiệu điều khiển tỉ lệ với tổng tích lũy sai lệch theo thời gian theo công thức (2.33), có tác dụng loại bỏ hoàn toàn sai lệch tĩnh, tuy nhiên tích phân tích lũy quá lớn có thể gây hiện tượng vọt lố (overshoot) và dao động. Khâu vi phân D tạo ra tín hiệu điều khiển tỉ lệ với tốc độ thay đổi của sai lệch theo công thức (2.34), có tính chất dự báo xu hướng thay đổi của sai lệch, giúp giảm vọt lố và cải thiện độ ổn định, tuy nhiên khâu D nhạy cảm với nhiễu tần số cao nên thường được kết hợp với bộ lọc thực tế [3]:

$$u_P(t) = K_p \cdot e(t) \quad (2.32)$$

$$u_I(t) = K_i \int_0^t e(\tau) d\tau \quad (2.33)$$

$$u_D(t) = K_d \cdot \frac{de(t)}{dt} \quad (2.34)$$

Bộ điều khiển PID có cấu trúc đơn giản, dễ hiểu và dễ triển khai trong thực tế, không yêu cầu mô hình toán học chính xác của đối tượng điều khiển, đồng thời linh hoạt khi có thể dùng dưới dạng P, PI, PD hoặc PID tùy yêu cầu hệ thống. Về nhược điểm, hiệu quả của PID giảm đáng kể với các đối tượng phi tuyến, có trễ lớn hoặc thông số thay đổi theo thời gian, việc chỉnh định đồng thời ba tham số có thể phức tạp đặc biệt với hệ thống bậc cao, khâu tích phân có thể gây bão hòa khi tín hiệu điều khiển bị giới hạn bởi cơ cấu chấp hành, và khâu vi phân khuếch đại nhiễu đo lường nên đòi hỏi phải lọc tín hiệu trước khi sử dụng [3].

2.8 GIỚI THIỆU LINH KIỆN

2.8.1 Động cơ JGB37-520

Dựa vào ước lượng trọng lượng tổng thể của robot nhóm đã quyết định sử dụng động cơ JGB37-520 để đảm bảo khả năng chịu tải và di chuyển ổn định của robot, động cơ JGB37 xem ở hình 2.9. Động cơ sẽ được điều khiển thông qua IC driver TB6612FNG được quản lý bởi khối xử lý trung tâm.



Hình 2.9 Động cơ JGB37-520

Bảng 2.1 Thông số kỹ thuật động cơ JGB37-520

Điện áp hoạt động	12V
Đường kính trục	6 mm
Dòng có tải tối đa	1.2A
Tốc độ	60 rpm
Moment xoắn cực đại	4,8 kg.cm

Bảng 2.1 mô tả một số thông số quan trọng của động cơ JGB37-520. Động cơ hoạt động ở mức điện áp DC 12V, đường kính trục 6mm, cần lựa chọn bánh xe với khớp nối phù hợp với trục 6mm. Dòng tiêu hao tối đa của động cơ khi có tải là 1.2A, tốc độ 60rpm tương ứng với 60 vòng trên phút, lực moment xoắn cực đại 4,8kg.cm.

2.8.2 IC điều khiển động cơ TB6612FNG

Hình 2.10 là IC TB6612FNG là IC điều khiển động cơ DC sử dụng cầu H kép, cho phép điều khiển đồng thời hai động cơ một chiều hoặc một động cơ bước. Mạch hỗ trợ điều khiển chiều quay và tốc độ động cơ thông qua các tín hiệu điều khiển số và tín hiệu PWM từ vi điều khiển. Với kích thước nhỏ gọn, hiệu suất

cao và mức tiêu thụ công suất thấp, TB6612FNG được sử dụng rộng rãi trong các ứng dụng robot di động và hệ thống điều khiển động cơ công suất nhỏ.



Hình 2.10 IC driver TB6612FNG

Bảng 2.2 Thông số kỹ thuật TB6612FNG

Thông số	Giá trị
Model	TB6612FNG
Điện áp logic (VCC)	2,7 V – 5,5 V
Điện áp động cơ (VM)	4,5 V – 13,5 V
Số kênh điều khiển	2 động cơ DC
Dòng liên tục tối đa	1,2 A/kênh
Dòng đỉnh tối đa	3,2 A/kênh
Điều khiển tốc độ	PWM
Điều khiển chiều quay	Cầu H (H-Bridge)

Bảng 2.2 mô tả thông số kỹ thuật của IC điều khiển động cơ TB6612FNG. Mức điện áp hoạt động của IC ở khoảng 2,7V tới 5,5V, khuyến nghị sử dụng 3,3V. Điều khiển được các loại động cơ DC có mức điện áp hoạt động tối đa 13,5V. có 2 kênh mỗi kênh điều khiển động cơ và điều khiển bằng phương pháp PWM.

2.8.3 Cảm biến Lidar A1M8

Hình 2.11 là Cảm biến LiDAR A1M8 là cảm biến đo khoảng cách bằng laser được sử dụng phổ biến trong các ứng dụng robot tự hành và robot di động. Cảm biến có khả năng quét môi trường xung quanh theo góc 360°, thu thập dữ liệu khoảng cách với tần số cao để xây dựng bản đồ và phát hiện vật cản. Với phạm vi đo lên đến khoảng 12 m cùng khả năng giao tiếp dữ liệu thông qua cổng UART, A1M8 phù hợp cho các ứng dụng định vị, dẫn đường và tránh vật cản

trong môi trường trong nhà. Nhờ kích thước nhỏ gọn, độ chính xác tương đối cao và khả năng tích hợp dễ dàng với ROS 2, cảm biến được lựa chọn làm cảm biến chính phục vụ quá trình nhận thức môi trường của robot.



Hình 2.11 Cảm biến lidar A1M8

Bảng 2.3 Thông số kỹ thuật cảm biến lidar A1M8

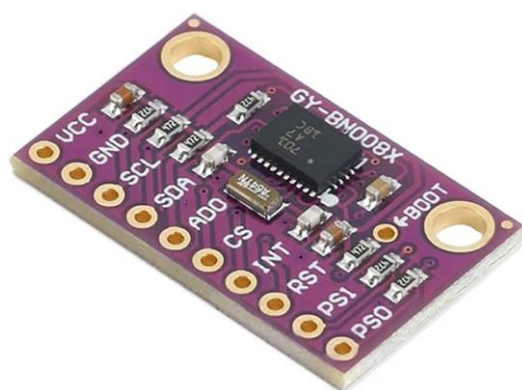
Thông số	Giá trị
Model	Triangulation LiDAR RPLIDAR A1M8-R6 360° Laser Range Scanner
Phương pháp đo	Triangulation
Động cơ	DC Motor
Điện áp hoạt động	5 VDC
Phạm vi quét	0,15 m – 12 m
Tốc độ lấy mẫu	8 kHz
Tần số quét	5 Hz – 10 Hz
Độ phân giải góc	0,36° (tại 8 Hz)
Độ phân giải khoảng cách	< 1% khoảng cách (< 12 m)
Độ chính xác ở tầm gần	1% khoảng cách (< 3 m) 2% khoảng cách (3 m – 5 m) 2,5% khoảng cách (5 m – 12 m)
Giao tiếp	UART (115200 bps)
Môi trường sử dụng	Trong nhà
Nhiệt độ hoạt động	0 °C – 40 °C
Kích thước	96,74 × 70,28 × 55 mm
Trọng lượng	170 g

Bảng 2.3 trình bày các thông số kỹ thuật chính của cảm biến LiDAR RPLIDAR A1M8. Cảm biến sử dụng nguyên lý đo tam giác, có khả năng quét 360° với phạm vi đo từ 0,15m đến 12m, với độ phân giải góc 0,36°, tốc độ lấy mẫu đạt 8.000 điểm đo mỗi giây và tần số quét từ 5 Hz đến 10 Hz. Thiết bị cung

cấp dữ liệu khoảng cách với tốc độ lấy mẫu 8 kHz, giao tiếp UART và hoạt động ở điện áp 5 VDC, phù hợp cho các ứng dụng robot tự hành trong môi trường trong nhà

2.8.4 Cảm biến gia tốc góc BNO080

Hình 2.12 là cảm biến gia tốc góc BNO080, cảm biến đo chuyển động 9 bậc tự do tích hợp gia tốc kế, con quay hồi chuyển và từ kế trong cùng một module. Cảm biến có khả năng cung cấp dữ liệu về gia tốc, vận tốc góc, góc định hướng thông qua các thuật toán hợp nhất cảm biến được tích hợp sẵn. Nhờ khả năng xử lý dữ liệu trực tiếp trên cảm biến, BNO080 giúp giảm tải tính toán cho bộ xử lý trung tâm và được sử dụng rộng rãi trong các ứng dụng robot tự hành, thiết bị định vị và hệ thống theo dõi chuyển động.



Hình 2.12 Cảm biến gia tốc góc BNO080

Bảng 2.4 Thông số kỹ thuật cảm biến gia tốc góc BNO080

Thông số	Giá trị
Loại cảm biến	IMU 9 bậc tự do (9-DOF)
Thành phần cảm biến	Gia tốc kế, con quay hồi chuyển, từ kế 3 trục
Điện áp hoạt động	2,4 V – 3,6 V
Giao tiếp	I2C, SPI, UART
Dải đo gia tốc	$\pm 2 \text{ g}$ đến $\pm 16 \text{ g}$
Dải đo vận tốc góc	$\pm 125^\circ/\text{s}$ đến $\pm 2000^\circ/\text{s}$
Dữ liệu đầu ra	Gia tốc, vận tốc góc, góc định hướng

Bảng 2.4 trình bày các thông số kỹ thuật của cảm biến IMU 9 bậc tự do, tích hợp gia tốc kế, con quay hồi chuyển và từ kế ba trục. Cảm biến hoạt động với

điện áp từ 2,4 V đến 3,6 V và hỗ trợ các giao tiếp I2C, SPI và UART. Thiết bị có dải đo gia tốc từ ± 2 g đến ± 16 g và dải đo vận tốc góc từ $\pm 125^\circ/\text{s}$ đến $\pm 2000^\circ/\text{s}$, cho phép thu thập dữ liệu chuyển động và góc định hướng phục vụ quá trình định vị và điều hướng của robot tự hành.

2.8.5 Mạch thu sóng RF và tay cầm RF

Hình 2.13 là mạch thu và tay cầm RF. Thiết bị này được sử dụng để điều khiển robot từ xa trong quá trình vận hành, kiểm tra và hiệu chỉnh hệ thống. Bộ thu có nhiệm vụ nhận tín hiệu không dây từ tay cầm và truyền dữ liệu nhận được đến vi điều khiển thông qua giao thức SPI. Khi người dùng thực hiện các thao tác trên tay cầm như nhấn nút hoặc điều khiển các cần joystick, thông tin điều khiển sẽ được mã hóa và truyền không dây đến bộ thu. Sau khi nhận được tín hiệu, bộ thu sẽ giải mã dữ liệu và gửi trạng thái của các nút nhấn, cần điều khiển và các lệnh chức năng đến vi điều khiển.



a. Mạch thu RF



b. Tay cầm RF

Hình 2.13 Mạch thu và tay cầm RF

Bảng 2.5 Mô tả nhóm chân của mạch thu RF

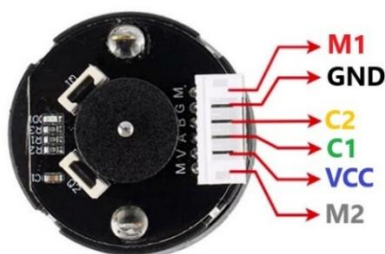
Chân	Chức năng
GND	Chân nối mass của hệ thống
3V3	Chân cấp nguồn 3V3
SS	Chân chọn thiết bị (Slave Select)
CLK	Chân xung đồng hồ đồng bộ dữ liệu
MISO	Chân truyền dữ liệu từ tay cầm về vi điều khiển
MOSI	Chân truyền dữ liệu từ vi điều khiển đến tay cầm

Bảng 2.5 mô tả các nhóm chân và chức năng các chân của mạch thu RF. Module này sử dụng nguồn 3V3, đã có sẵn anten thu, nhiệm vụ của nó là thu

sóng và chuyển đổi sang tín hiệu số, sau đó giao tiếp với vi điều khiển qua giao thức SPI. Các chân SPI thông dụng như MISO, MOSI, các chân xung như CLK, chân select để chọn thiết bị.

2.8.6 Bộ mã hóa

Hình 2.14 là ngoại quan và sơ đồ chân bộ mã hóa. Mạch hoạt động dựa trên nguyên lý tạo xung tương ứng với sự quay của trục động cơ. Khi động cơ quay, encoder sinh ra hai tín hiệu xung tại kênh A và B lệch pha nhau, từ đó cho phép xác định số vòng quay, tốc độ và chiều quay của động cơ. Cụm động cơ encoder gồm hai chân M+ và M- dùng để cấp nguồn điều khiển động cơ, chân VCC và GND dùng để cấp nguồn cho mạch encoder, cùng hai chân A và B nối với GPIO của vi điều khiển dùng để xuất tín hiệu xung về bộ điều khiển. Bộ mã hóa được gắn phía sau của động cơ.



Hình 2.14 Sơ đồ chân bộ mã hóa

Bảng 2.6 Thông số kỹ thuật bộ mã hóa

Thông số	Giá trị
Loại encoder	Encoder Hall từ tính gia tăng hai pha AB
Độ phân giải xung	$11 \text{ PPR} \times \text{tỷ số truyền hộp số}$
Điện áp hoạt động	DC 5V/3.3V
Mạch tích hợp	Tích hợp điện trở kéo lên, có thể kết nối trực tiếp với vi điều khiển
Kiểu giao tiếp	Đầu nối PH2.0
Loại tín hiệu đầu ra	Xung vuông hai pha AB
Tần số đáp ứng	100 kHz
Số xung cơ bản	11 PPR
Số cực từ của vòng từ	22 cực (11 cặp cực)

Bảng 2.6 trình bày các thông số kỹ thuật của bộ mã hóa từ tính gia tăng hai pha AB được sử dụng để đo tốc độ và quãng đường di chuyển của động cơ. Encoder tạo tín hiệu xung vuông hai pha AB với độ phân giải phụ thuộc vào số xung cơ bản 11 PPR và tỷ số truyền của hộp số. Thiết bị hoạt động ở mức điện áp 3,3 V hoặc 5 V, tích hợp sẵn điện trở kéo lên giúp kết nối trực tiếp với vi điều khiển, đồng thời hỗ trợ tần số đáp ứng lên đến 100 kHz, đáp ứng yêu cầu thu thập dữ liệu chuyển động của robot theo thời gian thực.

2.8.7 IC chuyển đổi USB UART

Hình 2.15 là IC CP2102, là IC chuyển đổi giao tiếp USB sang UART do Silicon Labs phát triển. IC cho phép các thiết bị sử dụng giao tiếp nối tiếp UART có thể kết nối với máy tính hoặc các bộ xử lý thông qua cổng USB. CP2102 hỗ trợ tốc độ truyền dữ liệu cao và được sử dụng rộng rãi trong các hệ thống nhúng để nạp chương trình, giám sát dữ liệu và truyền nhận thông tin giữa vi điều khiển với máy tính.



Hình 2.15 IC chuyển đổi USB UART CP2102

Bảng 2.7 Thông số kỹ thuật IC CP2102

Thông số	Giá trị
Chức năng	USB to UART Bridge
Chuẩn USB	USB 2.0
Điện áp hoạt động	3V ~ 3.6V
Tốc độ UART tối đa	12 Mbps
Tín hiệu hỗ trợ	TXD, RXD, RTS, CTS, DTR
Bộ dao động	Tích hợp
Nhiệt độ hoạt động	-40°C đến +85°C

Bảng 2.7 trình bày các thông số kỹ thuật của chip chuyển đổi USB sang UART, được sử dụng để truyền dữ liệu giữa máy tính và vi điều khiển. Thiết bị hỗ trợ chuẩn USB 2.0, tốc độ truyền UART tối đa lên đến 12 Mbps và tích hợp

sẵn bộ dao động bên trong giúp đơn giản hóa thiết kế phần cứng. Ngoài các tín hiệu truyền nhận dữ liệu TXD và RXD, chip còn hỗ trợ các tín hiệu điều khiển như RTS, CTS và DTR, đáp ứng yêu cầu giao tiếp nối tiếp ổn định trong hệ thống nhúng

2.8.8 IC mở rộng chân PCF8575

Hình 2.16 là IC PCF8575. Đây là mạch mở rộng GPIO 16 bit sử dụng giao tiếp I2C, cho phép tăng số lượng chân GPIO cho vi điều khiển. PCF8575 hoạt động bằng cách giao tiếp với bộ xử lý thông qua chuẩn I2C để mở rộng thêm 16 chân I/O số. Bộ xử lý gửi hoặc nhận dữ liệu qua hai đường SDA và SCL, từ đó điều khiển trạng thái mức logic hoặc đọc trạng thái tín hiệu tại các chân I/O của PCF8575. Nhờ sử dụng giao tiếp I2C, nhiều chân GPIO có thể được mở rộng mà chỉ cần sử dụng hai chân giao tiếp trên bộ xử lý.



Hình 2.16 IC mở rộng chân PCF8575

Bảng 2.8 Thông số kỹ thuật IC PCF8575

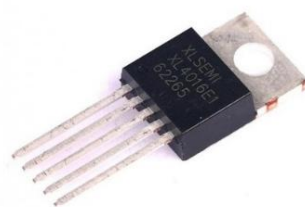
Thông số	Giá trị
Điện áp hoạt động	2,5 V – 5,5 V
Giao tiếp	I2C
Tốc độ truyền dữ liệu tối đa	400 kHz
Số lượng chân I/O	16
Số lượng địa chỉ khả dụng	8
Nhiệt độ hoạt động	-40 °C đến +85 °C

Bảng 2.8 trình bày các thông số kỹ thuật của IC mở rộng chân I/O PCF8575. IC hoạt động với điện áp từ 2,5 V đến 5,5 V và giao tiếp với vi điều khiển thông qua chuẩn I2C với tốc độ truyền dữ liệu tối đa 400 kHz. PCF8575 cung cấp 16 chân I/O số và hỗ trợ 8 địa chỉ thiết bị khác nhau trên cùng một bus I2C, giúp mở

rộng số lượng chân điều khiển khi hệ thống có nhiều thiết bị ngoại vi. Thiết bị có dải nhiệt độ hoạt động từ -40°C đến $+85^{\circ}\text{C}$, phù hợp cho các ứng dụng nhúng và tự động hóa.

2.8.9 IC hạ áp XL4016

Hình 2.17 là IC XL4016. Đây là bộ chuyển đổi nguồn hạ áp một chiều sang một chiều (DC-DC Buck Converter) được sử dụng để tạo ra điện áp đầu ra thấp hơn từ nguồn điện áp đầu vào. IC hoạt động theo nguyên lý điều chế độ rộng xung (PWM) với tần số chuyển mạch cố định 180 kHz, cho phép đạt hiệu suất chuyển đổi cao và giảm tổn hao công suất trong quá trình hoạt động. XL4016 hỗ trợ điện áp đầu vào rộng từ 8 V đến 40 V, điện áp đầu ra có thể điều chỉnh trong khoảng từ 1,25 V đến 36 V và khả năng cung cấp dòng tải lên đến 8A.



Hình 2.17 IC hạ áp XL4016

Bảng 2.9 Thông số kỹ thuật XL4016

Thông số	Giá trị
Tên linh kiện	XL4016
Chức năng	Bộ chuyển đổi nguồn hạ áp DC-DC (Buck Converter)
Điện áp đầu vào	8 V – 40 V
Điện áp đầu ra	1,25 V – 36 V (điều chỉnh được)
Dòng tải đầu ra tối đa	8 A
Tần số chuyển mạch	180 kHz
Hiệu suất chuyển đổi tối đa	96 %
Điện áp rơi tối thiểu (Dropout)	0,3 V
Chu kỳ công tác tối đa	100 %
Kiểu đóng gói	TO220-5L

Bảng 2.9 trình bày các thông số kỹ thuật của IC XL4016, một bộ chuyển đổi nguồn hạ áp DC-DC được sử dụng để cung cấp điện áp ổn định cho các khối trong hệ thống. IC hỗ trợ điện áp đầu vào từ 8 V đến 40 V và cho phép điều chỉnh điện áp đầu ra trong khoảng 1,25 V đến 36 V với dòng tải tối đa 8 A. XL4016 hoạt động ở tần số chuyển mạch 180 kHz, đạt hiệu suất chuyển đổi tối đa khoảng 96 %, giúp giảm tổn hao năng lượng trong quá trình hoạt động

2.8.10 IC ổn áp AMS1117-3.3

Hình 2.18 là IC AMS1117. Đây là bộ ổn áp tuyến tính có khả năng cung cấp dòng ra tối đa khoảng 1A. AMS1117 có điện áp sụt thấp điển hình khoảng 1.1V - 1.3V tại dòng tải 1A, giúp duy trì ổn áp khi chênh lệch giữa đầu vào và đầu ra nhỏ. IC được tích hợp sẵn các cơ chế bảo vệ như giới hạn dòng và bảo vệ quá nhiệt, đảm bảo an toàn trong quá trình hoạt động.



Hình 2.18 IC ổn áp AMS 1117-3.3

Bảng 2.10 trình bày các thông số kỹ thuật của IC AMS1117, một bộ ổn áp tuyến tính được sử dụng để tạo nguồn 3,3 V ổn định cho các mạch điện tử trong hệ thống. IC hỗ trợ điện áp đầu vào từ 4,5 V đến 15 V và cung cấp điện áp đầu ra cố định 3,3 V với dòng tải tối đa 1 A. AMS1117 có điện áp rơi khoảng 1,1 V và dòng tiêu thụ nội bộ khoảng 5 mA.

Bảng 2.10 Thông số kỹ thuật AMS1117-3.3

Thông số	Giá trị
Điện áp đầu vào	4,5 V – 15 V
Điện áp đầu ra	3,3 V
Dòng tải tối đa	1 A
Điện áp rơi	1,1 V
Dòng tiêu thụ nội bộ	5 mA

2.8.11 Relay

Hình 2.19 là Relay 12 V 5 chân 10 A. Đây là một thiết bị đóng cắt điện cơ được sử dụng để điều khiển các tải công suất lớn bằng tín hiệu điều khiển có công suất nhỏ. Relay gồm một cuộn dây điện từ hoạt động ở điện áp 12 V và hệ thống tiếp điểm cơ khí có khả năng chịu dòng tải tối đa 10 A. Cấu trúc 5 chân bao gồm hai chân cuộn dây, một chân chung (COM), một chân thường đóng (NC) và một chân thường mở (NO). Khi chưa cấp điện cho cuộn dây, chân COM được nối với chân NC. Khi cấp điện 12 V vào cuộn dây, từ trường sinh ra sẽ hút tiếp điểm chuyển sang nối COM với chân NO. Nhờ khả năng cách ly giữa mạch điều khiển và mạch công suất, relay được sử dụng rộng rãi trong các ứng dụng đóng cắt nguồn, bảo vệ hệ thống và điều khiển các thiết bị điện có dòng tiêu thụ lớn.



Hình 2.19 Relay

Bảng 2.11 trình bày các thông số kỹ thuật của rơ-le 12 VDC được sử dụng để đóng cắt và chuyển mạch nguồn trong hệ thống. Rơ-le hoạt động với điện áp cuộn dây 12 VDC, gồm 5 chân kết nối và hỗ trợ các tiếp điểm COM, NO và NC. Thiết bị có khả năng đóng cắt tải với dòng điện tối đa 10 A và điện áp tải lên đến 250 VAC hoặc 30 VDC, đáp ứng yêu cầu điều khiển các công suất lớn đồng thời đảm bảo cách ly giữa mạch điều khiển và mạch công suất.

Bảng 2.11 Bảng thông số kỹ thuật Relay

Thông số	Giá trị
Điện áp cuộn dây	12 VDC
Số chân	5 chân
Dòng tải tối đa	10 A
Loại tiếp điểm	COM, NO, NC
Điện áp tải tối đa	250 VAC / 30 VDC
Chức năng	Đóng cắt và chuyển mạch nguồn tự động

2.8.12 Raspberry pi 5

Hình 2.20 là máy tính nhúng Raspberry Pi 5. Đây là máy tính nhúng một bo mạch được phát triển bởi Raspberry Pi Foundation, được trang bị bộ vi xử lý ARM Cortex-A76 bốn nhân với hiệu năng xử lý cao hơn đáng kể so với các thế hệ trước. Thiết bị hỗ trợ nhiều giao diện kết nối như GPIO, USB, UART, I2C, SPI, Ethernet và camera, đáp ứng nhu cầu phát triển các hệ thống nhúng và robot tự hành. Với khả năng chạy hệ điều hành Linux và hỗ trợ các thư viện xử lý AI, thị giác máy tính và ROS 2, Raspberry Pi 5 phù hợp cho các ứng dụng điều khiển và xử lý dữ liệu phức tạp.



Hình 2.20 Board mạch raspberry pi 5

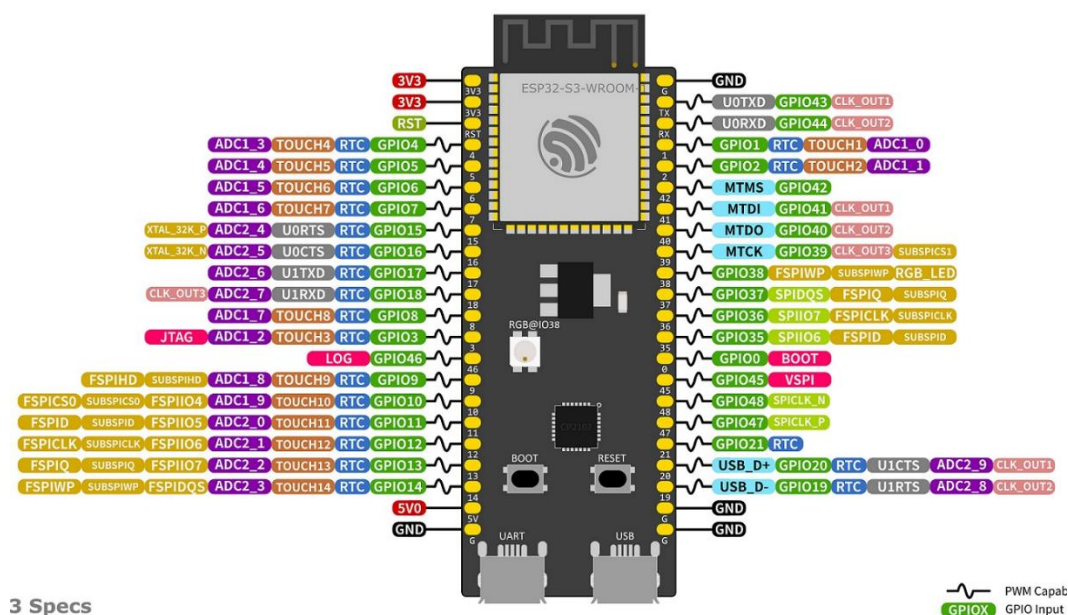
Bảng 2.12 Thông số kỹ thuật raspberry pi 5

Thông số	Giá trị
Model	Raspberry Pi 5
Bộ vi xử lý	Broadcom BCM2712, Quad-core ARM Cortex-A76 64-bit
Xung nhịp CPU	2,4 GHz
Bộ nhớ RAM	8 GB
Hệ điều hành hỗ trợ	Raspberry Pi OS, Ubuntu
Kết nối không dây	Wi-Fi 802.11ac, Bluetooth 5.0/BLE
Cổng mạng	Gigabit Ethernet
Cổng USB	2 × USB 3.0, 2 × USB 2.0
Giao tiếp ngoại vi	GPIO, UART, I2C, SPI, PWM
Điện áp cấp	5 VDC (USB-C)

Bảng 2.12 trình bày các thông số kỹ thuật của Raspberry Pi 5. Thiết bị được trang bị vi xử lý Broadcom BCM2712 gồm 4 nhân ARM Cortex-A76 64-bit hoạt động ở xung nhịp 2,4 GHz cùng bộ nhớ RAM 8 GB, đáp ứng yêu cầu xử lý các thuật toán ROS 2, SLAM và điều hướng. Raspberry Pi 5 hỗ trợ nhiều chuẩn giao tiếp như GPIO, UART, I2C và SPI, đồng thời tích hợp kết nối Wi-Fi, Bluetooth và Gigabit Ethernet, giúp thuận tiện trong việc kết nối các cảm biến, bộ điều khiển và thiết bị mạng. Thiết bị hoạt động với nguồn cấp 5 VDC thông qua cổng USB-C.

2.8.13 Esp32-s3

Hình 2.21 là ngoại quan và sơ đồ chân ESP32-S3. Đây là vi điều khiển hiệu năng cao được phát triển bởi Espressif, tích hợp bộ xử lý lõi kép Xtensa LX7 cùng các giao tiếp ngoại vi như UART, I2C, SPI, PWM và GPIO. Trong hệ thống robot tự hành, ESP32-S3 được sử dụng để thực hiện các tác vụ điều khiển thời gian thực như đọc dữ liệu encoder, thu thập dữ liệu từ các cảm biến ngoại vi và điều khiển động cơ. Vi điều khiển cũng đảm nhận vai trò giao tiếp với Raspberry Pi 5 để nhận lệnh vận tốc và thực thi các lệnh điều khiển chuyển động của robot.



Hình 2.21 Sơ đồ chân module Esp32-S3

Bảng 2.13 trình bày các thông số kỹ thuật của vi điều khiển ESP32-S3 được sử dụng trong hệ thống để giao tiếp và điều khiển các thiết bị ngoại vi. ESP32-S3 được trang bị bộ vi xử lý Xtensa® LX7 lõi kép 32-bit với xung nhịp tối đa 240 MHz, bộ nhớ SRAM 512 KB và Flash 8 MB, đáp ứng tốt các tác vụ xử lý dữ liệu thời gian thực. Ngoài khả năng kết nối Wi-Fi 2,4 GHz và Bluetooth 5 (LE), vi

điều khiển còn hỗ trợ nhiều chuẩn giao tiếp như UART, I2C, SPI, PWM, ADC và USB, cùng tối đa 45 chân GPIO, tạo sự linh hoạt trong việc kết nối cảm biến và các mô-đun điều khiển của robot.

Bảng 2.13 Thông số kỹ thuật ESP32-S3

Thông số	Giá trị
Model	ESP32-S3
Bộ vi xử lý	Xtensa® LX7 lõi kép 32-bit
Xung nhịp CPU	Lên đến 240 MHz
Bộ nhớ SRAM	512 KB
Bộ nhớ Flash	8 MB
Điện áp hoạt động	3V3
Wi-Fi	2,4 GHz IEEE 802.11 b/g/n
Bluetooth	Bluetooth 5 (LE)
GPIO	Tối đa 45 chân
Giao tiếp ngoại vi	UART, I2C, SPI, PWM, ADC, USB

CHƯƠNG 3

THIẾT KẾ HỆ THỐNG

3.1 YÊU CẦU CỦA HỆ THỐNG

Trong bối cảnh robot tự hành ngày càng được ứng dụng rộng rãi trong các môi trường trong nhà như văn phòng, nhà xưởng và hành lang tòa nhà, việc xây dựng hệ thống robot có khả năng lập bản đồ và điều hướng tự động trở thành một yêu cầu quan trọng. Trong đề án này, hệ thống robot tự hành 4 bánh được thiết kế nhằm thực hiện bài toán lập bản đồ, định vị và điều hướng trong môi trường trong nhà với các yêu cầu về khả năng di chuyển ổn định, thu thập dữ liệu không gian, xác định vị trí, lập kế hoạch đường đi và đảm bảo an toàn khi vận hành. Robot sử dụng cảm biến LiDAR kết hợp với thuật toán SLAM để xây dựng bản đồ lưới 2D, sau đó lưu trữ và tái sử dụng bản đồ cho các lần hoạt động tiếp theo. Khi vận hành trên bản đồ có sẵn, hệ thống sử dụng thuật toán AMCL kết hợp dữ liệu LiDAR và odometry để định vị, làm cơ sở cho quá trình lập kế hoạch và điều hướng tự động.

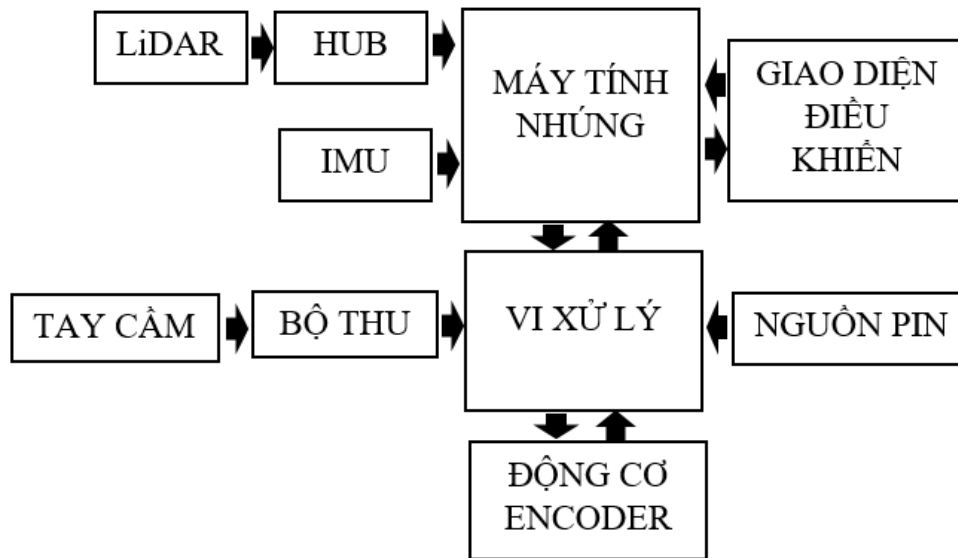
Dựa trên các yêu cầu thiết kế, hệ thống được triển khai dưới dạng mô hình tích hợp giữa phần cứng và phần mềm. Robot hỗ trợ hai chế độ hoạt động gồm điều khiển thủ công bằng tay cầm RF và điều hướng tự động. Trong giai đoạn xây dựng bản đồ, người dùng có thể điều khiển robot bằng tay cầm để thu thập dữ liệu môi trường. Dữ liệu từ cảm biến LiDAR được xử lý thông qua thuật toán SLAM nhằm tạo và lưu bản đồ 2D. Trong quá trình di chuyển tự động, hệ thống tiếp nhận vị trí mục tiêu từ người dùng, sử dụng thuật toán Weight A* để lập kế hoạch quỹ đạo tối ưu, kết hợp với cơ chế điều hướng của ROS 2 nhằm giúp robot bám theo đường đi và tránh các vật cản tĩnh trên bản đồ.

3.2 MÔ HÌNH HỆ THỐNG

Hệ thống robot tự hành trong đề tài được tổ chức theo kiến trúc gồm hai khối chính là máy tính nhúng và vi xử lý. Máy tính nhúng đảm nhiệm các chức năng xử lý cấp cao như thu nhận dữ liệu từ cảm biến LiDAR và IMU, xây dựng bản đồ, định vị và điều hướng robot, đồng thời cung cấp giao diện điều khiển cho người dùng. Vi xử lý đảm nhiệm các tác vụ điều khiển cấp thấp, bao gồm tiếp nhận tín hiệu từ bộ thu kết nối với tay cầm điều khiển, điều khiển khối động cơ encoder và trao đổi dữ liệu với máy tính nhúng trong quá trình vận hành. Toàn bộ

hệ thống được cấp nguồn từ khối nguồn pin để bảo đảm robot hoạt động ổn định và liên tục.

3.2.1 Sơ đồ khối tổng quát hệ thống



Hình 3.1 Sơ đồ khối tổng quát hệ thống

Hình 3.1 trình bày sơ đồ khối tổng quát của hệ thống robot tự hành. Trong đó, máy tính nhúng là khối xử lý trung tâm, đảm nhiệm các chức năng xử lý cấp cao của hệ thống như thu nhận dữ liệu cảm biến, xây dựng bản đồ, định vị và điều hướng robot. Dữ liệu từ LiDAR được truyền đến máy tính nhúng thông qua HUB, trong khi dữ liệu từ IMU được truyền trực tiếp đến máy tính nhúng để phục vụ quá trình nhận thức môi trường và ước lượng trạng thái của robot. Đồng thời, máy tính nhúng giao tiếp hai chiều với vi xử lý nhằm truyền các lệnh điều khiển và nhận dữ liệu phản hồi trong quá trình vận hành. Vi xử lý đảm nhiệm các tác vụ điều khiển cấp thấp, bao gồm tiếp nhận tín hiệu từ bộ thu kết nối với tay cầm điều khiển, xử lý các lệnh điều khiển và giao tiếp với khối động cơ encoder để điều khiển chuyển động cũng như thu nhận dữ liệu phản hồi từ các bánh xe của robot. Ngoài ra, hệ thống còn được trang bị giao diện điều khiển, cho phép người dùng theo dõi trạng thái và tương tác với robot trong quá trình hoạt động. Toàn bộ hệ thống được cấp nguồn từ khối nguồn pin để bảo đảm các khối chức năng hoạt động ổn định và liên tục.

3.2.2 Nguyên lý hoạt động tổng quát của hệ thống

Hệ thống robot tự hành hoạt động theo chu trình khép kín gồm các giai đoạn thu nhận dữ liệu cảm biến, xử lý thông tin, lập kế hoạch và điều khiển chuyển động. Trong quá trình vận hành, cảm biến LiDAR liên tục thu thập dữ liệu về môi trường xung quanh và truyền đến máy tính nhúng thông qua HUB, trong khi cảm biến IMU cung cấp trực tiếp dữ liệu về trạng thái chuyển động của robot. Dựa trên các dữ liệu thu nhận được, máy tính nhúng thực hiện quá trình nhận thức môi trường và ước lượng trạng thái của robot, từ đó xác định vị trí hiện tại và trạng thái hoạt động của hệ thống. Hệ thống có thể hoạt động ở hai chế độ chính là xây dựng bản đồ và điều hướng tự động. Trong chế độ xây dựng bản đồ, robot di chuyển trong môi trường để thu thập dữ liệu LiDAR và đồng thời thực hiện thuật toán SLAM nhằm tạo ra bản đồ của không gian làm việc. Trong chế độ điều hướng tự động, máy tính nhúng sử dụng bản đồ đã xây dựng để xác định vị trí hiện tại của robot và tính toán quỹ đạo di chuyển đến vị trí mục tiêu.

Sau khi quỹ đạo di chuyển được xác định, máy tính nhúng tạo ra các lệnh điều khiển và truyền đến vi xử lý để thực hiện các tác vụ điều khiển cấp thấp của robot. Vi xử lý tiếp nhận các lệnh điều khiển, tạo tín hiệu điều khiển cho khối động cơ encoder và điều khiển robot di chuyển theo quỹ đạo đã được lập kế hoạch. Trong quá trình robot di chuyển, khối động cơ encoder liên tục gửi dữ liệu phản hồi về tốc độ và quãng đường di chuyển của các bánh xe để cập nhật trạng thái chuyển động của robot. Dữ liệu phản hồi này được trao đổi giữa vi xử lý và máy tính nhúng nhằm hiệu chỉnh sai số và bảo đảm độ ổn định của hệ thống trong suốt quá trình vận hành. Bên cạnh chế độ tự động, hệ thống còn hỗ trợ điều khiển thủ công thông qua tay cầm và bộ thu tín hiệu, cho phép người vận hành trực tiếp kiểm tra và điều khiển robot trong các giai đoạn thử nghiệm. Toàn bộ các khối chức năng của hệ thống được cấp năng lượng từ nguồn pin để bảo đảm robot có thể hoạt động ổn định và liên tục trong nhiều điều kiện vận hành khác nhau.

3.3 THIẾT KẾ PHẦN CỨNG

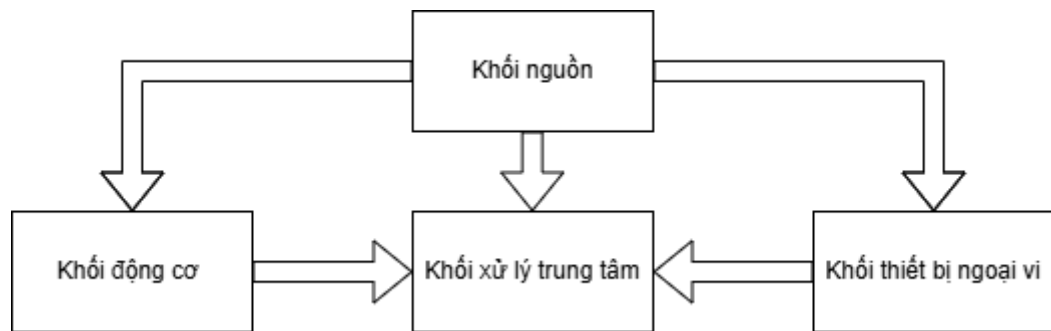
3.3.1 Chức năng của phần cứng

Hệ thống phần cứng của robot được thiết kế nhằm đáp ứng các yêu cầu về thu thập dữ liệu môi trường, xử lý thông tin, điều khiển chuyển động và cung cấp năng lượng cho toàn bộ hệ thống. Trong đó, các cảm biến có nhiệm vụ thu thập thông tin về môi trường xung quanh để phát hiện vật cản, xây dựng bản đồ và hỗ trợ quá trình định vị, đồng thời ghi nhận trạng thái chuyển động của robot nhằm phục vụ việc điều khiển và hiệu chỉnh quỹ đạo. Dữ liệu thu thập được truyền về khối xử lý trung tâm để thực hiện các thuật toán xử lý, phân tích thông tin và tạo tín hiệu điều khiển giúp robot hoạt động theo yêu cầu.

Bên cạnh khả năng xử lý dữ liệu, hệ thống phần cứng còn đảm nhiệm chức năng điều khiển chuyển động thông qua việc điều khiển các động cơ dẫn động, cho phép robot thực hiện các thao tác di chuyển như tiến, lùi, quay trái, quay phải và bám theo quỹ đạo đã lập kế hoạch. Đồng thời, hệ thống nguồn được thiết kế nhằm đảm bảo cung cấp điện áp và công suất ổn định cho các thiết bị phần cứng trong suốt quá trình vận hành. Từ các yêu cầu trên, hệ thống phần cứng được xây dựng gồm các khối chức năng chính bao gồm khối nguồn, khối xử lý trung tâm, khối cảm biến và khối truyền động, tạo thành nền tảng cho quá trình hoạt động tự hành của robot.

3.3.2 Sơ đồ khối phần cứng

Ở sơ đồ khối Hình 3.2, hệ thống robot tự hành được xây dựng từ các khối phần cứng chính gồm khối nguồn, khối xử lý trung tâm, khối cảm biến và khối động cơ. Khối nguồn đóng vai trò cung cấp năng lượng cho toàn bộ hệ thống, đồng thời đảm bảo nguồn điện đầu ra ổn định để các linh kiện hoạt động an toàn và tin cậy. Bên cạnh đó, khối cảm biến có nhiệm vụ thu thập thông tin về môi trường xung quanh và trạng thái hoạt động của robot, cung cấp dữ liệu đầu vào cho các quá trình xử lý và điều khiển.

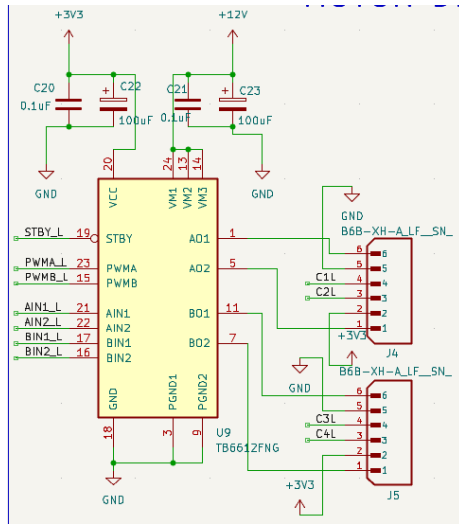


Hình 3.2 Sơ đồ khối hệ thống phần cứng

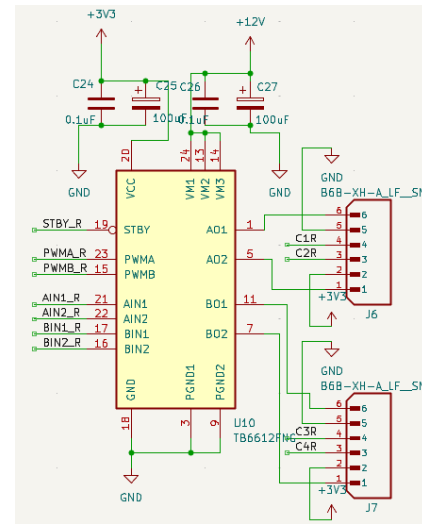
Khối xử lý trung tâm là bộ phận điều khiển chính của hệ thống, có nhiệm vụ tiếp nhận dữ liệu từ các cảm biến, thực hiện các thuật toán xử lý, định vị, xây dựng bản đồ và lập kế hoạch đường đi, sau đó tạo tín hiệu điều khiển gửi đến khối động cơ. Khối động cơ đóng vai trò cơ cấu chấp hành, giúp robot thực hiện các chuyển động như tiến, lùi và thay đổi hướng di chuyển theo các lệnh điều khiển nhận được. Sự phối hợp giữa các khối phần cứng cho phép robot thực hiện một chu trình hoạt động hoàn chỉnh từ thu thập dữ liệu môi trường, xử lý thông tin, đưa ra quyết định điều hướng đến điều khiển chuyển động để di chuyển tự động trong khu vực làm việc.

3.3.3 Khối động cơ

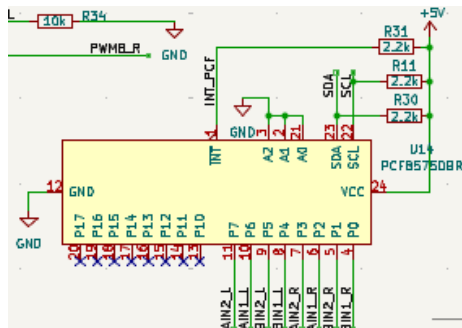
Trong hệ thống robot tự hành, sử dụng cơ cấu xe 4 bánh với 4 động cơ khác nhau. Dựa vào thông số dòng tải tối đa của động cơ là 1.2A, nhóm lựa chọn linh kiện để điều khiển động cơ là IC TB6612FNG với dòng tải tối đa mỗi kênh điều khiển là 1.2A. Mỗi IC điều khiển được tối đa 2 động cơ cùng lúc, cần 2 IC để điều khiển 4 động cơ, IC sẽ nhận lệnh và xung PWM từ vi điều khiển để vận hành động cơ.



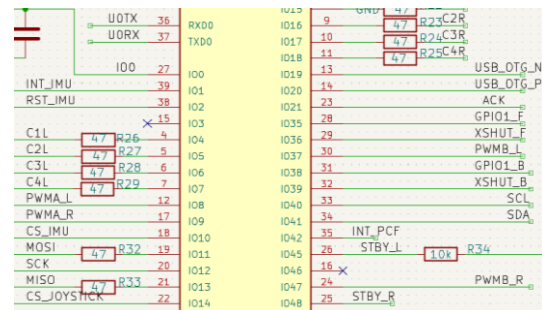
a. Kết nối TB6612FNG bên trái



b. Kết nối TB6612FNG bên phải



c. Kết nối TB6612 với PCF8575



d. Kết nối ESP với TB6612FNG

Hình 3.3 Sơ đồ nguyên lý kết nối TB6612FNG với ESP32-S3 và PCF8575

Tại bảng 3.1 và 3.2 là sơ đồ kết nối chân giữa 2 IC điều khiển động cơ TB6612FNG. Một IC dùng để điều khiển 2 động cơ bên trái robot và IC còn lại điều khiển 2 động cơ bên phải robot. IC sử dụng nguồn 3V3, các chân dữ liệu điều khiển chiều động cơ được kết nối với ngõ ra của IC PCF8575. IC mở rộng chân sẽ làm trung gian nhận dữ liệu từ ESP32 qua I2C, sau đó xuất tín hiệu điều khiển tới TB6612FNG. Các ngõ ra AO01, AO02, BO01, BO02 sẽ xuất nguồn cho động cơ hoạt động. Chân VM sẽ được nối tới nguồn 12V, IC điều chỉnh

nguồn này cấp cho động cơ. Chân VCC nối với nguồn 3V3 cấp cho IC hoạt động.

Bảng 3.1 Sơ đồ kết nối chân TB6612FNG trái với ESP32-S3 và PCF8575

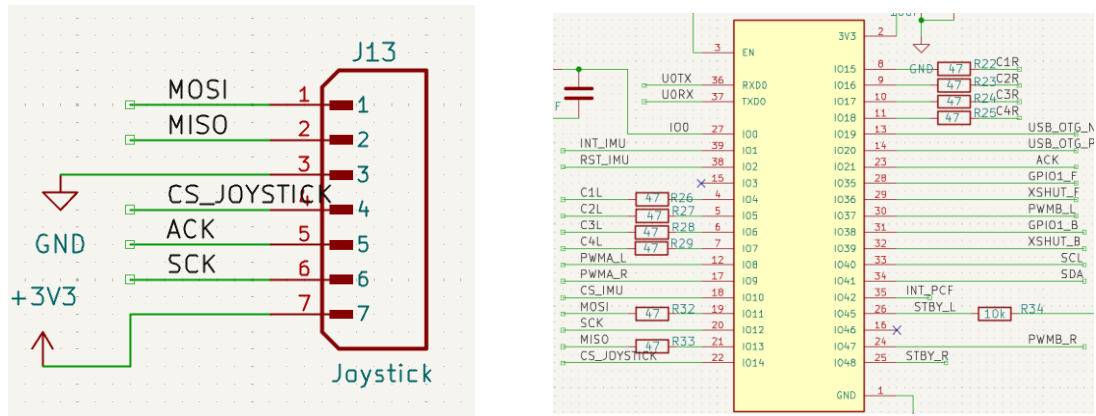
TB6612FNG trái	PCF8575	ESP32-S3	Động cơ
STBY	x	IO45	x
PWMA	x	IO8	x
PWMB	x	IO37	x
AIN1, AIN2	P6, P7	x	x
BIN1, BIN2	P4, P5	x	x
AO01	x	x	Motor1 -
AO02	x	x	Motor1 +
BO01	x	x	Motor2 -
BO02	x	x	Motor2 +
VCC	3V3	3V3	3V3
GND	GND	GND	GND

Bảng 3.2 Sơ đồ kết nối chân TB6612FNG phải với ESP32-S3 và PCF8575

TB6612FNG phải	PCF8575	ESP32-S3	Động cơ
STBY	x	IO48	x
PWMA	x	IO9	x
PWMB	x	IO47	x
AIN1, AIN2	P2, P3	x	x
BIN1, BIN2	P0, P1	x	x
AO01	x	x	Motor3 -
AO02	x	x	Motor3 +
BO01	x	x	Motor4 -
BO02	x	x	Motor4 +
VCC	3V3	3V3	3V3
GND	GND	GND	GND

3.3.4 Khối thiết bị ngoại vi

Thiết bị ngoại vi thứ nhất là mạch thu tín hiệu RF, sơ đồ nguyên lý kết nối mạch thu RF xem trong hình 3.4. Mạch thu được sử dụng để nhận tín hiệu từ tay cầm. Mạch chuyển đổi mức logic tín hiệu sau đó truyền dữ liệu tới ESP32-S3 thông qua giao thức SPI. Mục đích để sử dụng điều khiển robot từ xa trong quá trình vẽ map ban đầu.



a. Sơ đồ chân mạch thu RF

b. Kết nối ESP32-S3 với mạch thu RF

Hình 3.4 Sơ đồ nguyên lý mạch thu RF với ESP32-S3

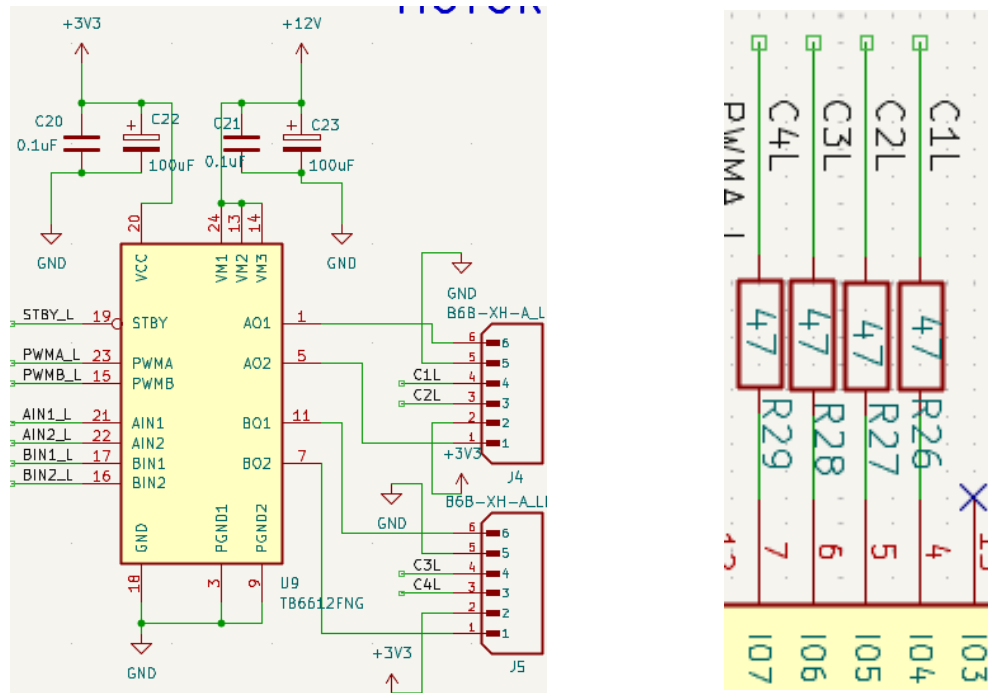
Bảng 3.3 Sơ đồ kết nối chân mạch thu RF và ESP32-S3

Mạch thu RF	ESP32-S3
MOSI	IO11
MISO	IO13
CS	IO14
ACK	IO21
SCK	IO12
VCC	3V3
GND	GND

Bảng 3.3 trình bày sơ đồ kết nối chân giữa mạch thu RF và ESP32-S3, mạch sử dụng nguồn 3V3, quá trình truyền nhận dữ liệu giữa hai thiết bị được thực hiện thông qua giao thức SPI với các chân MOSI, MISO, SCK và CS được kết nối trực tiếp đến các chân GPIO tương ứng của ESP32-S3. Ngoài ra, chân ACK được kết nối với chân IO21 để nhận tín hiệu xác nhận từ mạch thu trong quá

trình trao đổi dữ liệu. Cấu hình kết nối này cho phép ESP32-S3 đọc trạng thái các nút nhấn và cần điều khiển từ tay cầm RF.

Thiết bị ngoại vi thứ hai là bộ mã hóa, sơ đồ nguyên lý kết nối giữa bộ mã hóa với esp32-s3 xem tại hình 3.5. Bộ mã hóa được gắn liền với động cơ JGB37-520 để đo chiều và số xung nhịp khi xoay của động cơ. Mục đích nhằm kiểm soát tốc độ, số vòng xoay của động cơ, dữ liệu từ bộ mã hóa rất quan trọng cho quá trình tính toán odometry.



a. Kết nối giữa bộ mã hóa và TB6612FNG

b. Kết nối giữa ESP32-S3 với bộ mã hóa

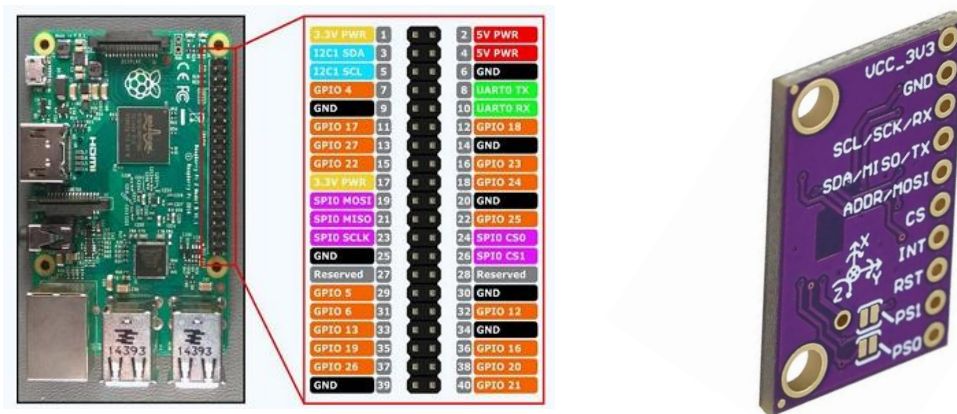
Hình 3.5 Sơ đồ kết nguyên lý nối giữa bộ mã hóa với ESP32-S3 và TB6612FNG

Bảng 3.4 mô tả sơ đồ kết nối chân giữa 4 bộ mã hóa của 4 động cơ với 2 IC điều khiển TB6612FNG và ESP32-S3. Có 4 bộ mã hóa lần lượt là E1 trái, E2 trái, E3 phải, E4 phải tương ứng với vị trí phía trước, phía sau bên trái robot và phía trước, phía sau bên phải robot. IC TB6612FNG kết nối với E1 và E2 là IC điều khiển phần động cơ bên trái, còn IC kết nối với E3 và E4 điều khiển phần động cơ bên phải. Các chân C1 và C2 của encoder gửi dữ liệu xung encoder của 2 pha A, B về vi điều khiển, được nối trực tiếp với IO của ESP32-S3 thông qua 1 điện trở 47 Ω để bảo vệ chân IO của ESP32-S3 và giảm nhiễu.

Bảng 3.4 Sơ đồ kết nối 4 bộ mã hóa với ESP32-S3 và TB6612fNG

Thứ tự bộ mã hóa	Bộ mã hóa	ESP32-S3	Tb6612FNG
E1 trái	M+	x	AO01
	GND	GND	GND
	C1	IO4	x
	C2	IO5	x
	VCC	3V3	3V3
	M-	x	AO02
E2 trái	M+	x	BO01
	GND	GND	GND
	C1	IO6	x
	C2	IO7	x
	VCC	3V3	3V3
	M-	x	BO02
E3 phải	M+	x	AO01
	GND	GND	GND
	C1	IO15	x
	C2	IO16	x
	VCC	3V3	3V3
	M-	x	AO02
E4 phải	M+	x	BO01
	GND	GND	GND
	C1	IO17	x
	C2	IO18	x
	VCC	3V3	3V3
	M-	x	BO02

Thiết bị ngoại vi thứ ba là cảm biến gia tốc góc BNO080, sơ đồ chân của BNO080 và Raspberry Pi 5 được trình bày trong Hình 3.6. Cảm biến được kết nối trực tiếp với các chân GPIO của Raspberry Pi 5 thông qua giao thức SPI. Dữ liệu thu thập từ cảm biến được Raspberry Pi 5 xử lý và phát hành dưới dạng các topic trong hệ thống ROS 2. BNO080 được sử dụng nhằm cung cấp thông tin về gia tốc và vận tốc góc của robot để hỗ trợ quá trình tính toán odometry, giúp giảm sai số do hiện tượng trượt bánh xe hoặc khi robot bị kẹt khiến encoder vẫn ghi nhận chuyển động trong khi vị trí thực tế của robot không thay đổi.



a. Sơ đồ chân I/O của raspberry pi 5

b. Sơ đồ chân của module BNO080

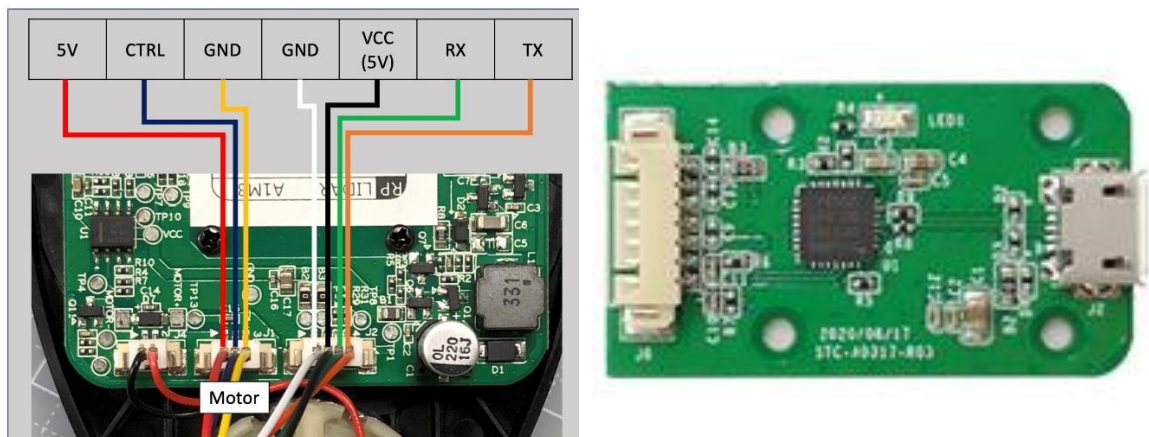
Hình 3.6 Sơ đồ chân của raspberry pi 5 và module BNO080

Bảng 3.5 Sơ đồ kết nối chân BNO080 với Raspberry pi 5

BNO080	Raspberry pi 5
VCC	3V3
GND	GND
SCK	GPIO11
MISO	GPIO 9
MOSI	GPIO 10
CS	GPIO 5
INT	GPIO 25
RST	GPIO 24

Bảng 3.5 mô tả sơ đồ kết nối chân giữa BNO080 và raspberry pi 5. Module cảm biến sử dụng nguồn 3.3V. Giao tiếp SPI được thực hiện thông qua các chân SCK, MISO, MOSI và CS, trong đó chân SCK dùng để đồng bộ xung nhịp truyền dữ liệu, chân MOSI dùng để truyền dữ liệu từ Raspberry Pi đến cảm biến, chân MISO dùng để truyền dữ liệu từ cảm biến về Raspberry Pi và chân CS được sử dụng để lựa chọn thiết bị trong quá trình giao tiếp. Chân INT được sử dụng để phát tín hiệu ngắt khi có dữ liệu mới cần xử lý, giúp giảm tải cho bộ xử lý trung tâm. Chân RST cho phép khởi động lại cảm biến khi cần thiết.

Thiết bị ngoại vi thứ tư là cảm biến LiDAR A1M8, được sử dụng để thu thập dữ liệu môi trường phục vụ cho quá trình xây dựng bản đồ, định vị và điều hướng robot. Cảm biến có khả năng quét 360° với phạm vi đo phù hợp cho các ứng dụng robot di động trong nhà, đáp ứng yêu cầu cập nhật môi trường theo thời gian thực. Bên cạnh đó, LiDAR A1M8 được hỗ trợ rộng rãi trên nền tảng ROS 2, có nhiều gói phần mềm và tài liệu phát triển sẵn, giúp đơn giản hóa quá trình tích hợp vào hệ thống. Cảm biến được kết nối với Raspberry Pi thông qua giao tiếp USB nhờ mạch chuyển đổi USB-UART tích hợp, cho phép thu nhận dữ liệu quét một cách thuận tiện. Sơ đồ chân lidar A1M8 và ngoại quan mạch chuyển đổi USB UART xem tại hình 3.7.



a. Sơ đồ chân lidar A1M8

b. Mạch chuyển đổi USB UART

Hình 3.7 Sơ đồ chân lidar A1M8 và mạch chuyển đổi USB UART

3.3.5 Khối xử lý trung tâm

Khối xử lý trung tâm bao gồm 2 phần chính là vi điều khiển ESP32-S3 được xem như bộ xử lý bậc thấp và máy tính nhúng raspberry pi 5 là bộ xử lý bậc cao. Đầu tiên là vi điều khiển ESP32-S3, đóng vai trò là bộ điều khiển thời gian thực của hệ thống. ESP32-S3 có nhiệm vụ thu thập dữ liệu từ các thiết bị ngoại vi như mạch thu RF, bộ mã hóa encoder và các lệnh điều khiển được gửi từ Raspberry Pi 5, sau đó xử lý và điều khiển tốc độ cũng như chiều quay của các động cơ DC

để robot di chuyển theo yêu cầu. Nhóm lựa chọn ESP32-S3 nhờ khả năng cấu hình chân linh hoạt thông qua kiến trúc ma trận GPIO, cho phép ánh xạ các giao thức truyền thông như SPI, I2C và UART đến nhiều chân GPIO khác nhau, giúp việc thiết kế phần cứng và bố trí mạch trở nên thuận tiện hơn. Đồng thời vì điều khiển này sở hữu lõi kép phù hợp cho việc vừa đọc dữ liệu thiết bị ngoại vi vừa giao tiếp với raspberry pi với tốc độ cao, đáp ứng yêu cầu thời gian thực. Sơ đồ kết nối giữa ESP32-S3 với các thiết bị ngoại vi được trình bày tại bảng 3.6.

Bảng 3.6 Kết nối giữa ESP32-S3 với các thiết bị ngoại vi

ESP32-S3	Mạch thu RF	Bộ mã hóa	PCF8575	TB6612FNG	CP2102
RX0	x	x	x	x	TXD
TX0	x	x	x	x	RXD
IO4,5 ,6, 7	x	C1_L, C2_L, C3_L, C4_L	x	x	x
IO8	x	x	x	PWMA_L	x
IO9	x	x	x	PWMA_R	x
IO11	MOSI	x	x	x	x
IO12	SCK	x	x	x	x
IO13	MISO	x	x	x	x
IO14	CS	x	x	x	x
IO15, 16, 17, 18	x	C1_R, C2_R, C3_R, C4_R	x	x	x
IO21	ACK	x	x	x	x
IO37	x	x	x	PWMB_L	x
IO40	x	x	SCL	x	x
IO41	x	x	SDA	x	x
IO42	x	x	INT	x	x
IO45	x	x	x	STBY_L	x
IO47	x	x	x	PWMB_R	x
IO48	x	x	x	STBY_R	x

Phần thứ hai trong khối xử lý chính là máy tính nhúng raspberry pi 5. Raspberry Pi 5 là máy tính nhúng có hiệu năng xử lý cao, được trang bị bộ vi xử lý ARM Cortex-A76 cùng khả năng hỗ trợ nhiều giao diện kết nối như USB, UART, SPI, I2C, GPIO và Ethernet. Thiết bị có khả năng chạy hệ điều hành Linux và đặc biệt là hệ điều hành robot ROS 2. Với khả năng đáp ứng tốt các yêu cầu tính toán của ROS 2, cùng hiệu năng xử lý đủ để triển khai các gói SLAM, Nav2 và xử lý dữ liệu cảm biến theo thời gian thực. Do đó Raspberry Pi 5 phù hợp với yêu cầu của hệ thống.

Bảng 3.7 Sơ đồ kết nối của raspberry pi 5 với các thiết bị khác

Raspberry pi 5	Thiết bị được kết nối
USB 2.0 A	ESP32-S3
USB 2.0 B	Lidar A1M8
GPIO 11	SCL (BNO080)
GPIO 9	MISO (BNO080)
GPIO 10	MOSI (BNO080)
GPIO 5	CS (BNO080)
GPIO 25	INT (BNO080)
GPIO 24	RST (BNO080)

3.3.6 Khối nguồn

Trong quá trình thiết kế hệ thống, việc tính toán công suất nguồn đóng vai trò quan trọng nhằm đảm bảo toàn bộ hệ thống hoạt động ổn định và an toàn trong mọi điều kiện vận hành. Công suất nguồn cần được xác định dựa trên tổng mức tiêu thụ của tất cả các thành phần như vi điều khiển, cảm biến, module truyền thông, động cơ và các thiết bị ngoại vi khác. Mỗi thành phần sẽ có mức điện áp và dòng điện tiêu thụ khác nhau, do đó cần quy đổi về cùng một hệ thống cung cấp để tính tổng công suất yêu cầu. Bên cạnh đó, khi thiết kế thực tế cần cộng thêm hệ số dự phòng (thường từ 20% đến 30%) để đảm bảo khả năng đáp ứng các trạng thái tải đột biến khi động cơ khởi động hoặc khi hệ thống hoạt động ở mức tải cao. Việc lựa chọn nguồn phù hợp không chỉ giúp hệ thống vận hành ổn định, giảm sụt áp và nhiễu điện, mà còn kéo dài tuổi thọ linh kiện và tăng độ tin cậy tổng thể của toàn bộ hệ thống. Để thiết kế được khối nguồn hạ áp cần phải tính được công suất tiêu thụ của các thiết bị trong hệ thống.

Công suất tiêu thụ của từng linh kiện trong hệ thống được xác định dựa trên điện áp hoạt động và dòng điện tiêu thụ của linh kiện đó theo công thức $P=U \times I$,

trong đó P là công suất (W), U là điện áp cấp (V) và I là dòng điện tiêu thụ (A). Giá trị điện áp và dòng điện được tham khảo từ datasheet hoặc tài liệu kỹ thuật của nhà sản xuất. Tổng công suất tiêu thụ của hệ thống được tính bằng tổng công suất của tất cả các linh kiện, từ đó làm cơ sở để lựa chọn nguồn cấp và pin có dung lượng phù hợp, đảm bảo hệ thống hoạt động ổn định trong quá trình vận hành.

Bảng 3.8 Công suất tiêu thụ các thiết bị

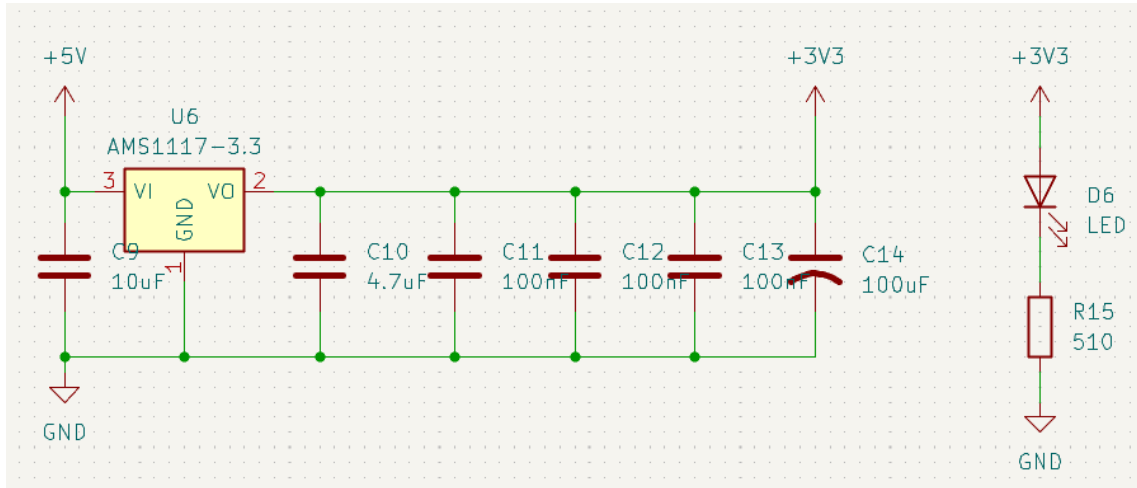
Thiết bị	Điện áp	Dòng tiêu thụ	Công suất
Raspberry Pi 5	5 V	5 A	25 W
ESP32-S3	3V3	0.35 A	0.5 W
RPLIDAR A1M8	3V3	0.4 A	2.0 W
BNO080	3V3	0.003 A	0.01 W
VL53L1X	3V3	0.02 A	0.07 W
PCF8575	3V3	0.001 A	0.003 W
Mạch thu RF	3V3	0.03 A	0.1 W
CP2102	3V3	0.025 A	0.05 W
TB6612FNG	3V3	0.0011A	0.003W
Động cơ encoder × 4	12V	1.2A	14.4 W

Bảng 3.9 Công suất tiêu thụ ở từng nhánh

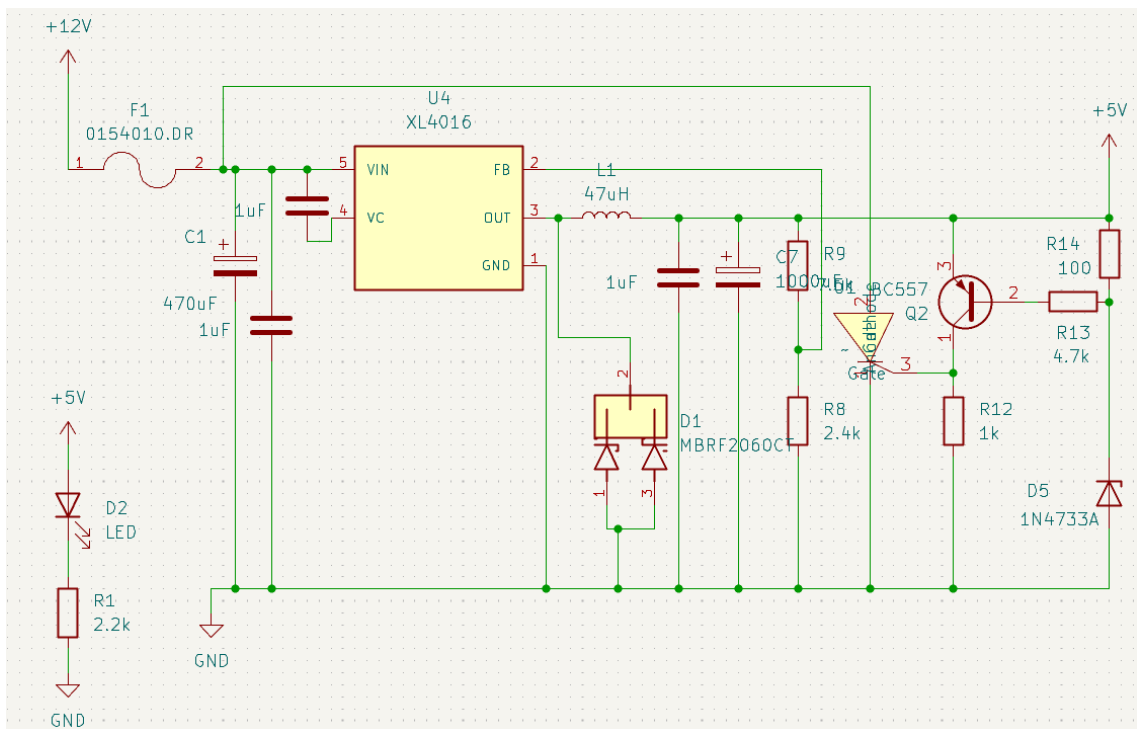
Nhánh tiêu thụ	Dòng tiêu thụ	Công suất tiêu thụ
3V3	0.43A	1.42W
5V	5.4A	27W
12V	4.8A	57.6W
Tổng	10.63a	86.02W

Bảng 3.1 là tổng công suất tiêu thụ của các thiết bị ở từng nhánh 3V3, 5V và 12V. Căn cứ vào số liệu này để lựa chọn thiết kế khối nguồn phù hợp, đáp ứng đủ công suất hoạt động của toàn bộ hệ thống một cách ổn định. Công suất khối nguồn phải lớn hơn công suất yêu cầu. Mục đích để đảm bảo khối nguồn hoạt động dưới 80% công suất tối đa, đảm bảo tính ổn định và hoạt động lâu dài.

Đối với nhánh 3V3, nhóm sử dụng IC AMS1117 là IC ổn áp 3V3 với dòng tải tối đa là 1A (3.3W), đáp ứng được yêu cầu của hệ thống 0.429A (1.4157W). IC AMS1117 là bộ ổn áp tuyến tính có khả năng cung cấp dòng ra tối đa khoảng 1A. AMS1117 có điện áp sụt thấp, giúp duy trì ổn áp khi chênh lệch giữa đầu vào và đầu ra nhỏ. sơ đồ nguyên lý mạch ổn áp 3V3 xem tại hình 3.8.



Hình 3.8 Sơ đồ nguyên lý mạch ổn áp dùng AMS1117



Hình 3.9 Sơ đồ nguyên lý mạch hạ áp 5V dùng XL4016

Đối với nhánh 5V, nhóm sử dụng IC XL4016 là IC hạ áp với dòng tải tối đa là 8A (Công suất tối đa 40W tại mức điện áp đầu ra 5V), đáp ứng được yêu cầu của

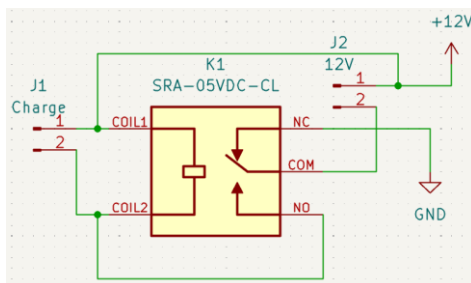
hệ thống 5.4A (24W). XL4016 được lựa chọn để tạo nguồn 5 V từ khối pin 12 V do có khả năng cung cấp dòng tải lớn, hiệu suất chuyển đổi cao và đáp ứng tốt nhu cầu cấp nguồn cho Raspberry Pi 5 cùng các thiết bị ngoại vi. Trong thiết kế, mạch hạ áp được tích hợp thêm mạch bảo vệ crowbar để bảo vệ quá áp tại ngõ ra 5V. Khi dòng tải quá lớn, một số mạch hạ áp có thể gặp sự cố ngõ ra không duy trì được như định mức. Mạch bảo vệ được thêm vào để lập tức ngắt mạch làm đứt cầu chì khi điện áp ngõ ra đột ngột tăng cao, bảo vệ các thiết bị sử dụng nguồn 5V. Sơ đồ nguyên lý mạch hạ áp 5V tại hình 3.9.

Toàn bộ nguồn của hệ thống được cung cấp bởi 1 khối pin pin lithium 3S2P (2 cục pin mắc song song thành 1 cặp, 3 cặp mắc nối tiếp tạo thành khối pin 3S2P). Tổng dung lượng pin 15000mAh, hệ số xả của mỗi cục pin là 3C, dòng xả mỗi cục pin là 7.5A, 2 khối pin mắc song song nên dòng xả tối đa khối pin là 15A, đáp ứng được yêu cầu của hệ thống.

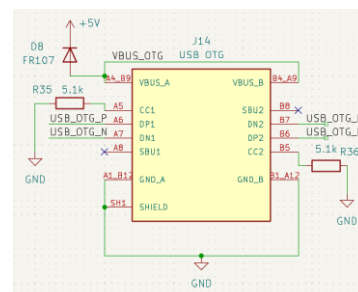
3.3.7 Thiết kế mạch điều khiển

Để đáp ứng yêu cầu tích hợp các thành phần phần cứng trên robot tự hành và đảm bảo hệ thống hoạt động ổn định, nhóm tiến hành thiết kế một mạch PCB. Việc sử dụng mạch PCB thay cho phương pháp kết nối dây rời giúp giảm số lượng dây dẫn trong hệ thống, thay thế việc sử dụng các dây dẫn rời rạc bằng các cục dây sử dụng chân cắm XH2.54 và các dây chuẩn typeC, USB để đảm bảo độ ổn định kết nối, tránh lỏng dây. Đồng thời cũng thuận tiện cho quá trình lắp ráp, sửa chữa.

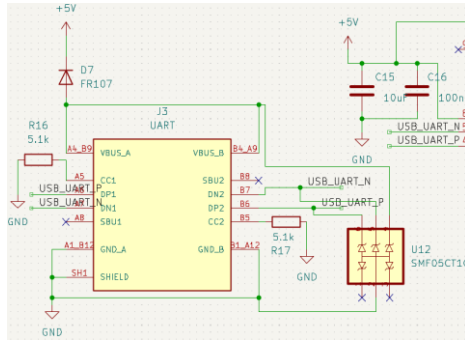
Trên mạch PCB, các khối chức năng chính của hệ thống được tích hợp bao gồm vi điều khiển ESP32-S3, các khối hạ áp nguồn 5V và 3V3, khối driver điều khiển động cơ, khối IC chuyển đổi giữa COM và UART với Raspberry Pi 5 và các cổng kết nối cảm biến. Việc tích hợp các khối chức năng trên cùng một bảng mạch giúp đơn giản hóa kiến trúc phần cứng, thuận lợi cho việc quản lý nguồn cũng như truyền nhận dữ liệu giữa các thành phần trong hệ thống, Sơ đồ nguyên lý mạch PCB xem tại hình 3.11.



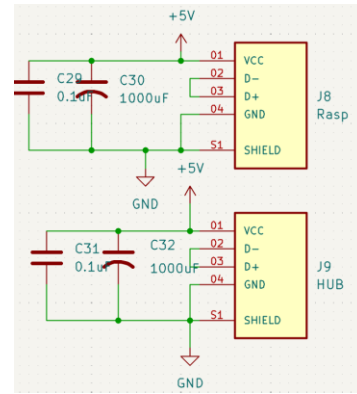
a. Ngắt nguồn khi sạc pin



a. Cổng typeC nạp code cho esp32



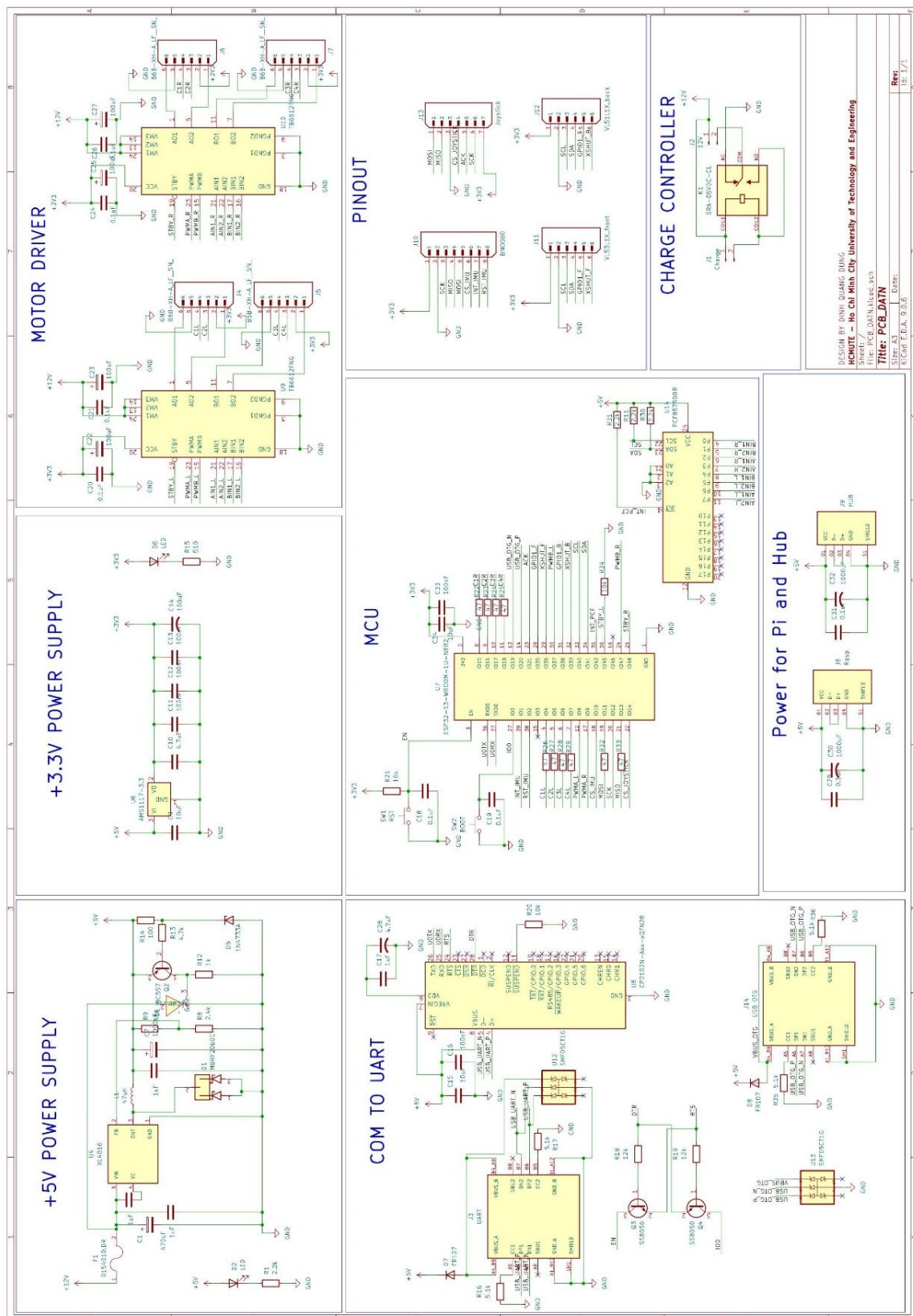
c. Cổng typeC truyền dữ liệu esp tới raspberry pi



d.2 cổng USB cấp nguồn cho HUB và raspberry pi

Hình 3.10 Một số cổng kết nối và linh kiện hỗ trợ mạch

Tại hình 3.10, một số cổng kết nối typeC và USB được thêm vào mạch được dùng để cấp nguồn cũng như truyền dữ liệu từ ESP32-S3 tới raspberry pi 5. Mục đích để đảm bảo tính ổn định kết nối hơn là sử dụng nhiều dây cắm rời tới các GPIO của raspberry. Tại hình 3.10a là relay được thêm vào để ngắt nguồn tải khi đang sạc khối pin 12V, hình 3.10b, 3.10c là cổng typeC để nạp code và truyền dữ liệu giữa ESP32 và raspberry pi. Hình 3.10d là cổng USB cấp nguồn 5V cho raspberry pi và cho HUB, đường nguồn và dữ liệu lidar được tách ra bởi HUB.

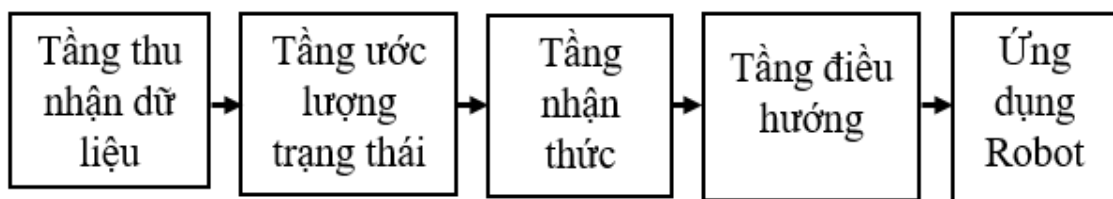


Hình 3.11 Sơ đồ nguyên lý mạch PCB

3.4 THIẾT KẾ PHẦN MỀM

Phần mềm đóng vai trò trung tâm trong việc điều khiển và quản lý hoạt động của robot tự hành. Trong đề tài này, hệ thống được phát triển trên nền tảng ROS 2 nhằm hỗ trợ việc trao đổi dữ liệu giữa các thành phần thông qua cơ chế node, topic, service, action và TF. Trên cơ sở đó, các chức năng như thu nhận dữ liệu cảm biến, tính toán odometry, xây dựng bản đồ, định vị, lập kế hoạch đường đi và điều hướng robot được triển khai thành các nhóm node chuyên biệt. Cách tổ chức này giúp hệ thống dễ dàng mở rộng, bảo trì và nâng cao tính linh hoạt trong quá trình phát triển.

3.4.1 Cấu trúc hệ thống phần mềm



Hình 3.12 Cấu trúc hệ thống phần mềm

Hệ thống phần mềm của robot tự hành được xây dựng trên nền tảng ROS 2 và được thiết kế theo kiến trúc phân tầng nhằm đảm bảo tính mô-đun, khả năng mở rộng và tính ổn định trong quá trình vận hành thực tế. Cấu trúc hệ thống phần mềm được thể hiện trong hình 3.12. Toàn bộ hệ thống được tổ chức thành nhiều lớp chức năng khác nhau, trong đó mỗi lớp đảm nhiệm một vai trò riêng. Các lớp được kết nối với nhau thông qua cơ chế giao tiếp của ROS 2 như topic, service, action và TF, từ đó hình thành một hệ thống thống nhất từ thu nhận dữ liệu đến điều khiển chuyển động. Ở tầng thấp nhất, các node thu nhận dữ liệu đảm nhiệm việc tiếp nhận tín hiệu từ LiDAR, IMU và encoder, sau đó thực hiện chuẩn hóa dữ liệu và cung cấp cho các tầng xử lý phía trên nhằm đảm bảo thông tin luôn được đồng bộ theo thời gian thực. Ở tầng xử lý trạng thái, dữ liệu từ encoder và IMU được kết hợp để tính toán odometry, qua đó xác định vị trí và hướng di chuyển tương đối của robot trong không gian, đồng thời cung cấp nền tảng cho các thuật toán định vị và xây dựng bản đồ. Trên cơ sở đó, tầng nhận thức môi trường sử dụng dữ liệu LiDAR kết hợp với trạng thái chuyển động của robot để thực hiện đồng thời việc xây dựng bản đồ và định vị thông qua các thuật toán SLAM, đồng thời duy trì sự nhất quán giữa các hệ tọa độ trong hệ thống thông qua cơ chế TF. Khi bản đồ đã được tạo sẵn, tầng điều hướng sẽ đảm nhiệm việc xác định vị trí hiện tại của robot trên bản đồ, sau đó lập kế hoạch đường đi và tạo

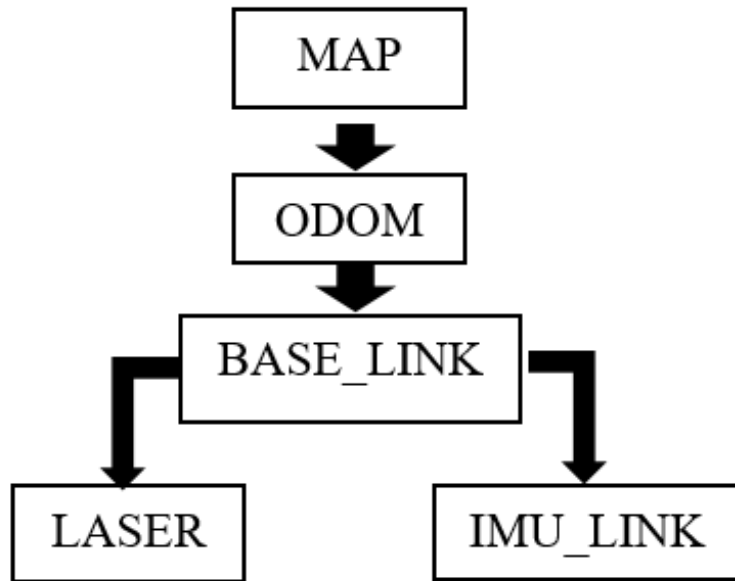
ra các lệnh điều khiển nhằm đưa robot di chuyển đến vị trí mục tiêu một cách chính xác và an toàn.

Các lệnh điều khiển này được truyền xuống tầng giao tiếp phần cứng, nơi một node trung gian đảm nhiệm việc truyền dữ liệu từ Raspberry Pi 5 đến ESP32-S3 thông qua giao tiếp UART, đồng thời tiếp nhận dữ liệu phản hồi từ encoder để cập nhật trạng thái chuyển động thực tế của robot trong suốt quá trình hoạt động. Toàn bộ hệ thống được tổ chức thành các gói ROS 2 riêng biệt tương ứng với từng nhóm chức năng như xử lý cảm biến, ước lượng trạng thái, xây dựng bản đồ, điều hướng và giao tiếp phần cứng, giúp các thành phần có thể hoạt động độc lập nhưng vẫn đảm bảo sự phối hợp thống nhất trong toàn bộ kiến trúc phần mềm. Cách tổ chức này đồng thời tạo điều kiện thuận lợi cho việc kiểm thử, bảo trì và mở rộng hệ thống trong tương lai.

3.4.2 Hệ tọa độ và cây TF của hệ thống

Trong hệ thống robot tự hành, dữ liệu được thu thập từ nhiều nguồn khác nhau như LiDAR, IMU và encoder. Tuy nhiên, các cảm biến này được lắp đặt tại những vị trí khác nhau trên robot nên dữ liệu thu được sẽ được biểu diễn trong các hệ tọa độ riêng biệt. Để các thành phần phần mềm có thể sử dụng và kết hợp dữ liệu một cách chính xác, ROS 2 sử dụng TF để quản lý mối quan hệ hình học giữa các hệ tọa độ trong hệ thống. Thông qua TF, vị trí và hướng của robot có thể được xác định liên tục trong môi trường, đồng thời dữ liệu cảm biến cũng được chuyển đổi về cùng một hệ quy chiếu trước khi đưa vào các thuật toán xử lý. Hình 3.2 trình bày cây TF được sử dụng trong hệ thống robot của đề tài.

Cây TF được tổ chức theo cấu trúc phân cấp từ hệ tọa độ bản đồ đến thân robot và các cảm biến gắn trên robot. Trong quá trình hoạt động, dữ liệu encoder và IMU được sử dụng để mô tả sự thay đổi vị trí của robot theo thời gian, trong khi dữ liệu LiDAR được sử dụng để xây dựng bản đồ hoặc xác định vị trí của robot trên bản đồ đã có. Từ các thông tin này, hệ thống thiết lập mối liên hệ giữa bản đồ môi trường, vị trí robot và các cảm biến được lắp đặt trên robot, giúp mọi dữ liệu đều được biểu diễn trong cùng một hệ quy chiếu thống nhất. Nhờ đó, SLAM Toolbox có thể xây dựng bản đồ môi trường, AMCL có thể xác định vị trí hiện tại của robot trên bản đồ và Nav2 có thể lập kế hoạch đường đi cũng như điều khiển robot di chuyển đến vị trí mục tiêu. Việc duy trì cây TF thống nhất là điều kiện cần để các thành phần trong hệ thống hoạt động đồng bộ và đảm bảo độ chính xác trong quá trình tự hành.

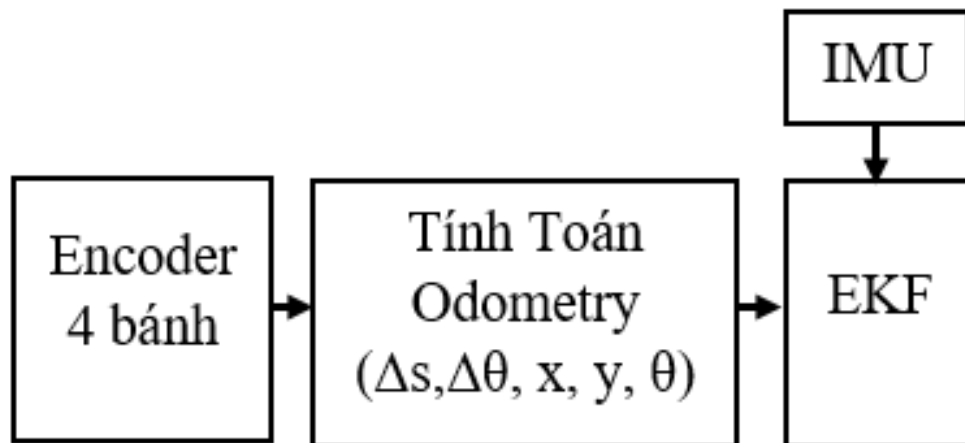


Hình 3.13 Cây TF của hệ thống

3.4.3 Tính toán odometry và hợp nhất dữ liệu cảm biến

Trong hệ thống robot tự hành, thông tin odometry đóng vai trò quan trọng trong quá trình xây dựng bản đồ, định vị và điều hướng. Odometry cung cấp thông tin về vị trí và hướng chuyển động tương đối của robot dựa trên dữ liệu phản hồi từ encoder gắn trên các động cơ. Trong phần này, nội dung tập trung mô tả cách triển khai quá trình tính toán và hợp nhất dữ liệu cảm biến trên hệ thống thực tế.

Hình 3.14 mô tả luồng xử lý dữ liệu được sử dụng để ước lượng trạng thái chuyển động của robot. Robot sử dụng bốn động cơ DC encoder để ghi nhận tốc độ quay của các bánh xe, trong đó dữ liệu encoder được ESP32-S3 thu thập và truyền đến Raspberry Pi 5 thông qua giao tiếp USB/UART. Tại Raspberry Pi 5, node odometry thực hiện tính toán trạng thái chuyển động của robot theo mô hình động học vi sai. Do robot sử dụng hai bánh bên trái và hai bánh bên phải, vận tốc của mỗi phía được xác định từ giá trị trung bình của hai encoder tương ứng nhằm giảm ảnh hưởng của sai số đo lường trên từng bánh xe. Từ các dữ liệu này, hệ thống xác định quãng đường dịch chuyển, góc quay và tư thế của robot trong mặt phẳng, bao gồm tọa độ x , y và góc định hướng θ . Tuy nhiên, odometry từ encoder thường xuất hiện sai số tích lũy trong quá trình vận hành do hiện tượng trượt bánh xe, sai số cơ khí của hệ truyền động, độ phân giải hữu hạn của encoder cũng như ảnh hưởng của điều kiện bề mặt di chuyển. Khi robot hoạt động trong thời gian dài, các sai số này có thể làm giảm độ chính xác của trạng thái ước lượng.



Hình 3.14 Odometry và hợp nhất dữ liệu cảm biến

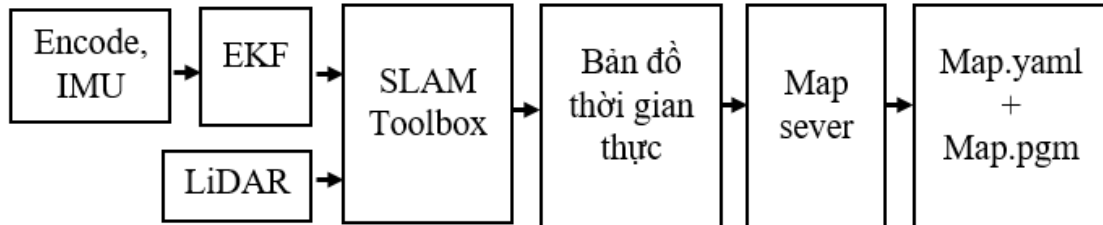
Để nâng cao độ chính xác của trạng thái robot, hệ thống sử dụng thêm cảm biến IMU BNO080 nhằm cung cấp thông tin quán tính về chuyển động của robot. Dữ liệu IMU sau khi được xử lý sẽ được đưa vào bộ lọc EKF cùng với dữ liệu odometry từ encoder để thực hiện quá trình hợp nhất cảm biến. Việc kết hợp hai nguồn dữ liệu này cho phép tận dụng ưu điểm của từng loại cảm biến, trong đó encoder cung cấp thông tin quãng đường di chuyển với độ phân giải cao, còn IMU hỗ trợ xác định hướng quay và các thay đổi trạng thái xảy ra trong thời gian ngắn. Nhờ đó, bộ lọc EKF có thể tạo ra thông tin trạng thái ổn định và tin cậy hơn so với việc chỉ sử dụng odometry từ encoder đơn lẻ. Kết quả hợp nhất này được sử dụng làm cơ sở cho các chức năng xây dựng bản đồ, định vị và điều hướng được trình bày trong các mục tiếp theo, góp phần nâng cao độ chính xác và độ ổn định của hệ thống robot tự hành trong môi trường thực tế.

3.4.4 Hệ thống xây dựng bản đồ bằng SLAM Toolbox

Xây dựng bản đồ là một trong những chức năng quan trọng của robot tự hành, cho phép robot nhận biết môi trường hoạt động và tạo dữ liệu phục vụ cho quá trình định vị, lập kế hoạch đường đi và điều hướng sau này. Trong đề tài này, quá trình xây dựng bản đồ được thực hiện bằng gói SLAM Toolbox trên nền tảng ROS 2. Hình 3.15 trình bày luồng xử lý dữ liệu trong chế độ xây dựng bản đồ.

Trong quá trình hoạt động, dữ liệu quét môi trường từ LiDAR và thông tin trạng thái chuyển động của robot từ bộ lọc EKF được đồng thời đưa vào SLAM Toolbox để thực hiện quá trình xây dựng bản đồ. Dữ liệu LiDAR cung cấp thông tin về các vật thể và đặc trưng trong môi trường, trong khi dữ liệu trạng thái từ EKF cung cấp thông tin về vị trí và hướng chuyển động của robot tại từng thời điểm. Dựa trên hai nguồn dữ liệu này cùng với hệ tọa độ TF của hệ thống, SLAM Toolbox thực hiện đồng thời hai nhiệm vụ là ước lượng vị trí của robot và xây

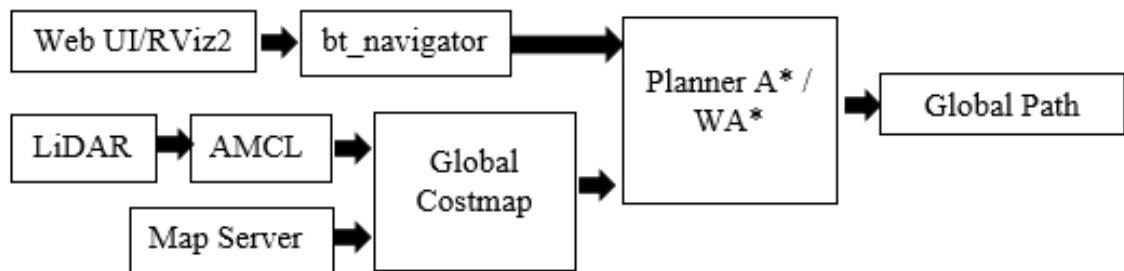
dựng bản đồ môi trường theo thời gian thực. Khi robot di chuyển, các lần quét LiDAR mới liên tục được so khớp với dữ liệu đã thu thập trước đó để xác định sự thay đổi vị trí của robot, đồng thời cập nhật bản đồ dựa trên những quan sát mới thu được từ môi trường.



Hình 3.15 Trình bày luồng xử lý dữ liệu trong chế độ xây dựng bản đồ

Kết quả của quá trình xử lý là một bản đồ thời gian thực dưới dạng Occupancy Grid Map, trong đó môi trường được biểu diễn bằng các ô lưới thể hiện vùng tự do, vùng có vật cản hoặc vùng chưa được quan sát. Sau khi robot hoàn thành quá trình khảo sát môi trường, bản đồ này được lưu lại thông qua công cụ Map Saver. Hệ thống tạo ra hai tệp dữ liệu gồm tệp hình ảnh bản đồ (Map.pgm) và tệp cấu hình bản đồ (Map.yaml), cho phép bản đồ được sử dụng lại trong các chế độ định vị và điều hướng mà không cần thực hiện quá trình xây dựng bản đồ từ đầu. Việc sử dụng SLAM Toolbox giúp hệ thống có khả năng cập nhật bản đồ theo thời gian thực, duy trì độ chính xác trong quá trình vận hành và tạo nền tảng dữ liệu cần thiết cho các chức năng tự hành ở các giai đoạn tiếp theo.

3.4.5 Hệ thống định vị và điều hướng bằng Nav2

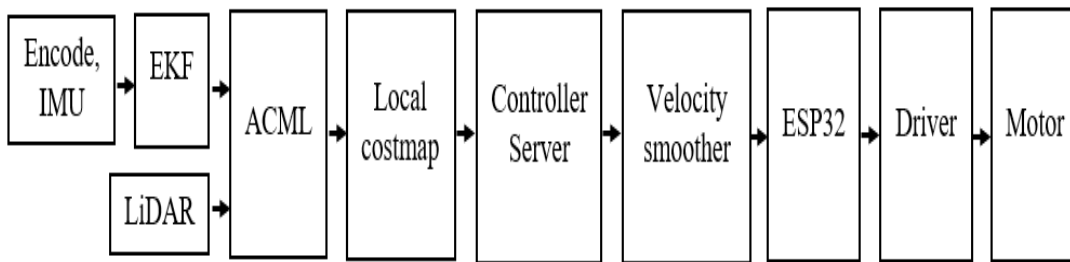


Hình 3.16 Quá trình lập kế hoạch đường đi

Sau khi quá trình xây dựng bản đồ môi trường được hoàn tất, hệ thống robot chuyển sang chế độ định vị và điều hướng tự động dựa trên nền tảng Nav2 nhằm thực hiện việc di chuyển từ vị trí hiện tại đến vị trí mục tiêu trong không gian làm việc. Quá trình lập kế hoạch đường đi mô tả trong sơ đồ hình 3.16. Toàn bộ quá trình điều hướng được thực hiện trên cơ sở kết hợp giữa bản đồ đã lưu, dữ liệu

cảm biến thời gian thực và hệ thống định vị, trong đó Nav2 đóng vai trò trung tâm điều phối toàn bộ luồng xử lý từ lập kế hoạch đường đi đến điều khiển chuyển động.

Trong giai đoạn này, khi người dùng lựa chọn vị trí đích thông qua giao diện Web UI hoặc RViz2, thông tin mục tiêu sẽ được gửi đến bộ điều phối `bt_navigator`, đây là thành phần trung tâm của Nav2 chịu trách nhiệm quản lý toàn bộ tiến trình điều hướng dựa trên kiến trúc cây hành vi. Song song với đó, bản đồ môi trường đã được lưu trước đó được nạp vào hệ thống thông qua `map_server` và được sử dụng để xây dựng `global_costmap`, trong đó thể hiện chi phí di chuyển trên toàn bộ khu vực hoạt động của robot. Trên cơ sở `global_costmap` kết hợp với thông tin vị trí hiện tại của robot, `planner_server` thực hiện việc tìm kiếm đường đi toàn cục bằng thuật toán A* cải tiến có tích hợp hệ số trọng số WA*, nhằm tối ưu hóa thời gian tìm kiếm và giảm số lượng nút mở rộng trong quá trình lập kế hoạch. Kết quả thu được là một quỹ đạo toàn cục được xuất bản dưới dạng `global path`, đóng vai trò làm đầu vào cho giai đoạn điều khiển bám quỹ đạo phía sau.



Hình 3.17 Quá trình bám quỹ đạo và điều khiển chuyển động

Hình 3.17 là sơ đồ quá trình bám quỹ đạo và điều khiển chuyển động. Sau khi quỹ đạo toàn cục được tạo ra, hệ thống chuyển sang giai đoạn điều khiển chuyển động, trong đó `controller_server` tiếp nhận `global path` và thực hiện tính toán vận tốc điều khiển để robot có thể bám theo quỹ đạo đã được lập kế hoạch. Trong quá trình này, `controller_server` liên tục kết hợp thông tin từ `global path`, `local_costmap` và trạng thái hiện tại của robot để đưa ra lệnh điều khiển phù hợp, đồng thời `local_costmap` được cập nhật liên tục từ dữ liệu LiDAR nhằm phản ánh các vật cản xuất hiện trong môi trường thực tế và hỗ trợ khả năng tránh va chạm theo thời gian thực. Thông tin trạng thái của robot được cung cấp bởi hệ thống ước lượng dựa trên bộ lọc EKF, trong đó dữ liệu encoder được truyền từ hệ truyền động về Raspberry Pi thông qua giao tiếp serial và được xử lý để tạo odometry, kết hợp với dữ liệu IMU đã được lọc nhiễu để tạo ra trạng thái chuyển động ổn định hơn. Dựa trên trạng thái này, `controller_server` tính toán lệnh vận

tốc và xuất ra tín hiệu điều khiển, sau đó tín hiệu này được làm mượt thông qua `velocity_smoother` nhằm hạn chế thay đổi đột ngột về gia tốc và đảm bảo chuyển động ổn định của robot. Cuối cùng, lệnh vận tốc sau khi xử lý được truyền qua node `bridge` đến vi điều khiển ESP32-S3 thông qua giao tiếp UART, tại đây tín hiệu được chuyển đổi thành PWM để điều khiển driver TB6612FNG và tạo chuyển động cho các động cơ DC encoder, đồng thời hệ thống tiếp tục nhận phản hồi từ encoder để tạo thành vòng điều khiển khép kín giữa cảm biến, bộ xử lý và cơ cấu chấp hành.

3.5 THUẬT TOÁN A* VÀ WA* TRONG LẬP KẾ HOẠCH ĐƯỜNG ĐI

Khi robot đã được định vị trên bản đồ môi trường, hệ thống điều hướng cần xây dựng quỹ đạo từ vị trí hiện tại đến vị trí mục tiêu nhằm phục vụ quá trình di chuyển tự động. Trong kiến trúc Nav2, nhiệm vụ này được thực hiện bởi Planner Server thông qua các thuật toán tìm đường trên bản đồ chi phí. Để nâng cao hiệu quả lập kế hoạch, đề tài nghiên cứu và áp dụng thuật toán A* cùng biến thể A* có trọng số (WA*). Nội dung của mục này trình bày mô hình bài toán tìm đường, nguyên lý hoạt động của các thuật toán và quá trình triển khai trong hệ thống robot tự hành.

3.5.1 Mô hình bài toán tìm đường đi trên bản đồ chi phí

Sau khi vị trí hiện tại của robot được xác định thông qua AMCL và vị trí đích được người dùng thiết lập trên giao diện RViz2, hệ thống điều hướng cần thực hiện quá trình lập kế hoạch đường đi nhằm đưa robot đến mục tiêu một cách an toàn và hiệu quả. Để thực hiện chức năng này, môi trường làm việc của robot được biểu diễn dưới dạng bản đồ chi phí, trong đó mỗi ô lưới phản ánh mức độ an toàn hoặc khả năng di chuyển của robot tại vị trí tương ứng. Bản đồ chi phí đóng vai trò là cơ sở để thuật toán lập kế hoạch đánh giá các phương án di chuyển và lựa chọn quỹ đạo phù hợp. Thông qua cách biểu diễn này, hệ thống có thể xem bài toán điều hướng như một bài toán tìm đường trên không gian lưới với điểm xuất phát và điểm đích đã được xác định trước.

Trong quá trình lập kế hoạch, vị trí hiện tại của robot được xem là điểm bắt đầu, trong khi vị trí mục tiêu được xem là điểm kết thúc của quá trình tìm kiếm. Các ô lưới có thể di chuyển được sẽ được sử dụng để xây dựng các nút của đồ thị tìm kiếm, còn các vùng chứa vật cản hoặc có chi phí cao sẽ bị hạn chế hoặc loại bỏ khỏi quỹ đạo. Trên cơ sở đó, thuật toán tìm đường cần xác định một chuỗi các nút liên tiếp kết nối từ điểm xuất phát đến điểm đích với tổng chi phí nhỏ nhất. Kết quả thu được là một quỹ đạo toàn cục thể hiện hướng di chuyển dự kiến của robot trong môi trường. Quỹ đạo này sau đó được chuyển cho bộ điều khiển để tạo ra các lệnh vận tốc tương ứng trong quá trình vận hành thực tế.

3.5.2 Thuật toán A* truyền thống

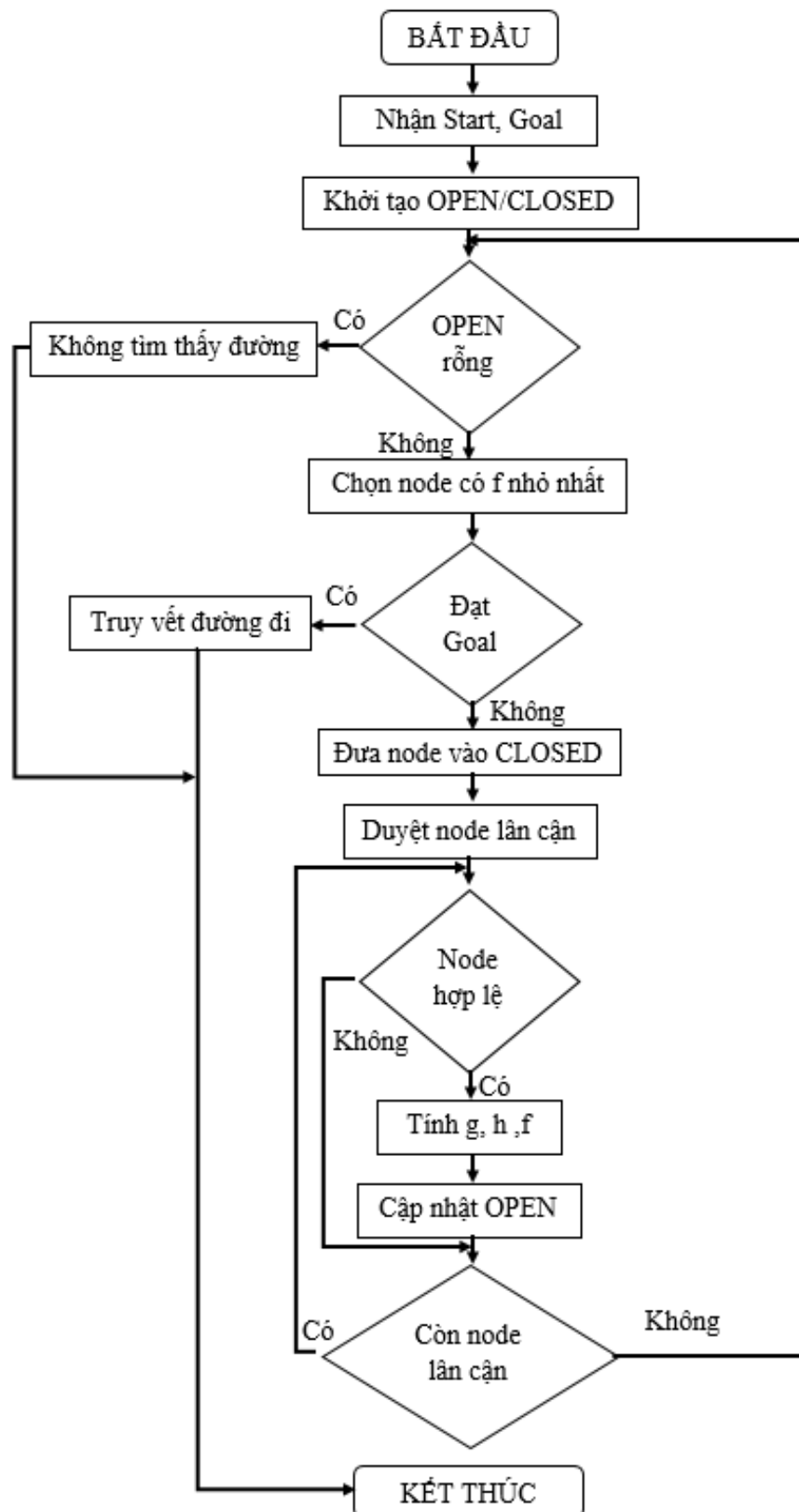
Để giải quyết bài toán tìm đường trên bản đồ chi phí đã trình bày ở mục trước, đề tài sử dụng thuật toán A* làm cơ sở cho quá trình lập kế hoạch đường đi. Thuật toán này kết hợp giữa chi phí di chuyển thực tế từ điểm xuất phát và chi phí ước lượng đến vị trí đích nhằm xác định hướng tìm kiếm phù hợp. Đối với mỗi node đang xét, giá trị đánh giá được xác định bằng hàm:

$$f(n)=g(n)+h(n), \quad (3.1)$$

trong đó $g(n)$ là chi phí thực tế từ vị trí xuất phát đến node hiện tại, $h(n)$ là chi phí ước lượng từ node hiện tại đến vị trí đích và $f(n)$ là tổng chi phí dùng để xác định mức độ ưu tiên của node. Nhờ sử dụng đồng thời hai thành phần chi phí này, thuật toán có khả năng định hướng quá trình tìm kiếm về phía mục tiêu thay vì phải duyệt toàn bộ không gian trạng thái. Điều này giúp giảm số lượng node cần xử lý và nâng cao hiệu quả tìm kiếm đường đi trong môi trường có kích thước lớn.

Quy trình thực hiện thuật toán được mô tả trong lưu đồ hình 3.18. Ban đầu, hệ thống tiếp nhận vị trí xuất phát và vị trí đích, sau đó khởi tạo hai tập dữ liệu OPEN và CLOSED để quản lý các node trong quá trình tìm kiếm. Node xuất phát được đưa vào tập OPEN và được sử dụng làm điểm bắt đầu của thuật toán. Trong mỗi vòng lặp, thuật toán kiểm tra trạng thái của tập OPEN nhằm xác định khả năng tiếp tục tìm kiếm. Nếu tập OPEN rỗng, hệ thống kết luận không tồn tại đường đi khả thi giữa vị trí xuất phát và vị trí đích. Ngược lại, node có giá trị f nhỏ nhất sẽ được lựa chọn để tiếp tục xử lý do đây là node có tiềm năng cao nhất tại thời điểm hiện tại.

Sau khi lựa chọn được node hiện tại, thuật toán tiến hành kiểm tra xem node này đã đạt đến vị trí đích hay chưa. Nếu node hiện tại chính là node đích, quá trình tìm kiếm kết thúc và thuật toán thực hiện truy vết ngược thông qua các node cha để xây dựng đường đi hoàn chỉnh. Trong trường hợp chưa đạt đến đích, node hiện tại sẽ được chuyển sang tập CLOSED và các node lân cận sẽ được xem xét. Đối với từng node lân cận hợp lệ, thuật toán tính toán các giá trị chi phí tương ứng và cập nhật thông tin nếu tìm được đường đi tốt hơn. Những node đủ điều kiện sẽ được đưa vào tập OPEN để tiếp tục xử lý trong các vòng lặp tiếp theo. Quá trình này được lặp lại liên tục cho đến khi tìm được vị trí đích hoặc không còn node khả thi nào trong tập OPEN. Kết quả cuối cùng của thuật toán là một chuỗi các node liên tiếp biểu diễn quỹ đạo từ vị trí xuất phát đến vị trí mục tiêu.



Hình 3.18 Lưu đồ thuật toán A*

3.5.3 Thuật toán WA*

Thuật toán A* có trọng số (Weighted A*) là một biến thể của thuật toán A* truyền thống, trong đó thành phần heuristic được nhân với một hệ số trọng số (W) nhằm tăng mức độ ưu tiên cho quá trình định hướng tìm kiếm về phía mục tiêu. Việc điều chỉnh hệ số trọng số cho phép giảm không gian tìm kiếm, từ đó cải thiện hiệu quả tính toán của thuật toán trong các bài toán lập kế hoạch đường đi.

```
1: procedure Weighted_Astar(M, s, g, W)
2:   Inputs
3:     M : Bản đồ chi phí toàn cục (Global Costmap)
4:     s : Node bắt đầu (vị trí hiện tại của robot)
5:     g : Node đích (vị trí mục tiêu)
6:     W : Hệ số trọng số,  $W \geq 1$ 
7:   Output
8:     P : Đường đi toàn cục từ s đến g
9:     hoặc  $\perp$  nếu không tồn tại đường đi khả thi
10:  Local
11:    OPEN  : Tập các node chờ xét
12:    CLOSED : Tập các node đã xét
13:    parent : Mảng lưu node cha
14:    g_cost : Chi phí thực tế từ node bắt đầu
15:    f      : Hàm đánh giá node
16:    OPEN  $\leftarrow \{s\}$ 
17:    CLOSED  $\leftarrow \emptyset$ 
18:    g_cost(s)  $\leftarrow 0$ 
19:    f(s)  $\leftarrow$  g_cost(s) +  $W \times h(s)$ 
20:    while OPEN  $\neq \emptyset$  do
21:      n  $\leftarrow$  Node trong OPEN có giá trị f nhỏ nhất
22:      if n = g then
23:        Truy vết các node cha và trả về đường đi P
24:      end if
```

```

25:   Chuyển n từ OPEN sang CLOSED
26:   for each node lân cận m của n do
27:       if m ∉ CLOSED then
28:           tentative_g ← g_cost(n) + c(n,m)
29:           if m ∉ OPEN
30:               hoặc tentative_g < g_cost(m) then
31:                   parent(m) ← n
32:                   g_cost(m) ← tentative_g
33:                   f(m) ← g_cost(m) + W × h(m)
34:                   if m ∉ OPEN then
35:                       Đưa m vào OPEN
36:                   end if
37:               end if
38:           end if
39:       end for
40:   return ⊥

```

Do WA* kế thừa toàn bộ cơ chế tìm kiếm của thuật toán A* truyền thống. Nên quy trình thực hiện thuật toán vẫn được mô tả bởi lưu đồ trong hình 3.18. Điểm khác biệt nằm ở hàm đánh giá node, được xác định theo hàm:

$$f(n)=g(n)+W \times h(n), \quad (3.2)$$

trong đó $(g(n))$ là chi phí thực tế từ node xuất phát đến node hiện tại, $(h(n))$ là chi phí ước lượng từ node hiện tại đến node đích và (W) là hệ số trọng số của hàm heuristic. Khi $(W=1)$, thuật toán trở thành A* truyền thống. Khi $(W>1)$, thành phần heuristic được ưu tiên nhiều hơn trong quá trình đánh giá node, giúp thuật toán tập trung tìm kiếm theo hướng mục tiêu và giảm số lượng node cần mở rộng. Nhờ đó, thời gian lập kế hoạch đường đi có thể được rút ngắn đáng kể. Tuy nhiên, việc tăng giá trị (W) cũng có thể làm giảm tính tối ưu của đường đi tìm được do thuật toán ưu tiên tốc độ tìm kiếm hơn chất lượng lời giải. Trong phạm vi đề tài này, thuật toán Weighted A* được sử dụng để thay thế hàm đánh giá của bộ lập kế hoạch đường đi toàn cục nhằm khảo sát ảnh hưởng của hệ số trọng số (W) đến hiệu năng lập kế hoạch đường đi của robot tự hành.

3.5.4 Quy trình thực hiện thuật toán WA^* trong hệ thống robot

Sau khi vị trí hiện tại của robot được xác định thông qua AMCL và vị trí đích được người dùng thiết lập trên giao diện RViz2, hệ thống điều hướng Nav2 sẽ thực hiện quá trình lập kế hoạch đường đi toàn cục. Trong kiến trúc này, BT Navigator chịu trách nhiệm tiếp nhận mục tiêu điều hướng và chuyển yêu cầu đến Planner Server. Đồng thời, thông tin vị trí hiện tại của robot được cung cấp thông qua dữ liệu pose và các phép biến đổi TF trong ROS 2. Trên cơ sở đó, Planner Server có thể xác định đầy đủ điểm xuất phát, điểm đích và trạng thái hiện tại của robot trước khi bắt đầu quá trình tìm đường.

Để thực hiện việc lập kế hoạch, Planner Server sử dụng Global Costmap được xây dựng từ bản đồ môi trường kết hợp với các thông tin chi phí an toàn xung quanh vật cản. Bản đồ chi phí này được chuyển đổi thành không gian tìm kiếm dạng lưới, trong đó mỗi ô lưới được xem như một node của đồ thị. Các vùng chứa vật cản hoặc nằm trong khu vực cấm di chuyển sẽ bị loại bỏ khỏi quá trình tìm kiếm nhằm đảm bảo quỹ đạo tạo ra có tính khả thi trong thực tế. Dựa trên các dữ liệu đầu vào đã được chuẩn bị, thuật toán Weighted A^* tiến hành tìm kiếm đường đi từ vị trí hiện tại của robot đến vị trí mục tiêu. Việc sử dụng hệ số trọng số W giúp tăng mức độ ưu tiên của thành phần heuristic, từ đó rút ngắn thời gian tìm kiếm và giảm số lượng node cần mở rộng so với A^* truyền thống.

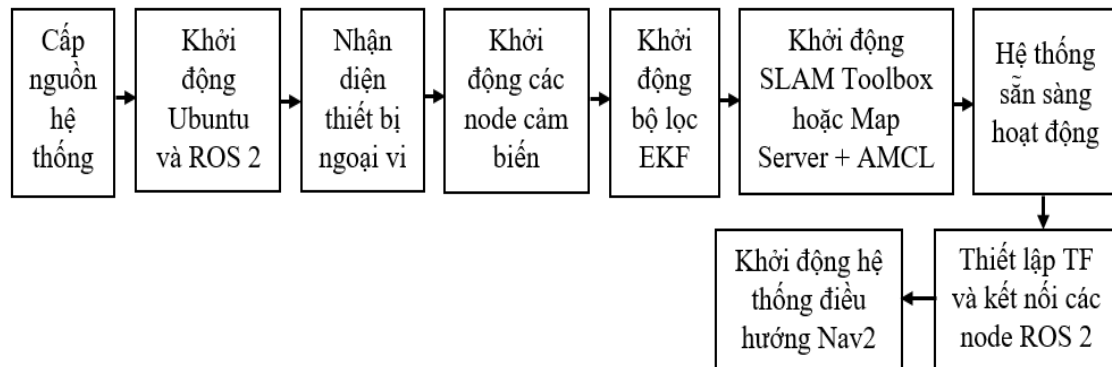
Khi quá trình tìm kiếm hoàn tất, Planner Server thu được một chuỗi các điểm trung gian biểu diễn quỹ đạo toàn cục của robot trên bản đồ môi trường. Quỹ đạo này được xuất bản dưới dạng topic /plan và được sử dụng làm đầu vào cho Controller Server của Nav2. Tiếp theo, bộ điều khiển cục bộ kết hợp quỹ đạo toàn cục với dữ liệu cảm biến thời gian thực để tính toán các lệnh vận tốc phù hợp cho robot. Các lệnh điều khiển sau đó được gửi đến ESP32 để điều khiển động cơ và thực hiện quá trình bám quỹ đạo. Nhờ sự phối hợp giữa AMCL, Global Costmap, Planner Server và Controller Server, robot có thể di chuyển từ vị trí hiện tại đến vị trí đích một cách tự động và an toàn trong môi trường hoạt động thực tế.

3.6 TÍCH HỢP VÀ VẬN HÀNH HỆ THỐNG

Việc xây dựng thành công các thành phần phần cứng, phần mềm và thuật toán mới chỉ tạo nên những khối chức năng riêng lẻ của hệ thống robot tự hành. Để robot có thể hoạt động trong môi trường thực tế, các thành phần này cần được tích hợp thành một kiến trúc thống nhất, trong đó dữ liệu được trao đổi liên tục giữa cảm biến, bộ xử lý trung tâm, bộ điều khiển và các node ROS 2. Đồng thời, hệ thống phải được tổ chức theo một quy trình khởi động và vận hành phù hợp nhằm bảo đảm các chức năng xây dựng bản đồ, định vị, lập kế hoạch đường đi và điều khiển chuyển động hoạt động đồng bộ. Vì vậy, mục này trình bày quá trình

tích hợp các thành phần trong hệ thống cũng như quy trình vận hành robot trong các chế độ làm việc khác nhau.

3.6.1 Quy trình khởi động hệ thống



Hình 3.19 Sơ đồ khối quy trình khởi động robot tự hành

Sau khi hoàn thành việc lắp ráp phần cứng và triển khai các thành phần phần mềm trên nền tảng ROS 2, hệ thống robot tự hành được khởi động theo một trình tự nhất định nhằm đảm bảo các thiết bị và phần mềm hoạt động đồng bộ. Đầu tiên, nguồn điện từ khối pin được cấp cho Raspberry Pi 5, ESP32-S3 và các cảm biến ngoại vi. Sau khi khởi động hệ điều hành Ubuntu, Raspberry Pi tiến hành nhận diện các thiết bị được kết nối như LiDAR, cảm biến IMU BNO080 và board điều khiển trung tâm thông qua giao tiếp USB. Tiếp theo, các node cảm biến được kích hoạt để thu thập dữ liệu môi trường và trạng thái chuyển động của robot. Dữ liệu từ LiDAR, IMU và encoder được truyền đến hệ thống ROS 2 nhằm phục vụ cho các chức năng định vị, xây dựng bản đồ và điều hướng ở các bước tiếp theo. Đồng thời, board ESP32-S3 liên tục gửi dữ liệu encoder về Raspberry Pi để hỗ trợ quá trình tính toán odometry của robot.

Sau khi dữ liệu đầu vào hoạt động ổn định, gói `robot_localization` được khởi chạy để thực hiện hợp nhất dữ liệu encoder và IMU thông qua bộ lọc EKF, từ đó tạo ra thông tin odometry có độ chính xác cao hơn. Trên cơ sở dữ liệu này, hệ thống sẽ khởi động SLAM Toolbox trong trường hợp xây dựng bản đồ mới hoặc khởi động Map Server kết hợp với AMCL khi thực hiện điều hướng trên bản đồ đã có sẵn. Cuối cùng, toàn bộ các thành phần của Nav2 bao gồm Planner Server, Controller Server, BT Navigator, Costmap và Velocity Smoother được kích hoạt để hình thành hệ thống điều hướng hoàn chỉnh. Khi các node đã được kết nối đầy đủ và các cây TF được thiết lập chính xác, robot sẵn sàng tiếp nhận mục tiêu điều hướng từ RViz2 hoặc giao diện Web để bắt đầu thực hiện nhiệm vụ.

3.6.2 Quy trình vận hành robot trong các chế độ làm việc

Hệ thống robot được thiết kế để hoạt động trong hai chế độ chính gồm chế độ xây dựng bản đồ và chế độ điều hướng tự động. Trong chế độ xây dựng bản đồ, robot được điều khiển thủ công thông qua tay cầm RF không dây để di chuyển trong khu vực làm việc. Trong quá trình di chuyển, dữ liệu quét từ LiDAR được thu thập liên tục và gửi đến SLAM Toolbox, đồng thời dữ liệu odometry và IMU được sử dụng để hỗ trợ quá trình ước lượng chuyển động của robot. Trên cơ sở các nguồn dữ liệu này, SLAM Toolbox thực hiện ghép nối các lần quét laser liên tiếp nhằm xây dựng bản đồ môi trường theo thời gian thực. Sau khi hoàn tất việc khảo sát khu vực làm việc, bản đồ được lưu dưới dạng các tệp .pgm và .yaml để sử dụng cho quá trình định vị và điều hướng ở các lần vận hành tiếp theo.

Trong chế độ điều hướng tự động, hệ thống trước tiên nạp bản đồ đã được xây dựng thông qua Map Server. Sau khi người vận hành thiết lập vị trí ban đầu của robot, AMCL sử dụng dữ liệu LiDAR kết hợp với bản đồ môi trường để xác định vị trí hiện tại của robot trên bản đồ. Kết quả định vị được cung cấp cho hệ thống Nav2 dưới dạng pose và cây TF, tạo cơ sở cho quá trình lập kế hoạch đường đi. Khi người dùng gửi mục tiêu điều hướng từ RViz2 hoặc giao diện Web, BT Navigator tiếp nhận yêu cầu và chuyển đến Planner Server để xây dựng quỹ đạo toàn cục bằng thuật toán Weighted A*. Quỹ đạo sau khi được tạo sẽ được gửi tới Controller Server nhằm tính toán các lệnh vận tốc phù hợp với trạng thái hiện tại của robot. Đồng thời, Local Costmap liên tục cập nhật thông tin vật cản từ dữ liệu LiDAR để hỗ trợ quá trình tránh va chạm trong khi di chuyển. Các lệnh vận tốc sau đó được làm mượt bởi Velocity Smoother trước khi truyền đến ESP32-S3 để điều khiển động cơ. Trong suốt quá trình vận hành, dữ liệu encoder tiếp tục được gửi ngược về hệ thống để cập nhật odometry và hỗ trợ các chức năng định vị, điều hướng. Chu trình này được lặp lại liên tục cho đến khi robot đến vị trí mục tiêu hoặc nhận lệnh dừng từ người vận hành, qua đó bảo đảm robot có thể di chuyển tự động với độ ổn định và độ chính xác cao trong môi trường thực tế.

CHƯƠNG 4

KẾT QUẢ VÀ ĐÁNH GIÁ MÔ HÌNH

4.1 KẾT QUẢ MÔ HÌNH THI CÔNG



a. Robot tại góc nghiêng 3/4

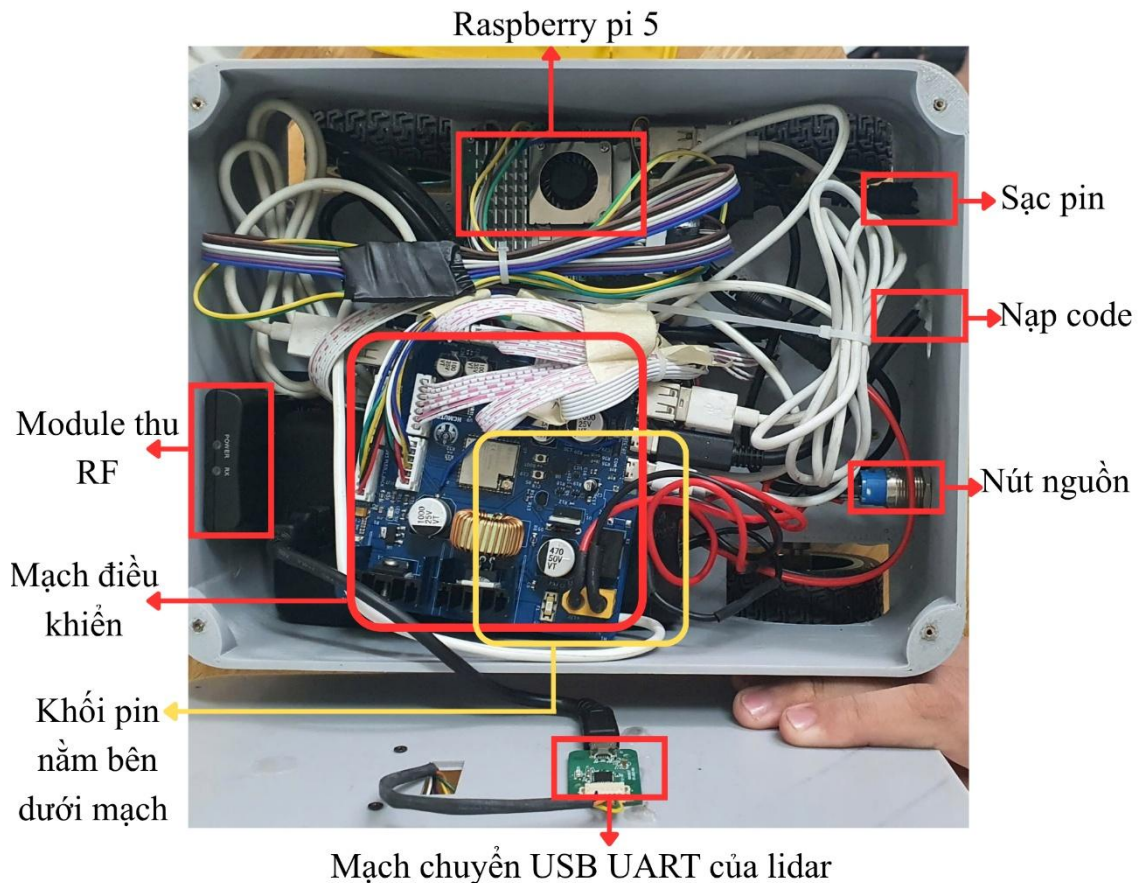


b. Mặt bên của robot

Hình 4.1 Kết quả mô hình thi công

Hình 4.1 thể hiện kết quả hoàn thiện mô hình thi công robot. Bao gồm kết quả mạch điều khiển sau khi đã đặt gia công và hàn linh kiện, robot sau khi đã lắp đặt các động cơ và bánh xe, ngoại quan tổng thể robot sau khi đã bố trí xong các thiết bị và lắp đặt hoàn thiện. Mạch điều khiển sau khi đã hoàn thiện tiến hành kiểm thử hoạt động, sau khi đã hoạt động ổn định thì thực hiện bố trí lắp đặt các thành phần linh kiện khác vào bên trong robot. Bốn bánh xe được bố trí cân xứng bên dưới khung robot, sử dụng các tấm ốp để cố định động cơ. Cảm biến lidar được đặt phía trên cùng, không bị che chắn.

Hình 4.2 thể hiện vị trí bố trí các linh kiện bên trong robot bao gồm: mạch điều khiển, khối pin, mạch thu RF, mạch chuyển đổi USB UART, HUB, các vị trí sạc pin, nạp code, nút nguồn. Khi hoạt động sẽ thực hiện điều khiển và làm việc với raspberry pi 5 thông qua SSH để kết nối với máy tính từ xa. Raspberry sẽ thực hiện nhận các lệnh từ xa để điều khiển robot hoạt động như mong muốn.

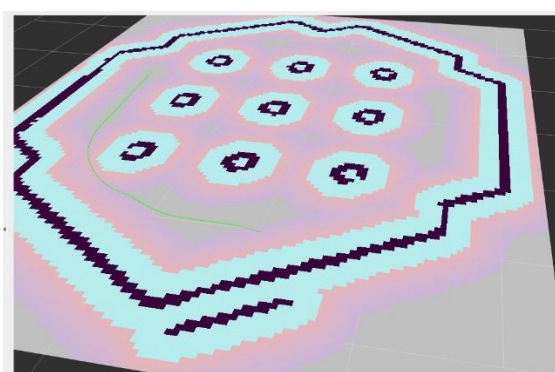


Hình 4.2 Vị trí bố trí linh kiện bên trong robot

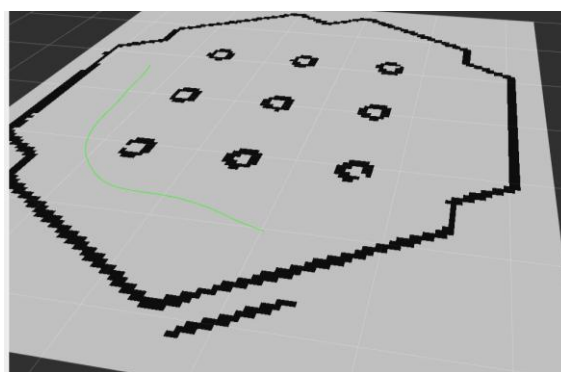
4.2 KẾT QUẢ MÔ PHỎNG

Trước khi triển khai lên phần cứng thực tế, hệ thống phần mềm lập kế hoạch đường đi được kiểm tra trong môi trường mô phỏng sử dụng Gazebo kết hợp với ROS 2 Humble và Nav2. Trong môi trường mô phỏng, robot thực hiện thành công quy trình vẽ được quỹ đạo đường đi bằng thuật toán A* với các trọng số khác nhau: sử dụng mẫu bản đồ có sẵn của ROS2, lập kế hoạch đường đi bằng thuật toán A* có trọng số W, và lấy kết quả dữ liệu về thời gian tính toán, số node duyệt, chiều dài quãng đường để lập ra được quỹ đạo đường đi. Kết quả mô phỏng xác nhận tính đúng đắn của thuật toán A* có trọng số W trước khi tiến hành thực nghiệm trên robot thực tế.

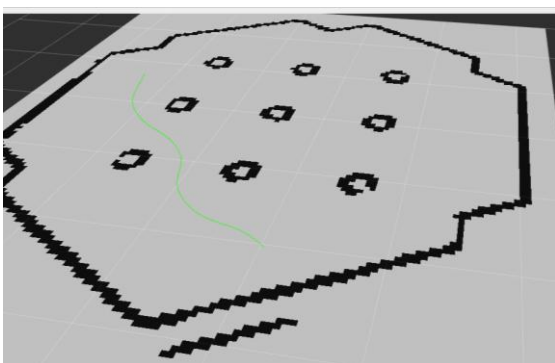
Dưới đây là hình ảnh kết quả mô phỏng quỹ đạo đường đi được lập ra với các trọng số W khác nhau, lần lượt là $W=0$, $W=1$, $W=1.5$, $W=2.0$ với điểm xuất phát giống nhau và 2 đích đến khác nhau lần 1 lượt gọi là Goal1 và Goal 2. Xem trong các hình 4.3.



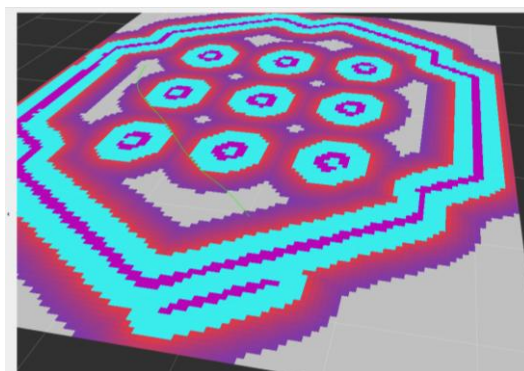
a. Quỹ đạo với $W = 0$



b. Quỹ đạo với $W = 1$



c. Quỹ đạo với $W = 1.5$



d. Quỹ đạo với $W = 2$

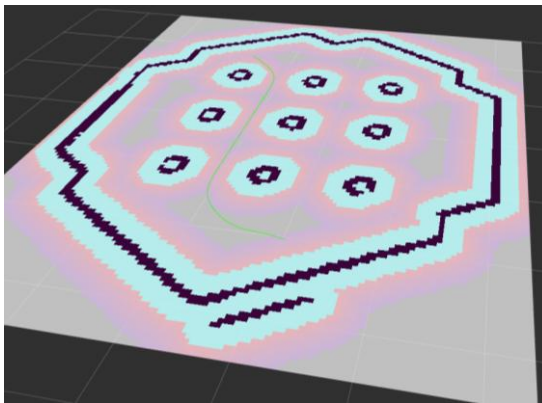
Hình 4.3 Các quỹ đạo tới Goal 1 khi thay đổi W

Các kết quả mô phỏng cho thấy thuật toán A* có trọng số W đều xác định được quỹ đạo khả thi từ vị trí xuất phát đến Goal 1 trong các trường hợp khảo sát với các giá trị trọng số khác nhau. Khi giá trị W tăng, quá trình tìm kiếm có xu hướng tập trung nhiều hơn về phía mục tiêu, từ đó làm giảm số lượng node cần mở rộng và rút ngắn thời gian xử lý. Tuy nhiên, việc tăng W quá lớn có thể làm tăng mức độ ảnh hưởng của thành phần heuristic trong hàm đánh giá, khiến thuật toán ưu tiên hướng tìm kiếm gần mục tiêu hơn và có khả năng làm giảm tính tối ưu của quỹ đạo trong một số trường hợp.

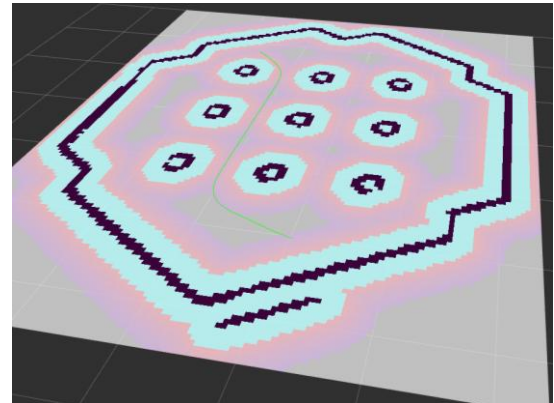
Bảng 4.1 Số node và thời gian lập kế hoạch ở Goal 1

Trọng số	Số node	Thời gian
$W = 0$	3991	0.00365
$W = 1$	1823	0.00283
$W = 1.5$	1198	0.00286
$W = 2$	79	0.00117

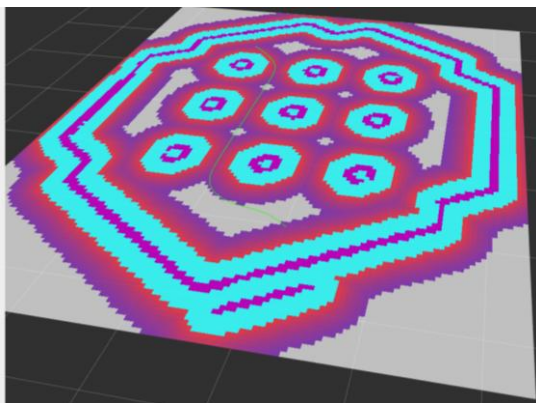
Dựa trên kết quả mô phỏng được trình bày trong Bảng 4.1, có thể thấy hệ số trọng số W có ảnh hưởng rõ rệt đến hiệu quả tìm kiếm của thuật toán A^* . Trường hợp $W=0$ tương ứng với việc loại bỏ hoàn toàn thành phần heuristic, khi đó thuật toán trở về dạng tìm kiếm theo chi phí đồng nhất (Uniform Cost Search). Kết quả cho thấy đây là trường hợp có số lượng node mở rộng lớn nhất, đạt 3991 node, cùng với thời gian xử lý 0,00365 s. Khi tăng giá trị W lên 1, số node mở rộng giảm xuống còn 1823 node và thời gian xử lý giảm còn 0,00283 s. Tiếp tục tăng W lên 1,5, số node tiếp tục giảm xuống 1198 node, tuy nhiên thời gian xử lý duy trì ở mức tương đương 0,00286 s. Đối với trường hợp $W=2$, hiệu quả tính toán được cải thiện rõ rệt, với số node giảm mạnh xuống còn 79 node và thời gian xử lý giảm xuống 0,00117 s. Các kết quả trên cho thấy việc tăng hệ số trọng số W làm gia tăng mức độ ưu tiên của thành phần heuristic trong hàm đánh giá, từ đó thu hẹp không gian tìm kiếm và giảm số lượng trạng thái cần mở rộng. Điều này giúp quá trình lập kế hoạch đường đi trở nên hiệu quả hơn về mặt tính toán, trong khi vẫn đảm bảo khả năng tìm được quỹ đạo hợp lệ trong môi trường mô phỏng.



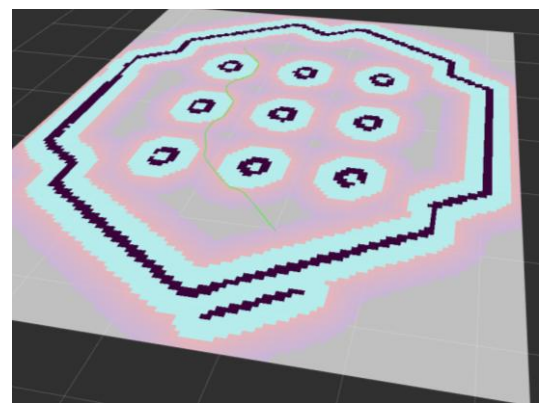
a. Quỹ đạo với $W = 0$



b. Quỹ đạo với $W = 1$



c. Quỹ đạo với $W = 1.5$



d. Quỹ đạo với $W = 2$

Hình 4.4 Các quỹ đạo tới Goal 2 khi W thay đổi

Để đánh giá khả năng hoạt động của thuật toán trong các điều kiện tìm kiếm khác nhau, mô phỏng tiếp tục được thực hiện với vị trí đích thứ hai, ký hiệu là Goal 2. Các giá trị trọng số W được giữ nguyên nhằm so sánh sự thay đổi về hiệu quả tìm kiếm khi robot thực hiện điều hướng đến một mục tiêu khác. Hình 4.4 thể hiện quỹ đạo di chuyển của robot khi thay đổi giá trị trọng số W đối với trường hợp Goal 2.

Các kết quả mô phỏng đối với Goal 2 cho thấy thuật toán A^* có trọng số W vẫn xác định được quỹ đạo khả thi từ vị trí xuất phát đến vị trí đích trong các trường hợp thay đổi hệ số trọng số. Mặc dù vị trí mục tiêu được thay đổi so với Goal 1, robot vẫn tìm được đường đi hợp lệ và duy trì khả năng tránh các vùng vật cản trên bản đồ. Kết quả này cho thấy việc điều chỉnh trọng số W không làm ảnh hưởng đến khả năng tìm kiếm đường đi trong môi trường mô phỏng đã xây dựng. Đồng thời, thuật toán có khả năng duy trì sự ổn định khi thực hiện lập kế hoạch đường đi với các kịch bản điều hướng khác nhau, đáp ứng yêu cầu của hệ thống robot tự hành.

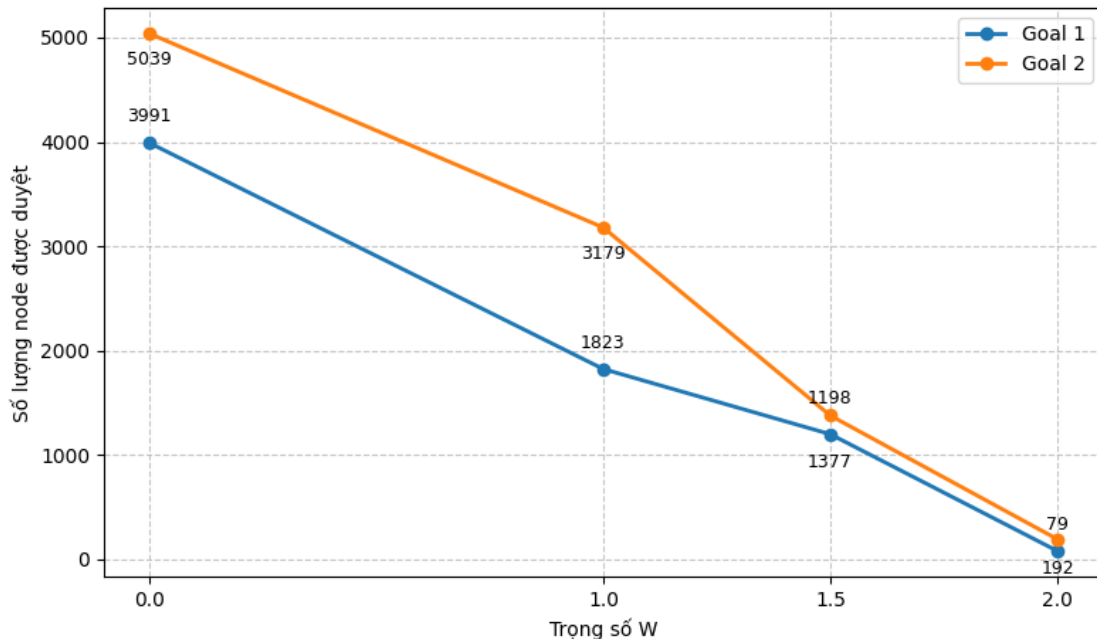
Bảng 4.2 Số node và thời gian lập kế hoạch ở Goal 2

Trọng số	Số node	Thời gian
$W = 0$	5039	0.00324
$W = 1$	3179	0.00286
$W = 1.5$	1377	0.00164
$W = 2$	192	0.00055

Kết quả mô phỏng được trình bày trong bảng, có thể nhận thấy hệ số trọng số W ảnh hưởng rõ rệt đến hiệu quả tìm kiếm của thuật toán A^* . Trường hợp $W=0$ tương ứng với việc loại bỏ thành phần heuristic trong hàm đánh giá, khi đó thuật toán tương đương với tìm kiếm theo chi phí đồng nhất (Uniform Cost Search). Đây cũng là cấu hình có hiệu suất thấp nhất, với số lượng node mở rộng đạt 5039 node và thời gian xử lý 0,00324 s. Khi W được tăng lên 1, số lượng node giảm xuống còn 3179 node, trong khi thời gian xử lý giảm nhẹ còn 0,00286 s. Tiếp tục tăng W lên 1,5, số node mở rộng giảm đáng kể xuống 1377 node, đồng thời thời gian xử lý còn 0,00164 s. Với $W=2$, hiệu quả tính toán được cải thiện rõ rệt, khi số node chỉ còn 192 node và thời gian xử lý đạt 0,00055 s. Các kết quả trên cho thấy việc gia tăng hệ số trọng số W làm tăng mức độ ảnh hưởng của thành phần heuristic trong hàm đánh giá, từ đó thu hẹp không gian tìm kiếm và giảm số trạng thái cần mở rộng. Nhờ đó, thuật toán định hướng quá trình tìm kiếm hiệu quả hơn về phía mục tiêu, cải thiện đáng kể hiệu suất tính toán trong quá trình lập kế

hoạch đường đi, đồng thời vẫn đảm bảo khả năng tìm được quỹ đạo khả thi trong môi trường mô phỏng.

Để đánh giá ảnh hưởng của trọng số heuristic đến hiệu quả tìm kiếm của thuật toán A*, các kết quả mô phỏng được trực quan hóa thông qua hình 4.5 và hình 4.6. Hai đồ thị lần lượt biểu diễn sự thay đổi của số lượng node được duyệt và thời gian lập kế hoạch theo các giá trị trọng số W khác nhau.

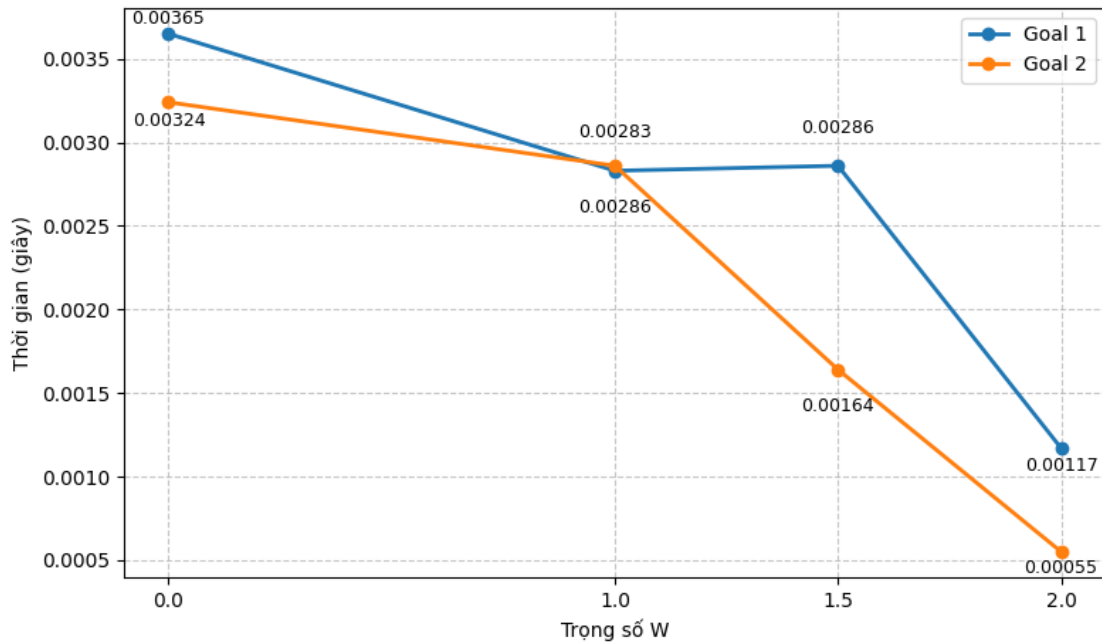


Hình 4.5 Biểu đồ số lượng node được duyệt theo trọng số W

Từ hình 4.5 có thể quan sát thấy mối quan hệ nghịch rõ rệt giữa trọng số W và số lượng node được mở rộng trong quá trình tìm kiếm. Đối với cả hai mục tiêu khảo sát, số node được duyệt đều giảm mạnh khi giá trị W tăng từ 0 lên 2. Xu hướng này phản ánh vai trò ngày càng nổi bật của thành phần heuristic trong hàm đánh giá, giúp thuật toán ưu tiên mở rộng các trạng thái có tiềm năng dẫn đến đích thay vì phân bổ tài nguyên tính toán cho các nhánh ít triển vọng hơn. Do đó, không gian tìm kiếm được thu hẹp đáng kể, góp phần nâng cao hiệu quả tìm kiếm tổng thể. Bên cạnh đó, Goal 2 luôn yêu cầu số lượng node được duyệt lớn hơn Goal 1 tại cùng một giá trị trọng số, cho thấy bài toán tương ứng có độ phức tạp cao hơn và đòi hỏi nhiều bước khám phá trạng thái hơn để tiếp cận mục tiêu.

Tại biểu đồ 4.6, xu hướng tương tự cũng được ghi nhận đối với thời gian lập kế hoạch. Khi trọng số W tăng, thời gian thực thi giảm đáng kể đối với cả hai goal. Sự suy giảm này có mối tương quan trực tiếp với việc giảm số lượng node được mở rộng, cho thấy chi phí tính toán của thuật toán phụ thuộc chủ yếu vào quy mô không gian trạng thái cần khảo sát. Trong phạm vi khảo sát, $W = 2$ cho thời gian thực thi thấp nhất ở cả hai mục tiêu, phản ánh khả năng định hướng tìm

kiểm hiệu quả của heuristic khi được gán mức ưu tiên cao hơn trong hàm đánh giá.



Hình 4.6 Biểu đồ thời gian lập kế hoạch theo trọng số W

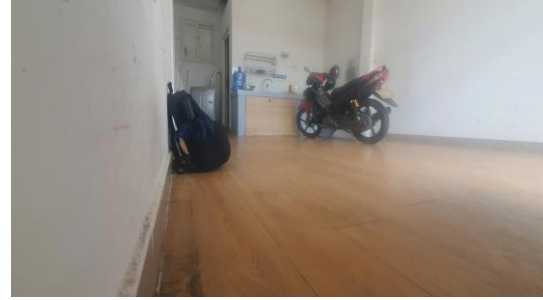
Tổng hợp các kết quả mô phỏng cho thấy việc gia tăng trọng số heuristic giúp cải thiện đáng kể hiệu suất tìm kiếm của thuật toán A*. Khi ảnh hưởng của heuristic được tăng cường, thuật toán có xu hướng tập trung vào các hướng đi tiềm năng hơn, từ đó làm giảm số lượng trạng thái cần mở rộng cũng như thời gian xử lý tương ứng. Mặc dù $W = 2$ đạt hiệu quả cao nhất xét trên các chỉ tiêu về số node mở rộng và thời gian lập kế hoạch, giá trị $W = 1.5$ được đánh giá là phù hợp hơn cho hệ thống xe tự hành do vẫn duy trì hiệu suất tính toán tốt đồng thời đảm bảo chất lượng quỹ đạo và tính ổn định trong quá trình di chuyển. Vì vậy, $W = 1.5$ được lựa chọn cho các thí nghiệm và đánh giá tiếp theo.

4.3 KẾT QUẢ VẼ VÀ LƯU BẢN ĐỒ TRÊN RVIZ 2

Trong quá trình thực nghiệm hệ thống robot tự hành, việc xây dựng và trực quan hóa bản đồ môi trường đóng vai trò quan trọng nhằm đánh giá khả năng nhận thức không gian của robot. Phần này trình bày kết quả thu được khi thực hiện quá trình vẽ bản đồ (mapping) trên phần mềm RViz trong ROS 2. Thông qua dữ liệu từ cảm biến LiDAR và hệ thống định vị, bản đồ được tái hiện theo thời gian thực, cho phép quan sát mức độ chính xác của quá trình quét môi trường, khả năng phát hiện vật cản cũng như sự ổn định của hệ thống trong quá trình di chuyển, kết quả vẽ bản đồ thực tế sẽ được thể hiện sau đây.



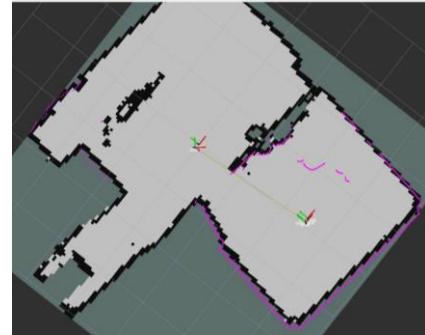
a. Góc bên phải phòng khách



b. Góc bên trái phòng khách



c. Phòng ngủ



d. Bản đồ được vẽ trên Rviz2

Hình 4.7 Hình ảnh bản đồ thực tế và sau khi được vẽ hoàn chỉnh

Các Hình 4.7a, 4.7b, 4.7c trình bày môi trường thực nghiệm dưới dạng ảnh chụp thực tế của khu vực khảo sát, trong khi Hình 4.7d thể hiện kết quả xây dựng bản đồ trên RViz2. Kết quả cho thấy môi trường đã được tái tạo với mức độ hoàn chỉnh cao, phản ánh tương đối chính xác cấu trúc không gian của khu vực thực nghiệm. Các thành phần cố định như tường và vật cản được cảm biến LiDAR ghi nhận đầy đủ, hình thành nên các biên dạng liên tục và có tính nhất quán trên bản đồ. Trong suốt quá trình thu thập dữ liệu, robot duy trì khả năng định vị ổn định, cho phép hệ thống tích lũy đủ thông tin để xây dựng bản đồ mà không xuất hiện các sai lệch đáng kể hoặc hiện tượng mất định vị kéo dài. Điều này cho thấy sự phối hợp hiệu quả giữa cảm biến LiDAR và thuật toán SLAM trong việc ước lượng đồng thời vị trí robot và cấu trúc môi trường xung quanh. Nhìn chung, kết quả xây dựng bản đồ đáp ứng các yêu cầu đặt ra đối với hệ thống, đảm bảo mức độ chính xác cần thiết cho các tác vụ định vị và lập kế hoạch đường đi ở các giai đoạn tiếp theo. Bản đồ thu được không chỉ cung cấp thông tin hình học của môi trường mà còn đóng vai trò là cơ sở quan trọng để nâng cao độ tin cậy của toàn bộ hệ thống điều hướng tự hành.

4.4 KẾT QUẢ THỰC NGHIỆM KHẢ NĂNG DI CHUYỂN ĐẾN ĐIỂM ĐÍCH CỦA ROBOT

Trong phần thí nghiệm tiếp theo, hệ thống robot tự hành được kiểm tra khả năng di chuyển đến nhiều vị trí đích khác nhau nhằm đánh giá độ ổn định và tính lặp lại của thuật toán lập kế hoạch đường đi, vị trí 3 goal trong thực tế xem ở hình 4.8. Cụ thể, robot được yêu cầu di chuyển đến ba vị trí đích khác nhau (Goal 1, Goal 2 và Goal 3), trong đó mỗi vị trí được thực hiện ba lần thử nghiệm liên tiếp với cùng một giá trị trọng số $W=1,5$. Trong mỗi lần thử nghiệm, các thông số quan trọng như thời gian di chuyển và sai số vị trí khi đến đích được ghi nhận và phân tích. Kết quả thu được được sử dụng để đánh giá độ chính xác, tính ổn định và khả năng lặp lại của hệ thống trong cùng điều kiện vận hành.

Để đánh giá độ chính xác và tính ổn định của hệ thống điều hướng, robot được thực hiện di chuyển từ vị trí xuất phát (Start) đến từng vị trí đích (Goal) trong 3 lần thử nghiệm độc lập. Sau mỗi lần thực hiện, sai số vị trí tại điểm đích và thời gian hoàn thành hành trình được ghi nhận và tổng hợp để phục vụ quá trình đánh giá hiệu năng của hệ thống.



a. Vị trí bắt đầu



b. Vị trí Goal 1



c. Vị trí Goal 2



d. Vị trí Goal 3

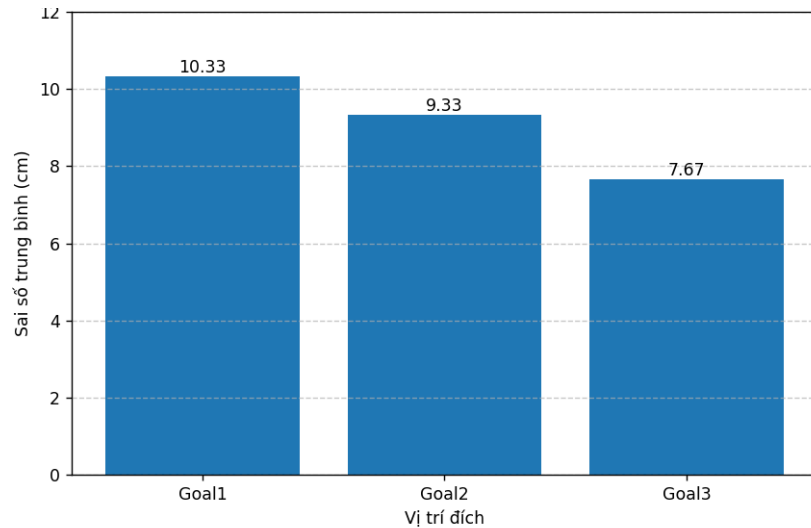
Hình 4.8 Vị trí bắt đầu và 3 Goal được sử dụng để thực nghiệm thực trong thực tế

Bảng 4.3 Kết quả đo đạt khả năng di chuyển tới 3 goal

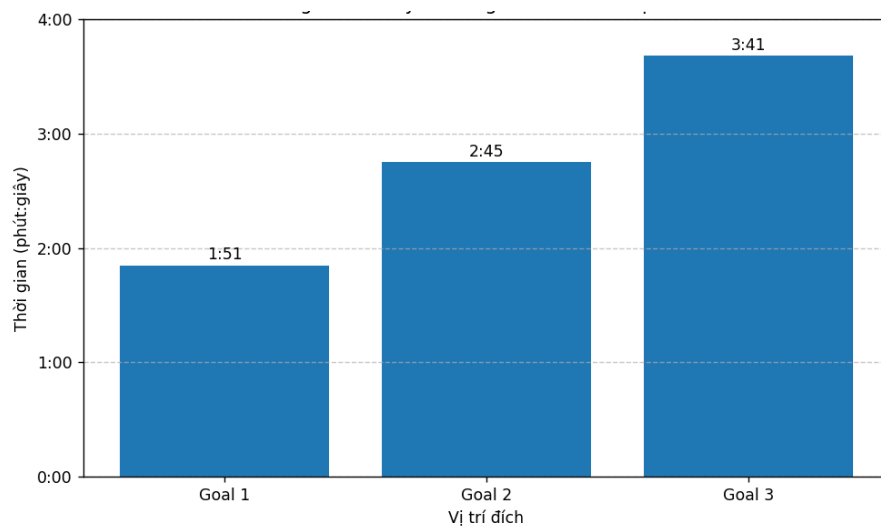
Goal	Số lần	Sai số (cm)	Thời gian
1	Lần 1	12	1'35s
	Lần 2	10	1'58s
	Lần 3	9	1'59s
2	Lần 1	7	3'13s
	Lần 2	12	2'43s
	Lần 3	9	2'20s
3	Lần 1	11	3'11s
	Lần 2	3	3'15s
	Lần 3	9	4'36s

Bảng 4.3 và các Hình 4.9, 4.10 cho thấy robot có khả năng di chuyển thành công đến cả ba vị trí đích được thiết lập trong môi trường thực nghiệm. Sai số vị trí ghi nhận được dao động trong khoảng từ 3 cm đến 12 cm, trong khi hệ thống được thiết lập với ngưỡng sai số chấp nhận khi đến đích là 15 cm. Do đó, toàn bộ các lần thử nghiệm đều đạt yêu cầu về độ chính xác định vị. Kết quả thống kê cho thấy sai số trung bình tại các vị trí Goal 1, Goal 2 và Goal 3 lần lượt là 10,33 cm, 9,33 cm và 7,67 cm. Các giá trị này đều nhỏ hơn ngưỡng cho phép, chứng tỏ hệ thống duy trì được độ chính xác tương đối tốt trong quá trình điều hướng. Trong đó, Goal 3 có sai số trung bình nhỏ nhất, tiếp theo là Goal 2 và Goal 1, tuy nhiên sự chênh lệch giữa các vị trí đích không đáng kể.

Về thời gian thực hiện, Goal 1 có thời gian trung bình ngắn nhất, khoảng 1 phút 51 giây, trong khi Goal 2 và Goal 3 có thời gian trung bình lần lượt là 2 phút 45 giây và 3 phút 41 giây. Kết quả này cho thấy thời gian di chuyển có xu hướng tăng theo khoảng cách đến vị trí đích và mức độ phức tạp của quỹ đạo di chuyển. Đặc biệt, tại Goal 3, thời gian thực hiện giữa các lần thử nghiệm có sự dao động tương đối lớn. Nguyên nhân có thể xuất phát từ sai số định vị tích lũy, quá trình tránh vật cản hoặc các yếu tố môi trường ảnh hưởng đến quá trình điều hướng của robot.



Hình 4.9 Biểu đồ sai số trung bình của robot tại các vị trí đích



Hình 4.10 Biểu đồ thời gian di chuyển trung bình của robot tại các vị trí đích

Nhìn chung, mặc dù vẫn tồn tại một số dao động về sai số vị trí và thời gian thực hiện giữa các lần thử nghiệm, hệ thống vẫn duy trì được khả năng điều hướng ổn định và có tính lặp lại tương đối tốt trong môi trường thực tế. Kết quả thực nghiệm cho thấy robot có thể di chuyển đến các vị trí đích với sai số nhỏ hơn ngưỡng cho phép, đáp ứng yêu cầu đặt ra đối với hệ thống robot tự hành. Đối với chức năng lập kế hoạch đường đi, kết quả thực nghiệm cho thấy thuật toán A* được tích hợp trong hệ thống Nav2 đã hỗ trợ robot xây dựng quỹ đạo khả thi và di chuyển thành công đến các vị trí đích. Điều này chứng tỏ phương pháp lập kế hoạch đường đi được lựa chọn phù hợp với yêu cầu của hệ thống

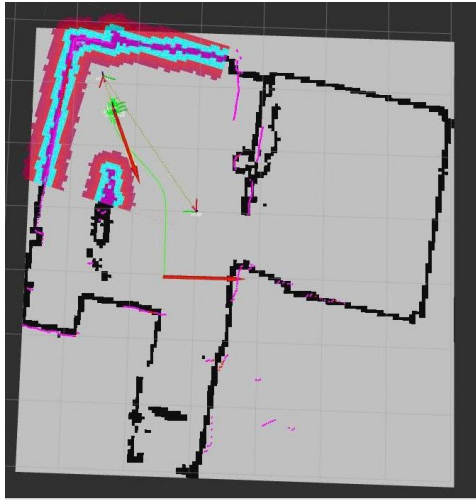
robot tự hành trong môi trường thực nghiệm, góp phần nâng cao độ chính xác và tính ổn định của quá trình điều hướng.

4.5 KẾT QUẢ THỰC NGHIỆM KHI THAY ĐỔI TRỌNG SỐ W

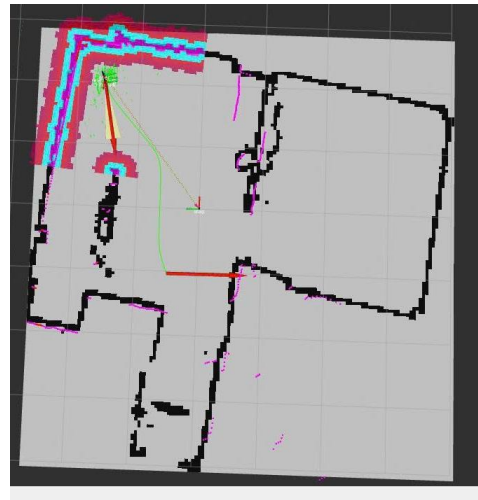
Trong phần thực nghiệm này, hệ thống robot được khảo sát nhằm đánh giá ảnh hưởng của trọng số W trong thuật toán A* đến hiệu quả lập kế hoạch đường đi và chất lượng quỹ đạo di chuyển. Robot được thiết lập xuất phát từ cùng một vị trí ban đầu (Start) và lần lượt di chuyển đến ba vị trí đích (Goal) trong cùng một môi trường thực nghiệm. Thí nghiệm được thực hiện với nhiều giá trị trọng số W khác nhau. Đối với mỗi trường hợp, hệ thống tiến hành lập kế hoạch đường đi và ghi nhận các thông số gồm số lượng node được mở rộng trong quá trình tìm kiếm, thời gian lập kế hoạch ở lần tính toán đầu tiên và quỹ đạo thu được. Các chỉ số này được sử dụng để đánh giá mức độ ảnh hưởng của trọng số W đến hiệu suất tìm kiếm cũng như chất lượng đường đi của thuật toán. Thông qua việc so sánh các kết quả thu được, có thể phân tích mối quan hệ giữa trọng số heuristic, chi phí tính toán và đặc tính của quỹ đạo, từ đó xác định giá trị W phù hợp cho hệ thống robot tự hành trong môi trường thực nghiệm.

4.5.1 Thực nghiệm tại vị trí Goal 1

Hình 4.1 là các quỹ đạo được vẽ ra tới Goal 1 khi W thay đổi. Kết quả thực nghiệm cho thấy khi $W = 0$ và $W = 1$, quỹ đạo di chuyển đến Goal 1 xuất hiện nhiều đoạn cong và thay đổi hướng hơn so với các trường hợp còn lại. Điều này làm cho lộ trình trở nên phức tạp và kém trực tiếp trong quá trình tiếp cận vị trí đích. Khi trọng số tăng lên $W = 1.5$ và $W = 2$, quỹ đạo có xu hướng bám sát hơn đường nối giữa vị trí xuất phát và vị trí đích, đồng thời giảm số lần chuyển hướng. Kết quả trên cho thấy việc gia tăng trọng số W làm tăng mức độ ảnh hưởng của thành phần heuristic trong hàm đánh giá, từ đó giúp thuật toán tập trung hiệu quả hơn vào các trạng thái có khả năng dẫn đến mục tiêu. Nhờ vậy, không gian tìm kiếm được thu hẹp, chi phí tính toán giảm xuống và quỹ đạo thu được trở nên đơn giản hơn. Tuy nhiên, nếu trọng số W quá lớn, thuật toán có thể ưu tiên tốc độ tìm kiếm hơn chất lượng đường đi. Do đó, việc lựa chọn giá trị W phù hợp là cần thiết nhằm đảm bảo sự cân bằng giữa hiệu suất tính toán và chất lượng quỹ đạo của hệ thống robot tự hành. Để đánh giá định lượng ảnh hưởng của trọng số W, các kết quả về số lượng node được duyệt, thời gian lập kế hoạch và hiệu quả di chuyển của robot được trình bày trong Bảng 4.4.



a. Quỹ đạo tới Goal 1 khi $W = 0$



b. Quỹ đạo tới Goal 1 khi $W = 11$



c. Quỹ đạo tới Goal 1 khi $W = 1.5$



d. Quỹ đạo tới Goal 1 khi $W = 2$

Hình 4.11 Các quỹ đạo di chuyển tới Goal 1 khi W thay đổi

Bảng 4.4 Số node, thời gian vẽ, thời gian di chuyển và chiều và chiều dài quãng đường khi thay đổi W tới Goal 1

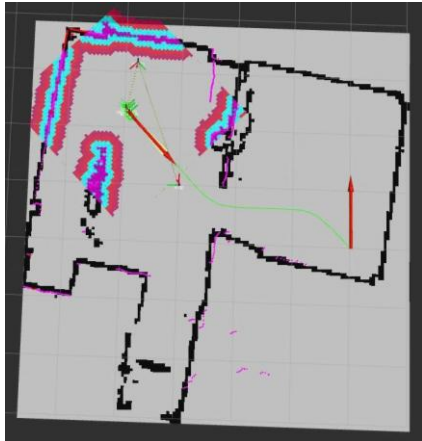
Trọng số	Số node	Thời gian vẽ quỹ đạo	Thời gian di chuyển	Chiều dài quãng đường(m)
$W = 0$	3224	0.004048s	3'00s	3.418
$W = 1$	1053	0.000989	2'40s	3.48
$W = 1.5$	63	0.000723	1'35s	3.318
$W = 2$	63	0,000768	2'01s	3.39

Kết quả trong Bảng 4.4 cho thấy hệ số trọng số W có ảnh hưởng đáng kể đến hiệu quả hoạt động của thuật toán lập kế hoạch đường đi. Khi giá trị W tăng từ 0 lên 1,5, số lượng node được duyệt giảm mạnh từ 3224 xuống còn 63 node. Đồng thời, thời gian lập kế hoạch đường đi cũng giảm từ 0,004048 s xuống còn 0,000723 s. Sự cải thiện này cho thấy việc tăng trọng số giúp thuật toán tập trung hơn vào hướng tiếp cận mục tiêu, từ đó hạn chế việc mở rộng các node không cần thiết trong không gian tìm kiếm và nâng cao hiệu quả tính toán. Xét về hiệu quả điều hướng, trường hợp $W = 1,5$ đạt kết quả nổi bật nhất với thời gian di chuyển ngắn nhất là 1 phút 35 giây và chiều dài quãng đường nhỏ nhất là 3,318 m. Kết quả này cho thấy giá trị $W = 1,5$ tạo được sự cân bằng hợp lý giữa tốc độ tìm kiếm và chất lượng quỹ đạo, cho phép robot di chuyển đến vị trí đích một cách hiệu quả.

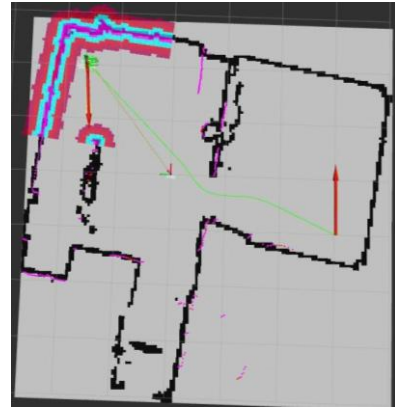
Khi tăng trọng số lên $W = 2$, số lượng node được duyệt vẫn duy trì ở mức thấp tương đương trường hợp $W = 1,5$. Tuy nhiên, thời gian di chuyển tăng lên 2 phút 01 giây và chiều dài quãng đường cũng lớn hơn. Điều này cho thấy việc tiếp tục gia tăng trọng số không mang lại sự cải thiện tương ứng về hiệu quả điều hướng. Mặc dù chi phí tìm kiếm được duy trì ở mức thấp, chất lượng quỹ đạo và thời gian thực hiện nhiệm vụ không được cải thiện so với trường hợp $W = 1,5$. Ngược lại, tại các trường hợp $W = 0$ và $W = 1$, số lượng node được duyệt lớn hơn đáng kể, dẫn đến thời gian lập kế hoạch và thời gian hoàn thành nhiệm vụ đều tăng. Kết quả này cho thấy trọng số nhỏ khiến thuật toán mở rộng nhiều node hơn trong quá trình tìm kiếm, làm giảm hiệu quả tính toán và kéo dài thời gian thực hiện nhiệm vụ.

Từ các kết quả thực nghiệm có thể nhận thấy rằng việc lựa chọn giá trị W phù hợp đóng vai trò quan trọng trong việc nâng cao hiệu quả của thuật toán lập kế hoạch đường đi. Trong phạm vi thực nghiệm của đề tài, giá trị $W = 1,5$ cho thấy hiệu quả tốt nhất, bảo đảm sự cân bằng giữa chi phí tính toán, thời gian thực hiện nhiệm vụ và chất lượng quỹ đạo của robot tự hành. Sau khi đánh giá ảnh hưởng của trọng số W đối với quá trình lập kế hoạch đường đi đến Goal 1, thí nghiệm tiếp tục được thực hiện với Goal 2 nhằm kiểm tra tính nhất quán của thuật toán trong điều kiện quãng đường và quỹ đạo di chuyển khác nhau.

4.5.2 Thực nghiệm tại vị trí Goal 2



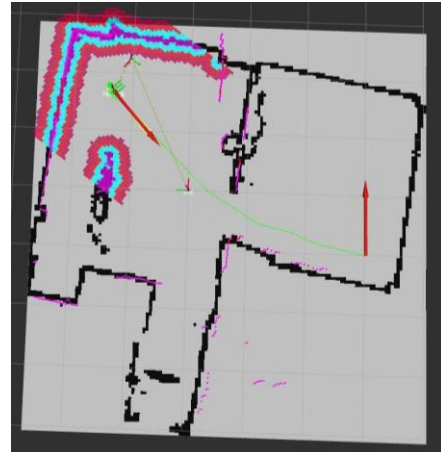
a. Quỹ đạo tới Goal 2 khi $W = 0$



b. Quỹ đạo tới Goal 2 khi $W = 1$



c. Quỹ đạo tới Goal 2 khi $W = 1,5$



d. Quỹ đạo tới Goal 2 khi $W = 2$

Hình 4.12 Các quỹ đạo tới Goal 2 khi W thay đổi

Tương tự kết quả tại Goal 1, khi $W = 0$ và $W = 1$, quỹ đạo di chuyển đến Goal 2 xuất hiện nhiều đoạn cong và thay đổi hướng hơn trong quá trình tiếp cận vị trí đích. Điều này làm cho quỹ đạo trở nên phức tạp và kém trực tiếp hơn. Khi tăng trọng số lên $W = 1.5$ và $W = 2$, quỹ đạo có xu hướng bám sát hơn đường nối giữa vị trí xuất phát và vị trí đích, đồng thời giảm số lần chuyển hướng trong quá trình di chuyển. Kết quả này cho thấy việc gia tăng trọng số W làm tăng mức độ ảnh hưởng của thành phần heuristic trong hàm đánh giá, giúp thuật toán tập trung hiệu quả hơn vào các trạng thái có khả năng dẫn đến mục tiêu. Nhờ đó, phạm vi tìm kiếm được thu hẹp và quỹ đạo thu được trở nên đơn giản hơn, góp phần nâng cao hiệu quả điều hướng của robot.

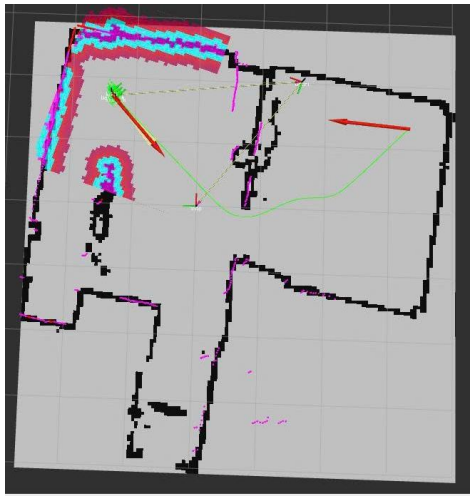
Bảng 4.5 Số node, thời gian vẽ, thời gian di chuyển và chiều và chiều dài quãng đường khi thay đổi W tới Goal 2

Trọng số	Số node	Thời gian vẽ quỹ đạo (giây)	Thời gian di chuyển	Chiều dài quãng đường(m)
W = 0	6101	0.003953	3'42s	5.88
W = 1	2196	0.003231	3'33s	5.76
W = 1.5	137	0.001003	3'13s	5.75
W = 2	127	0,000410	3'16s	5.72

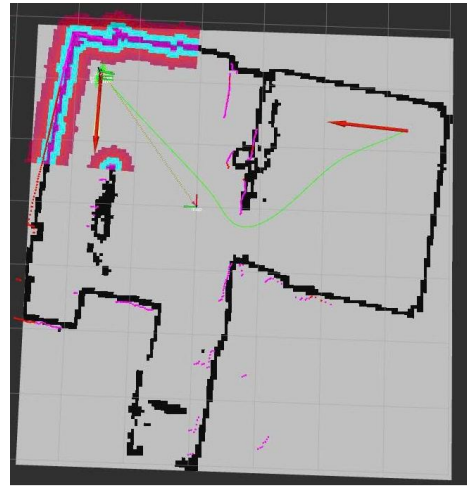
Từ bảng 4.5, thông số kết quả thực nghiệm cho thấy khi tăng trọng số W, số lượng node được duyệt giảm mạnh từ 6101 node (W = 0) xuống còn 137 node (W = 1.5) và 127 node (W = 2). Điều này cho thấy thuật toán đã thu hẹp đáng kể không gian tìm kiếm và tập trung hiệu quả hơn vào các trạng thái có khả năng dẫn đến mục tiêu, thay vì mở rộng lan tỏa trên toàn bộ bản đồ. Nhờ đó, thời gian lập kế hoạch đường đi cũng giảm đáng kể, từ 0.003953 s xuống còn 0.001003 s tại W = 1.5 và 0.000410 s tại W = 2, phản ánh sự cải thiện rõ rệt về hiệu quả tính toán. Về hiệu quả điều hướng, thời gian di chuyển có xu hướng giảm khi tăng trọng số W. Trong các trường hợp khảo sát, W = 1.5 cho kết quả tốt nhất với thời gian hoàn thành hành trình là 3 phút 13 giây. Điều này cho thấy giá trị W = 1.5 tạo ra sự cân bằng hợp lý giữa khả năng định hướng theo mục tiêu và chất lượng quỹ đạo trong môi trường thực tế. Khi tăng trọng số lên W = 2, mặc dù số lượng node được duyệt tiếp tục giảm và thời gian lập kế hoạch đạt giá trị nhỏ nhất, thời gian di chuyển lại tăng nhẹ lên 3 phút 16 giây. Kết quả này cho thấy việc tiếp tục gia tăng trọng số không mang lại thêm lợi ích đáng kể về hiệu quả điều hướng, đồng thời có thể làm giảm chất lượng quỹ đạo trong quá trình thực thi. Đối với chiều dài quãng đường, kết quả ghi nhận giá trị giảm nhẹ từ 5.88 m tại W = 0 xuống còn 5.72 m tại W = 2. Tuy nhiên, mức chênh lệch giữa các trường hợp không lớn, cho thấy ảnh hưởng của trọng số W chủ yếu thể hiện ở hiệu quả tìm kiếm và thời gian thực hiện hơn là sự thay đổi đáng kể về chiều dài quỹ đạo.

Từ các kết quả thực nghiệm có thể nhận thấy rằng việc lựa chọn trọng số W phù hợp đóng vai trò quan trọng trong việc nâng cao hiệu quả của thuật toán lập kế hoạch đường đi. Trong phạm vi thực nghiệm của đề tài, W = 1.5 cho thấy hiệu quả tổng thể tốt nhất, đảm bảo sự cân bằng giữa chi phí tính toán, thời gian thực hiện và chất lượng quỹ đạo của hệ thống robot tự hành.

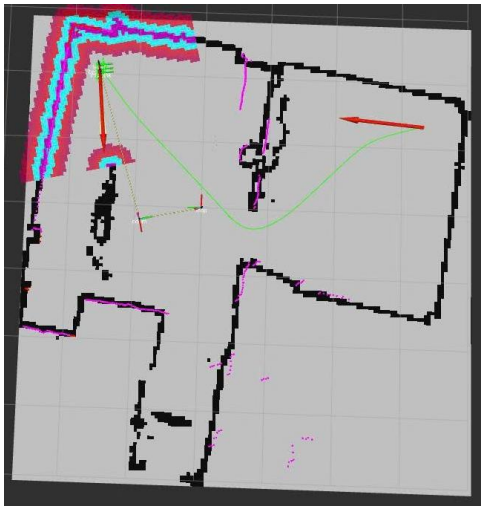
4.5.3 Thực nghiệm tại vị trí Goal 3



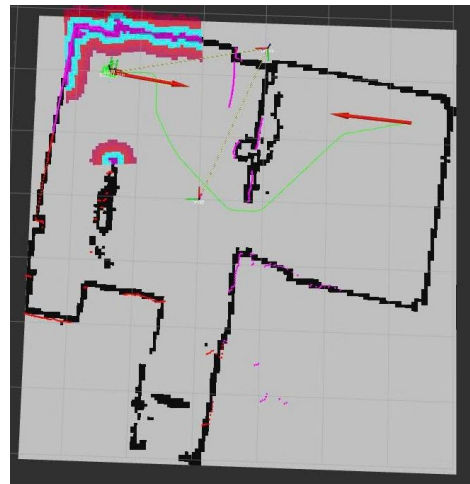
a. Quỹ đạo tới Goal 3 khi $W = 0$



b. Quỹ đạo tới Goal 3 khi $W = 1$



c. Quỹ đạo tới Goal 3 khi $W = 1.5$



d. Quỹ đạo tới Goal 3 khi $W = 2$

Hình 4.13 Các quỹ đạo tới Goal 3 khi W thay đổi

Từ hình 4.13 cho thấy kết quả thực nghiệm tại cả ba vị trí Goal cho thấy quỹ đạo di chuyển của robot có sự thay đổi khi giá trị trọng số W được điều chỉnh. Trong các trường hợp khảo sát, $W = 1.5$ luôn tạo ra quỹ đạo ổn định và có chiều dài ngắn hơn so với các giá trị trọng số còn lại. Đồng thời, quỹ đạo thu được ít xuất hiện các đoạn chuyển hướng không cần thiết, giúp robot tiếp cận mục tiêu một cách hiệu quả hơn. Kết quả này cho thấy $W = 1.5$ là giá trị trọng số phù hợp, đảm bảo sự cân bằng giữa hiệu suất tìm kiếm, chất lượng quỹ đạo và khả năng vận hành thực tế của hệ thống robot tự hành.

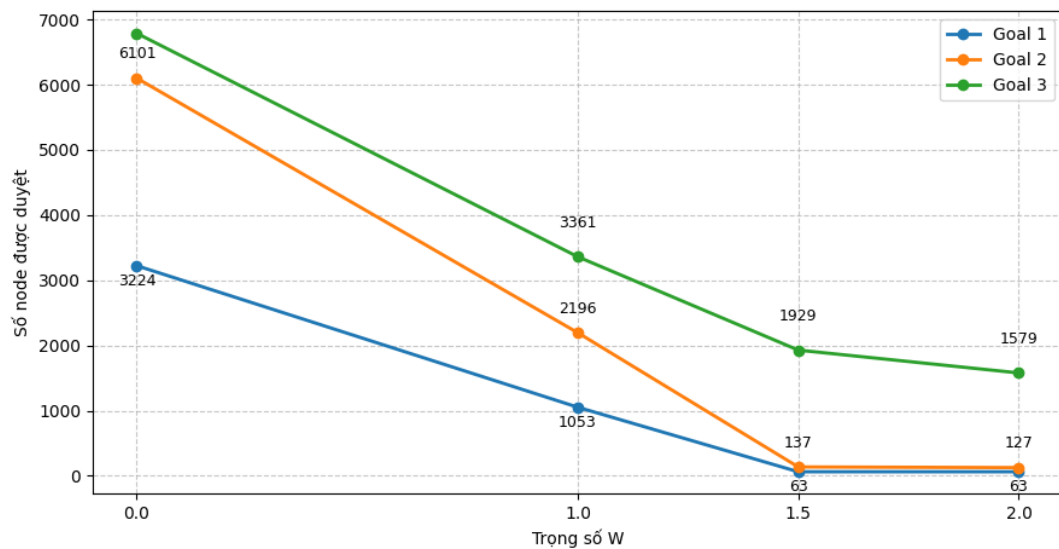
Bảng 4.6 Số node, thời gian vẽ, thời gian di chuyển và chiều và chiều dài quãng đường khi thay đổi W tới Goal 3

Trọng số	Số node	Thời gian vẽ quỹ đạo (giây)	Thời gian di chuyển	Chiều dài quãng đường(m)
W = 0	6789	0.004218	5'12s	5.88 6.35
W = 1	3361	0.003409	3'54s	5.76 6.51
W = 1.5	1929	0.001989	3'11s	5.75 6.49
W = 2	1579	0,00149	10'20s	5.72 6.84

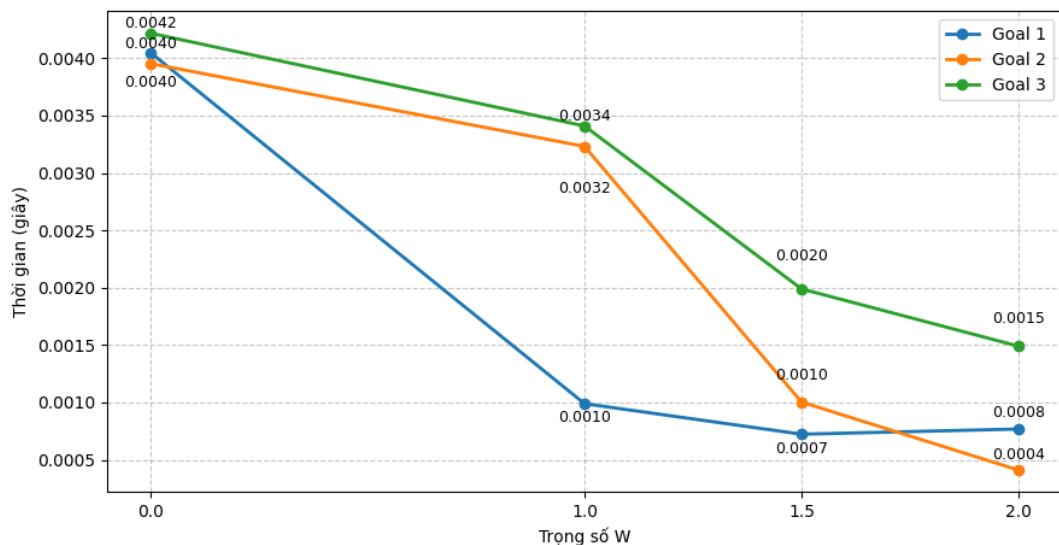
Kết quả thực nghiệm tại Goal 3 cho thấy trọng số W có ảnh hưởng đáng kể đến hiệu suất tìm kiếm và khả năng điều hướng của robot. Khi giá trị W tăng từ 0 lên 1 và 1,5, số lượng node được duyệt giảm mạnh từ 6789 node xuống còn 3361 node và 1929 node. Đồng thời, thời gian lập kế hoạch đường đi cũng giảm từ 0,004218 s xuống còn 0,001989 s. Xu hướng này cho thấy việc tăng trọng số giúp thuật toán tập trung hơn vào các trạng thái có khả năng dẫn đến mục tiêu, từ đó thu hẹp không gian tìm kiếm và nâng cao hiệu quả tính toán. Xét về hiệu quả điều hướng, trường hợp W = 1,5 cho kết quả nổi bật nhất với thời gian di chuyển ngắn nhất là 3 phút 11 giây, đồng thời duy trì số lượng node được duyệt ở mức tương đối thấp. Kết quả này cho thấy giá trị W = 1,5 tạo được sự cân bằng hợp lý giữa hiệu quả tìm kiếm và chất lượng quỹ đạo, giúp robot tiếp cận mục tiêu một cách ổn định trong môi trường thực nghiệm.

Khi tăng trọng số lên W = 2, số lượng node được duyệt tiếp tục giảm xuống còn 1579 node và thời gian lập kế hoạch đạt giá trị nhỏ nhất là 0,00149 s. Tuy nhiên, thời gian di chuyển lại tăng đáng kể lên 10 phút 20 giây, cao hơn nhiều so với các trường hợp còn lại. Điều này cho thấy việc tiếp tục gia tăng trọng số không mang lại sự cải thiện tương ứng về hiệu quả điều hướng. Mặc dù chi phí tính toán được giảm xuống, quỹ đạo thu được chưa bảo đảm hiệu quả di chuyển trong điều kiện thực nghiệm, dẫn đến thời gian hoàn thành nhiệm vụ tăng đáng kể. Đối với chiều dài quãng đường, kết quả ghi nhận sự thay đổi không lớn giữa các giá trị trọng số. Điều này cho thấy ảnh hưởng của W chủ yếu thể hiện ở hiệu quả tìm kiếm và thời gian thực hiện nhiệm vụ hơn là sự thay đổi đáng kể về chiều dài quỹ đạo.

Tổng hợp các kết quả tại Goal 3 cho thấy việc tăng trọng số W giúp cải thiện hiệu quả tính toán của thuật toán A^* . Tuy nhiên, khi giá trị W quá lớn, hiệu quả điều hướng có thể suy giảm mặc dù thời gian lập kế hoạch tiếp tục được rút ngắn. Trong phạm vi thực nghiệm của đề tài, $W = 1,5$ tiếp tục cho thấy hiệu quả tổng thể tốt nhất, bảo đảm sự cân bằng giữa chi phí tính toán, thời gian thực hiện nhiệm vụ và khả năng điều hướng của robot tự hành. Sau khi đánh giá ảnh hưởng của trọng số W đối với quá trình lập kế hoạch đường đi tại các vị trí đích Goal 1, Goal 2 và Goal 3, đề tài tiến hành tổng hợp kết quả nhằm phân tích xu hướng thay đổi của số lượng node được duyệt và thời gian lập kế hoạch theo từng giá trị trọng số. Các kết quả tổng hợp được thể hiện trong Hình 4.14 và Hình 4.15.



Hình 4.14 Biểu đồ ảnh hưởng của trọng số W đến số lượng node được duyệt



Hình 4.15 Biểu đồ ảnh hưởng của trọng số W đến thời gian lập kế hoạch

Các đồ thị trong Hình 4.33 và Hình 4.34 cho thấy khi giá trị trọng số W tăng, số lượng node được duyệt và thời gian lập kế hoạch đường đi đều có xu hướng giảm ở cả ba trường hợp Goal 1, Goal 2 và Goal 3. Điều này cho thấy việc tăng trọng số giúp thuật toán tập trung hơn vào hướng tiếp cận mục tiêu, từ đó giảm phạm vi tìm kiếm và nâng cao hiệu quả tính toán. Có thể nhận thấy mức giảm mạnh nhất xuất hiện trong khoảng từ $W = 1$ đến $W = 1,5$. Tại khoảng giá trị này, số lượng node được duyệt giảm đáng kể ở cả ba kịch bản, đồng thời thời gian lập kế hoạch cũng được rút ngắn rõ rệt. Điều này cho thấy việc tăng trọng số đến một giá trị phù hợp có thể cải thiện đáng kể hiệu quả tìm kiếm của thuật toán.

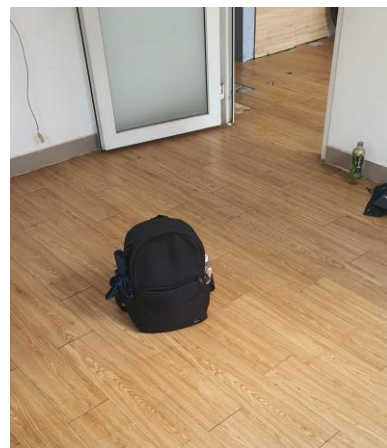
Tuy nhiên, khi tiếp tục tăng trọng số lên $W = 2$, mức cải thiện về số lượng node và thời gian lập kế hoạch không còn quá lớn so với trường hợp $W = 1,5$. Kết hợp với các kết quả thực nghiệm trên robot, điều này cho thấy việc lựa chọn trọng số quá lớn chưa chắc mang lại hiệu quả điều hướng tương ứng. Nhìn chung, giá trị $W = 1,5$ cho thấy sự cân bằng tốt giữa hiệu quả tính toán và hiệu quả điều hướng trong các trường hợp khảo sát.

4.6 KẾT QUẢ THỰC NGHIỆM KHẢ NĂNG TRÁNH VẬT CẢN

Trong thí nghiệm này, hệ thống robot được đánh giá về khả năng phát hiện và tránh vật cản trong môi trường đã được xây dựng bản đồ trước đó. Các vật cản được bố trí bổ sung sau khi quá trình xây dựng bản đồ hoàn tất nên không xuất hiện trong dữ liệu bản đồ ban đầu. Các vật cản này được giữ cố định trong suốt quá trình thử nghiệm nhằm mô phỏng những thay đổi có thể phát sinh trong môi trường thực tế sau khi robot đã hoàn thành quá trình xây dựng bản đồ.



a. Vật cản mới tại vị trí phòng khách



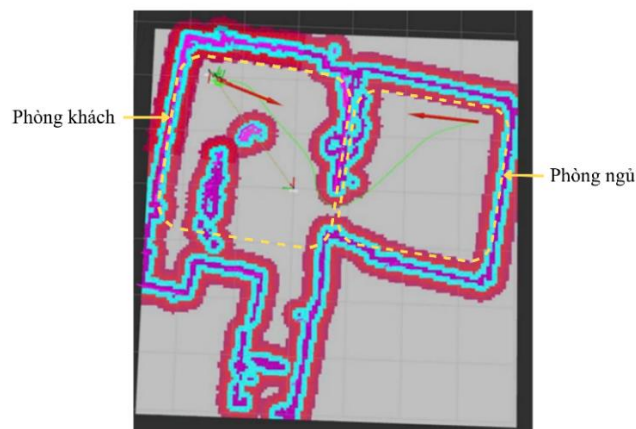
b. Vật cản mới tại vị trí phòng ngủ

Hình 4.16 Vật cản mới được thêm vào bản đồ

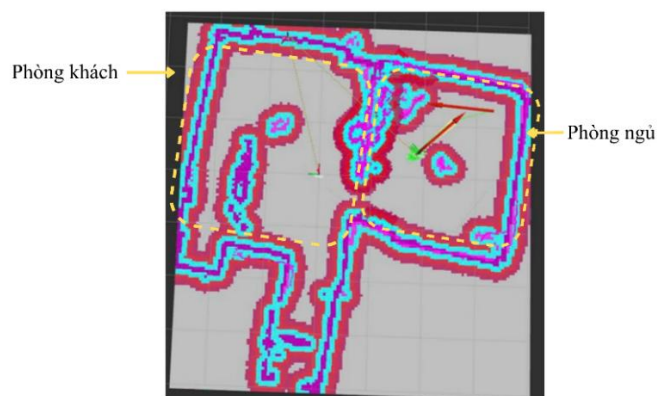
Mục đích của thí nghiệm là kiểm tra khả năng phản ứng của hệ thống điều hướng khi xuất hiện các chướng ngại vật không có trong bản đồ tĩnh, đồng thời đánh giá hiệu quả của cơ chế lập kế hoạch đường đi kết hợp tránh vật cản trong

thời gian thực. Kết quả thực nghiệm cho phép đánh giá mức độ tin cậy và khả năng thích ứng của hệ thống robot tự hành khi hoạt động trong môi trường có sự thay đổi so với dữ liệu bản đồ ban đầu.

Bài thực nghiệm được thực hiện với quỹ đạo di chuyển từ Start đến Goal 3. Hai vật cản tĩnh được bổ sung trên quỹ đạo ban đầu của robot sau khi quá trình xây dựng bản đồ hoàn tất, do đó các vật cản này không xuất hiện trong dữ liệu bản đồ được sử dụng cho quá trình lập kế hoạch đường đi. Thí nghiệm nhằm đánh giá khả năng phát hiện, tránh vật cản và tái lập kế hoạch quỹ đạo của hệ thống khi gặp các chướng ngại vật không xác định trước trong môi trường hoạt động. Đây là điều kiện thường gặp trong thực tế khi môi trường có sự thay đổi sau khi robot đã hoàn thành quá trình xây dựng bản đồ.



a. Quỹ đạo robot khi phát hiện vật cản tại phòng khách



b. Quỹ đạo robot khi phát hiện vật cản tại phòng ngủ

Hình 4.17 Quỹ đạo robot khi phát hiện vật cản mới

Theo kết quả thể hiện trong hình 4.17, robot đã phát hiện được các vật cản mới không xuất hiện trong dữ liệu bản đồ ban đầu và thực hiện tái lập kế hoạch đường đi để tiếp tục di chuyển đến vị trí đích. Quá trình này diễn ra tự động thông qua cơ chế cập nhật môi trường cục bộ của hệ thống điều hướng, cho phép robot phản ứng kịp thời khi xuất hiện chướng ngại vật trên quỹ đạo đã được lập kế hoạch trước đó. Trong quá trình thực nghiệm, robot vẫn duy trì khả năng tránh va chạm và hoàn thành nhiệm vụ di chuyển đến mục tiêu. Tuy nhiên, tại một số thời điểm khi tiếp cận gần vật cản, robot có xu hướng giảm tốc hoặc thay đổi hướng đột ngột. Hiện tượng này chủ yếu xuất phát từ ảnh hưởng của costmap cục bộ và các tham số tránh vật cản được thiết lập trong hệ thống điều hướng.

Nhìn chung, kết quả thực nghiệm cho thấy hệ thống có khả năng phát hiện vật cản, cập nhật môi trường và tái lập kế hoạch đường đi trong thời gian thực. Điều này góp phần bảo đảm khả năng vận hành an toàn và nâng cao tính thích ứng của robot tự hành khi môi trường hoạt động có sự thay đổi so với dữ liệu bản đồ ban đầu.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 KẾT LUẬN

Qua toàn bộ quá trình nghiên cứu và triển khai, nhóm đã hiện thực hóa thành công một nền tảng robot tự hành có khả năng hoạt động trong môi trường trong nhà. Từ giai đoạn lên ý tưởng, thiết kế cơ khí, tích hợp phần cứng cho đến phát triển và hiệu chỉnh phần mềm, mỗi bước đều đòi hỏi sự thử nghiệm và điều chỉnh liên tục để hệ thống đạt được sự ổn định trong thực tế vận hành. Kết quả thử nghiệm cho thấy robot có khả năng xây dựng bản đồ, định vị và di chuyển tự động đến các vị trí mục tiêu trong môi trường quen thuộc. Hệ thống vận hành ổn định khi sử dụng cấu hình phù hợp, thể hiện sự tương thích giữa mô hình thiết kế và hành vi thực tế của robot trên phần cứng.

Để đánh giá hiệu năng điều hướng, nhóm tiến hành thử nghiệm với ba vị trí đích khác nhau, mỗi vị trí được thực hiện lặp lại ba lần và sai số vị trí dừng được đo trực tiếp bằng thước. Kết quả cho thấy sai số dao động trong khoảng cho phép. Bên cạnh đó, đề tài đã nghiên cứu và đánh giá hiệu quả của thuật toán WA^* so với A^* truyền thống thông qua các tiêu chí số lượng nút được duyệt, thời gian lập kế hoạch và chiều dài quỹ đạo. Kết quả cho thấy việc bổ sung trọng số W giúp giảm đáng kể số nút mở rộng và rút ngắn thời gian tìm kiếm, tuy nhiên khi W quá lớn thì quỹ đạo tạo ra có xu hướng dài hơn, phản ánh rõ sự đánh đổi giữa tốc độ và tính tối ưu của nghiệm. Nhìn chung, hệ thống thể hiện khả năng điều hướng tương đối ổn định trong điều kiện thực tế trong nhà, tuy nhiên độ chính xác vẫn chịu ảnh hưởng bởi các yếu tố như bề mặt sàn không đồng đều, hiện tượng trượt bánh xe và giới hạn của cảm biến. Những kết quả đạt được không chỉ cho thấy đề án đã hoàn thành các mục tiêu đề ra mà còn giúp nhóm tích lũy nhiều kiến thức và kinh nghiệm thực tiễn về thiết kế hệ thống nhúng, lập trình robot, xử lý dữ liệu cảm biến và triển khai các thuật toán điều hướng trên nền tảng ROS 2, tạo nền tảng cho các nghiên cứu và cải tiến trong tương lai.

5.2 HƯỚNG PHÁT TRIỂN

Dựa trên những kết quả đạt được cũng như các hạn chế còn tồn tại trong quá trình thực nghiệm, đề tài đề xuất một số hướng phát triển nhằm nâng cao hiệu năng và mở rộng phạm vi ứng dụng của hệ thống trong tương lai. Trước hết, hệ thống định vị cần được tiếp tục cải thiện thông qua việc hiệu chỉnh chính xác hơn mô hình động học của robot và nghiên cứu tích hợp các thuật toán định vị nâng cao nhằm giảm sai số và tăng độ tin cậy khi hoạt động trong các môi trường phức tạp. Bên cạnh đó, khả năng cảm nhận môi trường có thể được nâng cao bằng cách bổ sung các cảm biến như camera độ sâu, cảm biến siêu âm hoặc IMU để tăng cường khả năng nhận thức không gian và xử lý các tình huống có nhiều nhiễu. Việc tối ưu hóa năng lượng cũng là một hướng nghiên cứu quan trọng nhằm kéo dài thời gian hoạt động của hệ thống khi tích hợp thêm các thiết bị ngoại vi có mức tiêu thụ điện năng cao. Ngoài ra, việc phát triển các cơ cấu phần cứng mở rộng như khay chứa, tay gắp hoặc cơ cấu vận chuyển sẽ giúp robot có tính ứng dụng thực tế cao hơn trong các bài toán vận chuyển và tương tác với môi trường. Song song đó, một giao diện giám sát trực quan hiển thị trạng thái pin, vị trí hiện tại và bản đồ môi trường cũng có thể được xây dựng để nâng cao khả năng theo dõi và quản lý hệ thống. Với kiến trúc mở và khả năng mở rộng linh hoạt, nền tảng robot được xây dựng trong đề tài có tiềm năng trở thành công cụ phục vụ đào tạo, nghiên cứu và phát triển các ứng dụng trong lĩnh vực robot học, điều khiển tự động, trí tuệ nhân tạo và hệ thống nhúng trong tương lai.

PHỤ LỤC

PHỤ LỤC: MÃ NGUỒN CỦA HỆ THỐNG

Mã nguồn chương trình firmware của mạch điều khiển

Để có thể sử dụng mã nguồn của mạch điều khiển, truy cập vào đường dẫn sau:
<https://github.com/QDung888/Autonomous-Robot>

Chương trình firmware sử dụng nền tảng ESP-IDF phiên bản 6.0, đối với dòng vi điều khiển ESP32-S3.

TÀI LIỆU THAM KHẢO

- [1] Morgan Quigley, Brian Gerkey and William D. Smart, Programming Robots with ROS: A Practical Introduction to the Robot Operating System. Sebastopol, CA, USA: O'Reilly Media, 2015.
- [2] S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, Intelligent Robotics and Autonomous Agents. Cambridge, MA, USA: MIT Press, 2005.
- [3] Nguyễn Thị Phương Hà, Huỳnh Thái Hoàng, *Lý thuyết điều khiển tự động*, tái bản lần 2. Tp. Hồ Chí Minh, Việt Nam: NXB Đại học Quốc gia Tp.HCM, 2005.
- [4] D. L. Poole and A. K. Mackworth, *Artificial Intelligence: Foundations of Computational Agents*, 3rd ed. Cambridge, U.K.: Cambridge University Press, 2023. [Online]. Available: <https://www.artint.info/3e/html/ArtInt3e.Ch3.S6.html>
- [5] Phan Hoàng Anh, Nguyễn Thị Thanh Vân, Trần Hữu Quốc Đông, Bùi Duy Nam và Vũ Tiến Đạt, “Nghiên cứu phát triển robot xây dựng bản đồ và định vị đồng thời trên nền tảng ROS,” trong Hội nghị Quốc gia lần thứ XXIII về Điện tử, Truyền thông và Công nghệ Thông tin (REV-ECIT 2020), Việt Nam, 2020.
- [6] Nguyễn Anh Tú, Vũ Công Thành, Nguyễn Trọng Hải, Nguyễn Trọng Duy, Hồ Văn Hoàng và Mai Duy Quang, “Nghiên cứu, thiết kế hệ thống định vị cho robot di động trên hệ điều hành ROS,” Tạp chí Khoa học và Công nghệ Công nghiệp, số 31, tr. 69-75, 2021.
- [7] Phạm Thị Lan Hương và Nguyễn Thị Duyên, “Xây dựng quỹ đạo cục bộ cho robot tự hành trong nhà lưới nông nghiệp trên nền tảng hệ điều hành ROS,” Tạp chí Khoa học Nông nghiệp Việt Nam, tập 21, số 10, tr. 1282-1293, 2023.
- [8] Shengmin Zhao and Seung-Hoon Hwang, “ROS-Based Autonomous Navigation Robot Platform with Stepping Motor,” Sensors, vol. 23, no. 7, pp. 3648, Mar. 2023.
- [9] Jianwei Zhao, Shengyi Liu and Jinyu Li, “Research and Implementation of Autonomous Navigation for Mobile Robots Based on SLAM Algorithm under ROS,” Sensors, vol. 22, no. 11, pp. 4172, May 2022.
- [10] H. Durrant-Whyte and T. Bailey, “Simultaneous Localization and Mapping: Part I,” IEEE Robotics & Automation Magazine, vol. 13, no. 2, pp. 99–110, 2006.
- [11] Jie Yu, Toan Tan Nguyen, Thi-Kien Dao, and Trong-The Nguyen, “Global path planning algorithm for mobile robots: A review,” Journal of Network Intelligence, vol. 9, no. 2, pp. 643-657, May 2024.

- [12] Peter E. Hart, Nils John Nilsson, and Bertram Raphael, “A formal basis for the heuristic determination of minimum cost paths,” IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100-107, Jul. 1968. DOI: 10.1109/TSSC.1968.300136.
- [13] Rina Dechter and Judea Pearl, “Generalized best-first search strategies and the optimality of A*,” Journal of the ACM, vol. 32, no. 3, pp. 505-536, Jul. 1985. DOI: 10.1145/3828.3830.
- [14] Rüdiger Ebendt and Rolf Drechsler, “Weighted A* search - unifying view and application,” Artificial Intelligence, vol. 173, no. 14, pp. 1310-1342, Sep. 2009. DOI: 10.1016/j.artint.2009.06.004.
- [15] M. Filipenko and I. Afanasyev, “Comparison of Various SLAM Systems for Mobile Robot in an Indoor Environment,” arXiv preprint arXiv:2501.09490, 2025.
- [16] Mobile Industrial Robots, “AMR vs AGV: Key Differences Explained,” [Online]. Available:
<https://mobile-industrial-robots.com/blog/agv-vs-amr-whats-the-difference>
- [17] R. Keith and H. M. La, “Review of Autonomous Mobile Robots for the Warehouse Environment,” arXiv preprint arXiv:2406.08333, Jun. 2024. [Online]. Available:
<https://arxiv.org/abs/2406.08333>
- [18] Carnegie Mellon University. *Lecture 2-3: Robot pose - representation and transformation* [Online]. Available:
<https://web2.qatar.cmu.edu/~gdicaro/16311-Fall17/slides/16311-2-3-Pose-representation-and-transformation.pdf>
- [19] ROS 2 Control. *diff_drive_controller* [Online]. Available:
https://control.ros.org/humble/doc/ros2_controllers/diff_drive_controller/doc/userdoc.html
- [20] ROS 2 Control. *Wheeled Mobile Robot Kinematics* [Online]. Available:
https://control.ros.org/humble/doc/ros2_controllers/doc/mobile_robot_kinematics.html#mobile-robot-kinematics
- [21] ROS 2 Documentation. *ROS 2 Documentation: Humble documentation* [Online]. Available:
<https://docs.ros.org/en/humble/index.html>
- [22] Nav2 Documentation. *Nav2 Documentation* [Online]. Available:
<https://docs.nav2.org/>

